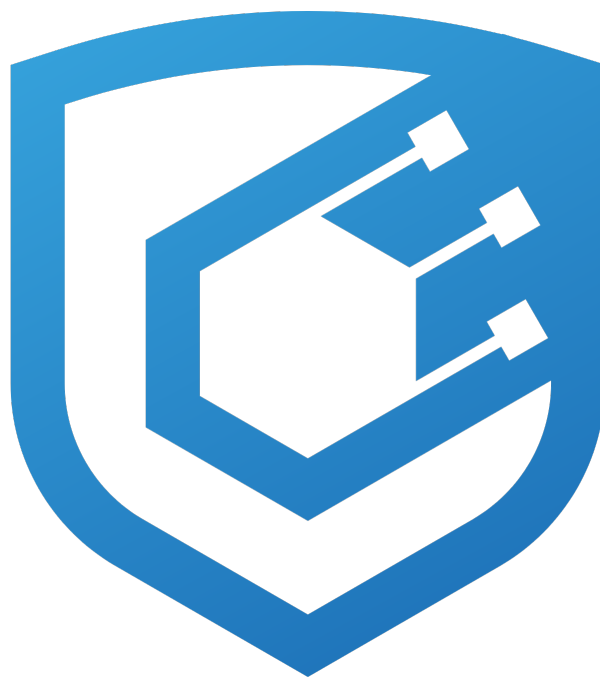




Note Systems Front-End Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Raiders](#)

October 2, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
5	Audit Scope	4
6	Executive Summary	4
6.1	Trust Assumptions and Centralization Risks	5
6.1.1	Roles and Administrative Capabilities	5
6.1.2	Centralization Disclosures	5
6.1.3	External Trust Dependencies	6
7	Findings	10
7.1	Medium Risk	10
7.1.1	Proposal action rows hide the arguments of calls outside ACTION_ABI and show decoded arguments shortened or unsanitised	10
7.1.2	Unwound series show withdraw available but the Portfolio has no action that sends withdrawUnwound	11
7.1.3	Redeem sheet counts unclaimed coupons that redeem does not pay, and the Portfolio then drops the row that holds them	12
7.1.4	Governance log scans read deployBlock to latest in one eth_getLogs call and show an empty proposal list when the RPC rejects it	13
7.1.5	COUPON leg bond markets use 18 decimals for a 6 decimal leg unit, so normal amounts are refused and the only accepted amounts show a payment of 0.00	15
7.1.6	Proposals are identified by proposalId alone, so an identical proposal on the other governor receives the votes cast on the original	16
7.1.7	Votes are cast on the proposalId from the ProposalCreated log without checking that it hashes the displayed actions and description	17
7.2	Low Risk	20
7.2.1	A rejected or failed operator revoke in the COUPON-leg bond flow reports a completed purchase as failed and hides the remaining grant	20
7.2.2	Governance pages show a hard-coded 48 hour Note timelock delay instead of the live TimeLock.getMinDelay	21
7.2.3	Stale feed fallback is unlabelled, and before the strike it renders a 0.00 price and an empty SHIELD book	23
7.2.4	Date formatters throw RangeError past year 275760, so a bond market with a uint48 max conclusion blanks the app	24
7.2.5	Approve-then-act flows simulate the final call only after the approval is signed	25
7.2.6	A failed Desk paused() read is coerced to false and opens the deposit form while the Desk is paused	27
7.2.7	Failed RPC reads render as zero or empty results on the Notes, Series, Portfolio and Governance pages	29
7.2.8	Deposit confirm sheets state the maturity breach payoff at the latest print for the whole deposit	32
7.2.9	Subscription progress adds COUPON deposits and SHIELD notional and compares the sum with a per side notional cap	33
7.2.10	Notes and Portfolio print different distance to barrier figures for the same series and colour them on different scales	34
7.2.11	Analytics series table labels the per observation coupon as "Coupon p.a."	36
7.2.12	Desk withdraw card compares the amount with idle assets instead of the idle left after queue reservations	36
7.2.13	runSteps assumes an EOA signer, so the offered Safe connector cannot complete a write	39

7.2.14	A route chunk that fails to load unmounts the whole app because no error boundary wraps the lazy routes	41
7.2.15	ConfirmSheet re-runs its focus effect on every parent render and moves keyboard focus back to Cancel each second	43
7.2.16	runSteps accepts the receipt of a cancelling transaction as confirmation of the step it replaced	44
7.2.17	Stock Token bond markets store the 6 decimal Treasury.valueOf result as a WAD, so the token value renders 1e12 too small	45
7.2.18	Two flows from one wallet can both pass simulation against the same allowance and the second reverts on chain	46
7.2.19	Series pages never render a deferred observation because the app does not read the oracle adapter state	48
7.2.20	Lock form rounds the unlock down to a Thursday boundary, so a 1 week lock can end within seconds and cannot vote on gauges	49
7.2.21	The bundle defines no Buffer global, so every transaction from a Coinbase Wallet mobile app session fails before it reaches the wallet	50
7.2.22	describeError reads the calldata printed in a viem error message as revert data, so a wallet on another network is reported as a contract revert	51
7.2.23	Effective strike close is discarded because viem decodes uint40 as a number, so calendar changes never reach the strike date or deposit deadline	52
7.2.24	Lock figures add stakedNoteEq to veNOTE LockedBalance.amount, which already includes it, so the sNOTE part of every lock is counted twice	53
7.2.25	Buttons outside ConfirmSheet skip the preview gate, so a ?preview=1 link sends real transactions built from mock figures	54
7.2.26	rollAll and rollRange are sent after an empty simulation, and the toast lists the stale preview's series as opened	55
7.2.27	Stake APY keeps annualising the last sNOTE reward stream after it has ended and labels it as trailing 30 d rewards	57
7.2.28	note.systems lacks DNSSEC, a CAA record, a registrar update lock and HSTS preload, so a DNS or first-visit attacker can serve a different app	58
7.2.29	note.systems mail policy does not reject spoofed mail, and its MX publishes no MTA-STS policy	59
7.3	Informational	60
7.3.1	Review sheets hard-code the Network row as Robinhood Chain on the testnet build	60
7.3.2	Live routes render mock-provider copy and the ?preview flag gives a false reason for disabled transactions	61
7.3.3	App implements eight presentation rules differently from its specs	62
7.3.4	Failure toast does not say that approvals confirmed earlier in the flow remain on chain	63
7.3.5	Portfolio Remaining column prints a minus zero and the full prefund as coming back for SHIELD positions before strike	64
7.3.6	npm test suite does not check what the step builders ask the wallet to sign	65
7.3.7	Amount forms enable Review from Number(amt), so input finer than the token decimals encodes a zero amount	66
7.3.8	Both viem clients use one rate limited public RPC endpoint with no fallback transport	67
7.3.9	Reference price, market status and the Governance core label read bundled addresses instead of the modules NoteCore and SeriesRegistry resolve	68
7.3.10	CSP connect-src omits https://www.walletlink.org, so the Coinbase Wallet SDK cannot fetch WalletLink responses missed while its socket was down	70
7.3.11	Six UI strings and figures contradict what the deployed contracts and the wallet SDK do	71
7.3.12	CI app job runs the ABI check without compiled contracts, so the check passes without comparing anything	72
7.3.13	App hard-codes 6 decimals for the quote token instead of reading NoteCore.quoteDecimals	73
7.3.14	Access gate covers only /app/, so the analytics page stays public when the gate is switched on	74

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Note Systems issues autocallable barrier notes on tokenised US equities on Robinhood Chain. Each series splits into two ERC-1155 legs. COUPON holders deposit USDG and earn a coupon at each observation while the stock stays above the barrier. SHIELD holders deposit Stock Tokens and prefund the coupons to buy principal protection; if the barrier is breached at maturity, their stock goes to the COUPON side. Around the notes sit a USDG vault (the Desk), NOTE and its staking vault sNOTE funded by a fee buyback, a Treasury, a bond depository, and veNOTE governance with two governors: one for parameters behind a 48 hour timelock and one for upgrades behind a 7 day timelock.

The web application is a static single-page app with no backend. It reads all protocol state from one public RPC and builds every transaction the user signs. It supports injected wallets, Coinbase Wallet and a Safe connector, and ships with a public analytics page and an optional access gate. The network is fixed at build time: the testnet build targets Robinhood Chain Testnet, and the mainnet build has no protocol addresses yet and runs in preview mode.

5 Audit Scope

The review covered the web application under `web/` at commit `cfe6d34eb5c081edb9458f836a3d6e03ab31f779` (`web/` is identical at `c0e9b01`):

Path	Contents	Lines
<code>web/app/src/</code>	Routes, components, ABIs, address manifest, transaction builders, data providers, wallet layer	12,529
<code>web/app/analytics/</code>	Analytics page	1,680
<code>web/app/qa/</code>	QA harnesses and ABI check	1,670
<code>web/app/public/gate/</code> , <code>web/shared/</code>	Access gate (client side) and wallet catalogue	340
<code>web/app/</code>	Build configuration and entry HTML	355
Total		16,574

The smart contracts were read only to confirm what the app's inputs do on chain; no finding is a contract defect. At the client's request, the review also covered the `note.systems` domain, DNS and mail configuration.

The review focused on:

- transactions whose effect differs from what the app shows before signing
- displayed figures that are wrong, stale, or a failed read shown as a value
- flows that strand approvals, misreport outcomes or break when the wallet, network or RPC changes
- governance views that hide what a proposal executes or where a vote lands
- build, hosting and domain weaknesses that let a third party change the served app

Findings were reproduced on Robinhood Chain Testnet, on local forks of it, and on production builds of the reviewed commit.

Limitations:

- The server-side half of the access gate (the `.htaccess` rewrite written by `web/qa/assemble.sh`) is not in the repository and was not reviewed. It can be reviewed separately on request.
- The live site is built from a client-internal commit and includes changes that are not part of this review. The findings apply to `cfe6d34`.
- Wallet behaviour was tested with stub wallets that follow each wallet's documented behaviour, not with real wallet extensions.

6 Executive Summary

Over the course of 10 days, the Cyfrin team conducted an audit on the [Front-End](#) code provided by [Note Systems](#). The findings consist of 7 Medium & 29 Low severity issues with the remainder being informational.

No Critical or High issues were found. The Medium issues concern governance pages that do not show voters what a proposal will do or where their vote goes, and position pages that miss funds users are owed. Most Low issues come from failed reads shown as real values, multi-step transaction flows that handle failures poorly, and errors in displayed figures. The domain and mail setup also lack standard protections against hijacking and spoofing.

Note Systems provided a trust model, a list of known issues and accepted risks, per-contract notes and app specifications. Items already covered there are not repeated in this report.

6.1 Trust Assumptions and Centralization Risks

The app has no backend. What it shows comes from one RPC, and what it asks users to sign targets addresses compiled into the bundle. The points below cover who can change those inputs.

6.1.1 Roles and Administrative Capabilities

Role	Production	Testnet deployment
Owner of every protocol module	Timelock, 48 h delay	Timelock, 0 s delay
Upgrade authority for Desk, Treasury, BondDepository, RevenueRouter	UpgradeTimelock, 7 d delay	UpgradeTimelock, 7 d delay
Proposers	NoteGovernor and UpgradeGovernor	The governors and the deployer EOA
Guardian (pause, cancel)	Security Council	Deployer EOA
Price feeds	Chainlink-style aggregators with a Pyth Pro backup	Mock feeds updated by the deployer EOA

Governance can change fees, caps, rewards and other parameters, create and unwind series, and swap the oracle, calendar and fee sink modules after a 24 hour grace. Series parameters are fixed at creation. The app reads current values from chain, except the timelock delays it prints as constants. It does not follow module swaps: it keeps using the addresses it was built with until it is rebuilt.

An upgrade replaces the code behind a proxy the user may have approved. The app approves exact amounts that the same flow spends, so no allowance normally remains, unless a revoke step fails. The guardian can pause but not unpauses, and user exits are never pausable.

On testnet the deployer EOA can schedule and execute any owner action immediately, acts as guardian, and updates the price feeds. The Governance page still describes the guardian as unable to propose, execute or move funds.

6.1.2 Centralization Disclosures

- **Hosting.** The registrar, DNS, CDN hosting and mail for `note.systems` sit in one Hostinger account. Whoever controls it can change the app users load and send mail as the domain.
- **Builds.** The served bundle is built outside CI and uploaded by FTP. Users cannot check that what they load matches reviewed code: there is no published bundle hash or signed release, and the wallet prompt is the only independent view of what is signed. The strict CSP limits third-party script injection but not the host operator.
- **Access gate.** The gate is off. When on, it covers `/app/` only, and its enforcing rule lives in host configuration outside the repository.

6.1.3 External Trust Dependencies

- **RPC.** Each build uses one public endpoint with no fallback. The RPC operator controls every figure the app shows, the quotes it turns into calldata, the simulation result and the receipts. It cannot redirect an approval spender or recipient, which come from the bundle.
- **Wallet SDKs.** The Coinbase Wallet SDK is bundled and talks to Coinbase and WalletLink servers; the Safe connector accepts messages only from `app.safe.global` origins. The wallet vendor controls the signing prompt. Dependencies are locked with integrity hashes, and the build fails on high-severity `npm audit` advisories.
- **Chain.** Robinhood Chain's sequencer orders and can delay transactions. The app uses the browser clock for deadlines, so a wrong local clock can make a time-bound transaction revert.

Summary

Project Name	Front-End
Repository	notesystems-project
Commit	cfe6d34eb5c0...
Fix Commit	a285a1c00b4f...
Audit Timeline	Sep 17th - Sep 30th, 2026
Methods	Manual Review

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	7	7	0
Low Risk	29	27	2
Informational	14	13	1
Gas Optimizations	0	0	0
Total	50	47	3

Summary of Findings

[M-1] Proposal action rows hide the arguments of calls outside <code>ACTION_ABI</code> and show decoded arguments shortened or unsanitised	Resolved
[M-2] Unwound series show withdraw available but the Portfolio has no action that sends <code>withdrawUnwound</code>	Resolved
[M-3] Redeem sheet counts unclaimed coupons that <code>redeem</code> does not pay, and the Portfolio then drops the row that holds them	Resolved

[M-4] Governance log scans read <code>deployBlock</code> to latest in one <code>eth_getLogs</code> call and show an empty proposal list when the RPC rejects it	Resolved
[M-5] COUPON leg bond markets use 18 decimals for a 6 decimal leg unit, so normal amounts are refused and the only accepted amounts show a payment of 0.00	Resolved
[M-6] Proposals are identified by <code>proposalId</code> alone, so an identical proposal on the other governor receives the votes cast on the original	Resolved
[M-7] Votes are cast on the <code>proposalId</code> from the <code>ProposalCreated</code> log without checking that it hashes the displayed actions and description	Resolved
[L-01] A rejected or failed operator revoke in the COUPON-leg bond flow reports a completed purchase as failed and hides the remaining grant	Resolved
[L-02] Governance pages show a hard-coded 48 hour Note timelock delay instead of the live <code>Timelock.getMinDelay</code>	Resolved
[L-03] Stale feed fallback is unlabelled, and before the strike it renders a 0.00 price and an empty SHIELD book	Resolved
[L-04] Date formatters throw <code>RangeError</code> past year 275760, so a bond market with a <code>uint48</code> max conclusion blanks the app	Resolved
[L-05] Approve-then-act flows simulate the final call only after the approval is signed	Resolved
[L-06] A failed <code>Desk.paused()</code> read is coerced to <code>false</code> and opens the deposit form while the Desk is paused	Resolved
[L-07] Failed RPC reads render as zero or empty results on the Notes, Series, Portfolio and Governance pages	Resolved
[L-08] Deposit confirm sheets state the maturity breach payoff at the latest print for the whole deposit	Resolved
[L-09] Subscription progress adds COUPON deposits and SHIELD notional and compares the sum with a per side notional cap	Resolved
[L-10] Notes and Portfolio print different distance to barrier figures for the same series and colour them on different scales	Resolved
[L-11] Analytics series table labels the per observation coupon as "Coupon p.a."	Resolved
[L-12] Desk withdraw card compares the amount with idle assets instead of the idle left after queue reservations	Resolved
[L-13] <code>runSteps</code> assumes an EOA signer, so the offered Safe connector cannot complete a write	Resolved
[L-14] A route chunk that fails to load unmounts the whole app because no error boundary wraps the lazy routes	Resolved
[L-15] <code>ConfirmSheet</code> re-runs its focus effect on every parent render and moves keyboard focus back to Cancel each second	Resolved
[L-16] <code>runSteps</code> accepts the receipt of a cancelling transaction as confirmation of the step it replaced	Resolved
[L-17] Stock Token bond markets store the 6 decimal <code>Treasury.valueOf</code> result as a WAD, so the token value renders 1e12 too small	Resolved

[L-18] Two flows from one wallet can both pass simulation against the same allowance and the second reverts on chain	Resolved
[L-19] Series pages never render a deferred observation because the app does not read the oracle adapter state	Resolved
[L-20] Lock form rounds the unlock down to a Thursday boundary, so a 1 week lock can end within seconds and cannot vote on gauges	Resolved
[L-21] The bundle defines no <code>Buffer</code> global, so every transaction from a Coinbase Wallet mobile app session fails before it reaches the wallet	Resolved
[L-22] <code>describeError</code> reads the calldata printed in a viem error message as revert data, so a wallet on another network is reported as a contract revert	Resolved
[L-23] Effective strike close is discarded because viem decodes <code>uint40</code> as a number, so calendar changes never reach the strike date or deposit deadline	Resolved
[L-24] Lock figures add <code>stakedNoteEq</code> to <code>veNOTE LockedBalance.amount</code> , which already includes it, so the sNOTE part of every lock is counted twice	Resolved
[L-25] Buttons outside <code>ConfirmSheet</code> skip the preview gate, so a <code>?preview=1</code> link sends real transactions built from mock figures	Resolved
[L-26] <code>rollAll</code> and <code>rollRange</code> are sent after an empty simulation, and the toast lists the stale preview's series as opened	Resolved
[L-27] Stake APY keeps annualising the last sNOTE reward stream after it has ended and labels it as trailing 30 d rewards	Resolved
[L-28] <code>note.systems</code> lacks DNSSEC, a CAA record, a registrar update lock and HSTS preload, so a DNS or first-visit attacker can serve a different app	Acknowledged
[L-29] <code>note.systems</code> mail policy does not reject spoofed mail, and its MX publishes no MTA-STS policy	Acknowledged
[I-01] Review sheets hard-code the Network row as Robinhood Chain on the testnet build	Resolved
[I-02] Live routes render mock-provider copy and the <code>?preview</code> flag gives a false reason for disabled transactions	Resolved
[I-03] App implements eight presentation rules differently from its specs	Resolved
[I-04] Failure toast does not say that approvals confirmed earlier in the flow remain on chain	Resolved
[I-05] Portfolio Remaining column prints a minus zero and the full prefund as coming back for SHIELD positions before strike	Resolved
[I-06] <code>npm test</code> suite does not check what the step builders ask the wallet to sign	Resolved
[I-07] Amount forms enable Review from <code>Number(amt)</code> , so input finer than the token decimals encodes a zero amount	Resolved
[I-08] Both viem clients use one rate limited public RPC endpoint with no fallback transport	Resolved
[I-09] Reference price, market status and the Governance core label read bundled addresses instead of the modules <code>NoteCore</code> and <code>SeriesRegistry</code> resolve	Resolved

[I-10] CSP connect-src omits https://www.walletlink.org , so the Coinbase Wallet SDK cannot fetch WalletLink responses missed while its socket was down	Resolved
[I-11] Six UI strings and figures contradict what the deployed contracts and the wallet SDK do	Resolved
[I-12] CI app job runs the ABI check without compiled contracts, so the check passes without comparing anything	Resolved
[I-13] App hard-codes 6 decimals for the quote token instead of reading <code>NoteCore.quoteDecimals</code>	Resolved
[I-14] Access gate covers only <code>/app/</code> , so the analytics page stays public when the gate is switched on	Acknowledged

7 Findings

7.1 Medium Risk

7.1.1 Proposal action rows hide the arguments of calls outside ACTION_ABI and show decoded arguments shortened or unsanitised

Description: ChainProvider.decodeAction cannot decode a proposal or timelock action whose function is not in ACTION_ABI, and for those it shows only the first 16 bytes of the first argument. ACTION_ABI is the union of the ABIs the app uses for its own reads and writes, so it leaves out the owner-only functions a timelock executes, such as upgradeToAndCall, ERC-20 transfer, Treasury.withdrawERC20, mintNote and setKeeperReward.

```
// web/app/src/data/ChainProvider.ts:870-879
private decodeAction(target: Address, data: Hex, value: bigint): ProposalAction {
  try {
    const d = decodeFunctionData({ abi: ACTION_ABI, data });
    // ...
    return { target, targetName: this.label(target), signature: sig, args: (d.args ?? []).map((x) =>
      ↪ (typeof x === 'string' && x.startsWith('0x') && x.length === 42 ? this.label(x) :
      ↪ String(x))), value };
  } catch {
    return { target, targetName: this.label(target), signature: data.slice(0, 10), args: [data.length
      ↪ > 10 ? `${data.slice(10, 42)}` : ''], value };
  }
}
```

An address is right-aligned in its word, so the 16 bytes are 12 zero bytes and the first 4 bytes of the address, and a small uint256 shows as 32 zeros. Every later argument is dropped, including the payload of upgradeToAndCall and the recipient and amount of withdrawERC20. On testnet, all 91 scheduled timelock operations and the only open proposal take this branch.

The decoded branch has two smaller gaps:

- Addresses outside the address book are shortened by label to their first and last four hex characters, so two recipients that share those characters render the same.
- string arguments go through String(x) without cleanText, so right-to-left overrides and zero-width characters reach the page.

The Actions table on the proposal page prints exactly these fields and is the app's only statement of what a proposal will execute.

Impact: Voters cannot see in the app what a proposal will execute, including contract upgrades, Treasury withdrawals and NOTE mints. A proposer above the 0.5% threshold can describe one action and execute another, and veNOTE holders who rely on the app vote for it. Harm needs that proposal to reach quorum and a majority; the testnet Timelock then has no delay.

Proof of Concept:

1. Open `#/governance/proposals` on the testnet build and open "QA testnet proposal 1789577781: re-affirm NoteCore.keeperRewardQuote (no-op)".
2. The Actions card shows target NoteCore, Call 0x9797fab8 and the argument line 00000000000000000000000000000000....
3. The calldata in the ProposalCreated log is 0x9797fab8...001e8480, which is setKeeperReward(2000000). The value appears nowhere on the page.

Recommended Mitigation: Add the owner-only functions the timelocks call to ACTION_ABI, list every 32-byte word when a selector is still unknown, and render decoded arguments in full: address book name or the whole address, strings through cleanText, arrays and structs expanded. Generating ACTION_ABI from contracts/out stops the list drifting from the contracts.

```

    return { target, targetName: this.label(target), signature: sig, args: (d.args ?? []).map((x) =>
      ↪ this.argText(x)), value };
  } catch {
    return { target, targetName: this.label(target), signature: data.slice(0, 10), args:
      ↪ (data.slice(10).match(/.{1,64}/g) ?? []).map((w) => `0x${w}`), value };
  }
  // ...
private argText(x: unknown): string {
  if (typeof x === 'string' && /^0x[0-9a-fA-F]{40}$/.test(x)) return this.names[lower(x)] ?? x;
  if (typeof x === 'string') return /^0x[0-9a-fA-F]*$/.test(x) ? x : JSON.stringify(cleanText(x,
    ↪ 200));
  if (Array.isArray(x)) return `[${x.map((y) => this.argText(y)).join(', ')}]`;
  if (x !== null && typeof x === 'object') return `{${Object.entries(x).map(([k, v]) => `${k}:
    ↪ ${this.argText(v)}').join(', ')} }`;
  return String(x);
}

```

Note Systems: Fixed in commit [97df402](#).

Cyfrin: Verified.

7.1.2 Unwound series show withdraw available but the Portfolio has no action that sends withdrawUnwound

Description: The app never builds a withdrawUnwound call and computes no payout for an unwound series, so a holder of a series unwound after strike has no button that pays the position. The status pill still says a withdrawal is available (format.ts#L164: 'Unwound · withdraw available').

The Portfolio, the only route that sends NoteCore exits, picks its action from claim, redeem and refund only:

```

// web/app/src/routes/Portfolio.tsx:255
const primary = p.claimableQuote > 0n ? 'claim' : p.redeemableQuote +
  ↪ p.redeemableStock > 0n ? 'redeem' : p.refundableQuote + p.refundableStock > 0n ?
  ↪ 'refund' : null;

```

getPortfolio reads the leg payout only for Autocalled or Matured series, so an unwound position always has redeemableQuote = redeemableStock = 0:

```

// web/app/src/data/ChainProvider.ts:389
const ended = s.status === 'Autocalled' || s.status === 'Matured';

```

For a fully matched position of a series unwound while Live, coupons and refunds are usually zero as well, so primary is null and the row shows only "Detail". On chain, withdrawUnwound(seriesId) pays every leg of the series and cannot be paused.

Impact: When governance unwinds a Live series, its holders cannot recover their principal from the app: neither the Portfolio nor the Series page offers a withdraw. The funds are not lost, since withdrawUnwound works from a block explorer or cast. Series unwound before strike still show a Refund button, and no series is unwound on testnet today.

Proof of Concept:

1. On a testnet fork, pause NoteCore as the guardian, then proposeUnwind(10) and, after the delay, unwind(10) as the Timelock.
2. Connect the holder 0xbd03cb1250513c6f3080e6a72fde32efc105c34f and open /portfolio. The series 10 row shows "Unwound · withdraw available" and only a "Detail" button; the Collect table has no entry for it.
3. Open /series/10: there is no withdraw, redeem or claim control.
4. Call withdrawUnwound(10) from the same wallet. It pays 352.50 USDG and 1.2964 stock.

Recommended Mitigation: Read the leg payout for unwound struck series, and give unwound rows a single Withdraw action that sends `withdrawUnwound`. Its sheet should total every leg the holder has in the series, because one call pays them all.

```
export function withdrawUnwoundSteps(seriesId: number): TxStep[] {
  return [{ label: 'Withdraw unwound', address: A().noteCore, abi: noteCoreAbi, functionName:
    ↪ 'withdrawUnwound', args: [BigInt(seriesId)] }];
}
```

```
-      const ended = s.status === 'Autocalled' || s.status === 'Matured';
+      const ended = s.status === 'Autocalled' || s.status === 'Matured' || (s.status === 'Unwound' &&
    ↪ s.s0 !== null);
```

In `Portfolio.tsx`, add a withdraw action kind and choose it for unwound rows:

```
function unwoundQuote(p: Position): bigint { return p.claimableQuote + p.redeemableQuote +
    ↪ p.refundableQuote; }
function unwoundStock(p: Position): bigint { return p.redeemableStock + p.refundableStock; }

function primaryAction(p: Position, s: Series): ActionKind | null {
  if (s.status === 'Unwound') return unwoundQuote(p) + unwoundStock(p) > 0n ? 'withdraw' : null;
  return p.claimableQuote > 0n ? 'claim' : p.redeemableQuote + p.redeemableStock > 0n ? 'redeem' :
    ↪ p.refundableQuote + p.refundableStock > 0n ? 'refund' : null;
}
```

Note Systems: Fixed in commit [4babc2c](#).

Cyfrin: Verified.

7.1.3 Redeem sheet counts unclaimed coupons that redeem does not pay, and the Portfolio then drops the row that holds them

Description: The Redeem confirm sheet adds the position's unclaimed coupons to what the holder receives, but `redeemSteps` sends a single `redeem`, which pays only principal. After the `redeem`, `getPortfolio` keeps a `COUPON` row only while leg units are held, so the coupons that are still claimable on chain disappear from the Portfolio.

The sheet copy:

```
// web/app/src/routes/Portfolio.tsx:510
plain = <>The series has settled. You redeem your {legName} leg for {/* ... */}{p.claimableQuote >
    ↪ 0n && <>, plus <strong>{fmtUsd(p.claimableQuote)} USD</strong> of unclaimed coupons</>. The
    ↪ leg tokens are burned.</>;
```

The step it sends:

```
// web/app/src/chain/actions.ts:99-101
export function redeemSteps(seriesId: number, leg: 0 | 1, units: bigint): TxStep[] {
  return [{ label: leg === 0 ? 'Redeem COUPON' : 'Redeem SHIELD', address: A().noteCore, abi:
    ↪ noteCoreAbi, functionName: leg === 0 ? 'redeem' : 'redeemShield', args: [BigInt(seriesId), units]
    ↪ }];
}
```

After the `redeem` the units are 0, the position is settled and there is no refund, so the row filter drops the row. It also zeroes `claimableQuote` whenever no units are held:

```
// web/app/src/data/ChainProvider.ts:411-417
if (couponUnits > 0n || (pv && pv.couponDeposit > 0n && (inSub || !pv.settled)) || refundCoupon >
    ↪ 0n) {
  const units = couponUnits > 0n ? couponUnits : pv!.couponDeposit;
  // ...
  claimableQuote: couponUnits > 0n ? accrued : 0n,
```

NoteCore (reference) books accrued coupons into `Position.claimable` on the burn: the leg token calls `onLegTransfer`, which settles the sender's accrual (`contracts/src/core/NoteCore.sol#L1300-L1306`). `_burnAndPay` pays only the leg payout, and only `claim()` pays `claimable`.

Impact: Any COUPON holder of an autocalled or matured series who presses Redeem before Claim is told they receive principal plus their unclaimed coupons, and receives principal only. "Collectable now" lists Coupons (Claim) and Redemption (Redeem) side by side, and Redeem is one click. After it the coupons stay payable through `claim(seriesId)` (never pausable), but the Portfolio no longer lists the series, "Collectable now" drops by the coupon amount and the Series page has no claim control, so a holder using the app cannot see or collect them.

Proof of Concept:

1. Connect a wallet that holds COUPON legs of an autocalled series with unclaimed coupons (on testnet today: `0x7CD5365B4a2d49D1145728cC4025617BCE6b4694`, series 0) and open `/portfolio`.
2. "Collectable now" lists "Coupons 3.40" and "Redemption 100.00" for the series. Press Redeem.
3. The sheet reads "You redeem your COUPON leg for 100.00 USDG, plus 3.40 USDG of unclaimed coupons." The wallet asks to sign `redeem(0, 100000000)`.
4. After mining, the wallet has received 100.00 USDG. The COUPON row and the Coupons entry are gone, also after reload. `claim(0)` from the same wallet still pays 3.40 USDG.

Recommended Mitigation: Keep the COUPON row while `accruedCoupon` is non-zero, even with no units held, and state in the Redeem sheet that redeem does not pay the coupons and that they stay claimable. An alternative is to prepend `claimSteps(seriesId)` to the redeem when `claimableQuote > 0`, at the cost of a second signature.

1. In `ChainProvider.getPortfolio` (`web/app/src/data/ChainProvider.ts`), keep a claim-only COUPON row:

```
const claimOnly = couponUnits === 0n && accrued > 0n;
if (couponUnits > 0n || claimOnly || (pv && pv.couponDeposit > 0n && (inSub || !pv.settled)) ||
    refundCoupon > 0n) {
  const units = couponUnits > 0n || claimOnly ? couponUnits : pv!.couponDeposit;
  positions.push({
    // ...
    claimableQuote: couponUnits > 0n || claimOnly ? accrued : 0n,
```

2. In `ActionSheet` (`web/app/src/routes/Portfolio.tsx`), drop the ", plus ... of unclaimed coupons" clause from the redeem copy and append:

```
{p.claimableQuote > 0n && <> Redeem does not pay your <strong>{fmtUsd(p.claimableQuote)} USDG</strong>
  of unclaimed coupons; they stay claimable with Claim.</>}
```

Note Systems: Fixed in commit [e59f9a8](#).

Cyfrin: Verified.

7.1.4 Governance log scans read `deployBlock` to latest in one `eth_getLogs` call and show an empty proposal list when the RPC rejects it

Description: `ChainProvider.proposals` and `ChainProvider.lockerCount` request every `ProposalCreated` and `veNOTE Deposit` log from `deployBlock` to latest in one `eth_getLogs` call and turn a rejected call into an empty result. The public RPC accepts any block range but rejects a query that matches more than 10,000 logs (logs matched by query exceeds limit of 10000).

```
// web/app/src/data/ChainProvider.ts:811-814
private async proposals(owner: Address | null): Promise<Proposal[]> {
  const govts: Array<[Address, Proposal['governor']]> = [[ADDRESSES.noteGovernor!, 'NoteGovernor'],
    [ADDRESSES.upgradeGovernor!, 'UpgradeGovernor']];
  const created = (await Promise.all(govts.map([a] => getLogs(this.client, { address: a, event:
    evt(governorAbi, 'ProposalCreated'), fromBlock: this.fromBlock, toBlock: 'latest' })).catch(()
    => []))).flat();
```

```
if (created.length === 0) return [];
```

lockerCount does the same with a catch { return 0; }, and the timelockOps, ProposalExecuted and Vote-Cast scans follow the same pattern. getProposal filters the same list, so a failed scan returns null. The pages cannot tell a failed read from an empty chain: the list shows "No proposals match" and the proposal page shows "Proposal not found".

Impact: Anyone holding a governor's proposal threshold (683 veNOTE) can hide all its live proposals from the app by pushing it past the 10,000 ProposalCreated log cap, for about 25 transactions and under 0.01 ETH of gas on testnet. App voters then see no proposals and no error, so they cannot see or vote against them. The veNOTE Deposit scan needs no threshold and reaches the cap unaided in about 240 days.

Proof of Concept:

1. Open `#/governance/proposals` on the testnet build: one proposal is listed.
2. Call `eth_getLogs` on the public RPC for every NoteCore event from `deployBlock` to `latest`. It returns `-32000: logs matched by query exceeds limit of 10000`.
3. On a fork, create more than 10,000 proposals on the Note Governor so its `ProposalCreated` scan returns the same error, and point the app at the fork.
4. Reload the page: it shows "No proposals match", `#/governance` shows "No open proposals", and the first proposal's own URL shows "Proposal not found".

Recommended Mitigation: Read the governance history through a helper that splits a rejected block range in half until each part is accepted. When a range cannot be read even at one block, surface the error on the pages instead of an empty list.

In `web/app/src/data/logPages.ts`:

```
export async function rangeLogs<L>(fetch: (from: bigint, to: bigint) => Promise<L[]>, from: bigint, to:
  ↳ bigint): Promise<L[]> {
  if (to < from) return [];
  try {
    return await fetch(from, to);
  } catch (e) {
    if (from === to) throw e;
    const mid = from + (to - from) / 2n;
    return [...(await rangeLogs(fetch, from, mid)), ...(await rangeLogs(fetch, mid + 1n, to))];
  }
}
```

In `ChainProvider.ts`, route the governance scans through a private `allLogs` built on `rangeLogs`, drop the `.catch(() => [])` on `ProposalCreated`, and record the failure in a new `GovernanceState.proposalsError` field:

```
const created = (await Promise.all(govs.map([a] => this.allLogs({ address: a, event:
  ↳ evt(governorAbi, 'ProposalCreated') }))))).flat();
// ...
this.proposals(owner).catch((e: unknown) => { proposalsError = cleanText(e instanceof Error ?
  ↳ e.message : String(e), 160) || 'log read failed'; return [] as Proposal[]; } ),
```

The governance pages render a "Proposals unavailable" state when `proposalsError` is set, and the proposal page does the same when `useProposal` returns an error.

Note Systems: Fixed in commit [13c4520](#).

Cyfrin: Verified.

7.1.5 COUPON leg bond markets use 18 decimals for a 6 decimal leg unit, so normal amounts are refused and the only accepted amounts show a payment of 0.00

Description: ChainProvider.getBonds treats one COUPON leg unit as 1e18 raw units and gives COUPON leg markets 18 decimals. BondDepository and the app's Position.units use a 6 decimal unit.

```
// web/app/src/data/ChainProvider.ts:634-647
const oneUnit = (m: M) => { const k = kindOf(m); return k === 'USDG' ? 1_000_000n : k === 'STOCK' ?
  ↳ 10n ** BigInt(stockMeta.get(lower(m.quoteToken)).dec ?? 18) : WAD; };
// ...
this.mc(pre.map((x) => c('payoutFor', [BigInt(x.id), oneUnit(x.m!)]))),
// ...
const decimals = kind === 'USDG' ? 6 : kind === 'STOCK' ? (sm?.dec ?? 18) : 18;
```

priceWad becomes payoutFor(id, 1e18), the NOTE payout for 1e12 USDG of face. BondCard multiplies the typed number by that figure for "You receive" and the capacity and debt checks, parses the input with 18 decimals for the calldata, and prints the typed number at 2 decimals as the payment:

```
// web/app/src/routes/Bonds.tsx:154-160
const qDec = m.quoteToken.decimals;
// ...
const n = Number(amt) > 0 ? Number(amt) : 0;
const notePerQuote = toNumber(m.priceWad, 18);
const payout = n * notePerQuote;
const cap = toNumber(m.capacity, 18);
const over = payout > cap;
```

```
// web/app/src/routes/Bonds.tsx:212-215
rows=[
  { k: 'Pay', v: `${fmtNum(n, dp)} ${unitLabel}` },
  { k: 'Receive', v: `${fmtNum(payout, 2)} NOTE` },
]
```

```
// web/app/src/routes/Bonds.tsx:231
tx.send('Bond purchase', () => bondBuySteps(owner, m.id, m.quoteToken.kind, quote,
  ↳ toUnits(amt, qDec), maxPrice, m.quoteToken.symbol)).then((r) => { if (r) setAmt(''); });
```

In the contract, quoteValueWad (contracts/src/token/BondDepository.sol#L518) scales a COUPON leg amount by 10 ** (18 - QUOTE_DECIMALS) with QUOTE_DECIMALS = 6 on testnet, so 1e6 raw units are 1 USDG of face.

Impact: COUPON leg holders cannot bond through the app on 40 of the 42 testnet bond markets: whenever a market accepts legs, the leg amount shown in Portfolio is refused as "Exceeds capacity". The only accepted inputs are below about 6.8e-10 and move real legs, so 0.00000000015 signs a transfer of 150 legs (150 USDG of face) while the sheet says "Pay 0.00 leg units". The holder receives the NOTE the sheet shows (3,000 NOTE), and this path has no leg balance check.

Proof of Concept:

1. On testnet during US market hours, connect a wallet holding COUPON legs of a Live series, for example series 10.
2. Open #/bonds and select the COUPON leg market for that series (market 11).
3. Type 150 in "You pay". "You receive" shows 3,000,000,000,000,000.00 NOTE and the button reads "Exceeds capacity".
4. Type 0.00000000015. The button reads "Review bond". Open the sheet: "Pay 0.00 leg units", "Receive 3,000.00 NOTE".
5. Confirm. The wallet shows setApprovalForAll (when the operator approval is missing), then depositWithLimits(11, 150000000, ...), then the revoke. After the purchase the wallet holds 150 fewer COUPON legs.

Recommended Mitigation: Use the 6 decimal quote unit for COUPON legs in both the payoutFor probe and the market decimals, the same unit as USDG. In ChainProvider.getBonds (web/app/src/data/ChainProvider.ts):

```
-    const oneUnit = (m: M) => { const k = kindOf(m); return k === 'USDG' ? 1_000_000n : k === 'STOCK' ?  
→ 10n ** BigInt(stockMeta.get(lower(m.quoteToken))?.dec ?? 18) : WAD; };  
+    // One COUPON leg unit is 1 USDG of face (1e6 raw): BondDepository.quoteValueWad scales leg amounts  
→ by 10^(18 - QUOTE_DECIMALS).  
+    const oneUnit = (m: M) => (kindOf(m) === 'STOCK' ? 10n **  
→ BigInt(stockMeta.get(lower(m.quoteToken))?.dec ?? 18) : ONE_USDG);  
    // ...  
-    const decimals = kind === 'USDG' ? 6 : kind === 'STOCK' ? (sm?.dec ?? 18) : 18;  
+    const decimals = kind === 'STOCK' ? (sm?.dec ?? 18) : USDG_DECIMALS;
```

Note Systems: Fixed in commit [30f54ca](#).

Cyfrin: Verified.

7.1.6 Proposals are identified by proposalId alone, so an identical proposal on the other governor receives the votes cast on the original

Description: The app treats proposalId as a global key, but OpenZeppelin Governor's hashProposal does not include the governor address, so the same proposal content gets the same id on NoteGovernor and on UpgradeGovernor. Anyone at the proposal threshold can submit a copy of a live proposal to the other governor.

ChainProvider.proposals() builds one row per ProposalCreated log from both governors and matches the wallet's VoteCast logs to every row with that id, whichever governor emitted them:

```
// web/app/src/data/ChainProvider.ts:842  
const mine = myVotes.filter((l) => String((l.args as { proposalId: bigint }).proposalId) ===  
→ id);
```

Every row links to /governance/proposals/<id>, and the detail lookup returns the first match in a newest-first list:

```
// web/app/src/data/ChainProvider.ts:909-912  
async getProposal(id: string, owner: Address | null): Promise<Proposal | null> {  
    const all = await this.proposals(owner);  
    return all.find((p) => p.id === id) ?? null;  
}
```

So both rows open the newer copy, the vote is sent to that copy's governor, and the vote confirm sheet names the title but not the governor. Once the wallet has voted on either copy, both rows show "you voted" and the vote form is hidden.

Impact: An account at the proposal threshold (707 veNOTE on testnet, not spent) can copy a live proposal onto the other governor with one propose call, and every app voter who opens the original then votes on the copy. The original can then miss quorum or lose to Against votes; no funds move. A team resubmitting a proposal to the other governor causes the same result.

Proof of Concept:

1. On a fork of Robinhood Chain testnet, account A proposes NoteGovernor.setProposalThresholdBps(60) on NoteGovernor; account B submits the same content to UpgradeGovernor. Both return the same id.
2. After the voting delay, open #/governance/proposals with a voter wallet. Two rows with the same title, Governor "Note" and "Upgrade", link to the same URL.
3. Click the "Note" row. The page reads "UPGRADE GOVERNOR" and shows proposer B.
4. Vote For and confirm. The wallet sends the transaction to UpgradeGovernor.
5. UpgradeGovernor.hasVoted(id, voter) is true, NoteGovernor.hasVoted(id, voter) is false, and both rows now say "you voted".

Recommended Mitigation: Key every proposal by (governor, proposalId): put the governor in the route and every link, match VoteCast and ProposalExecuted logs by the emitting governor, resolve the page with both values, flag ids that exist on both governors, and name the governor in the vote confirm sheet.

```
export const proposalPath = (p: { governor: GovernorKind; id: string }) =>
↳ `/${governance/proposals}/${GOV_SLUG[p.governor]}/${p.id}`;
```

```
-      const mine = myVotes.filter((l) => String((l.args as { proposalId: bigint }).proposalId) ===
↳ id);
+      const mine = myVotes.filter((l) => lower(l.address) === lower(gov[0]) && String((l.args as {
↳ proposalId: bigint }).proposalId) === id);
```

```
const m = all.filter((p) => p.id === id && (!governor || p.governor === governor));
return m.length === 1 ? m[0] : null;
```

Note Systems: Fixed in commit [acba0d3](#).

Cyfrin: Verified.

7.1.7 Votes are cast on the proposalId from the ProposalCreated log without checking that it hashes the displayed actions and description

Description: ChainProvider.proposals() takes the proposal id and the proposal content (targets, values, call-datas, description) from the same ProposalCreated log and never checks that the id is the hash of that content, so the page can show one proposal while the vote is signed for another. In OpenZeppelin Governor the id is uint256(keccak256(abi.encode(targets, values, calldatas, descriptionHash))), and neither governor overrides it.

```
// web/app/src/data/ChainProvider.ts:838-862
const id = a.proposalId.toString();
const { title, summary, risk } = parseDescription(a.description);
// ...
id, governor: gov[1], title, summary, risk, proposer: a.proposer,
// ...
actions: a.targets.map((t, k) => this.decodeAction(t, a.calldatas[k], a.values[k])),
descriptionHash: keccak256(toBytes(a.description)),
```

The confirm sheet shows p.title, and castVoteSteps(governor, p.id, ...) casts the vote on BigInt(p.id).

For logs the governor emitted, the id always matches the content. A compromised or DNS-hijacked rpc.testnet.chain.robinhood.com, the app's only transport, can keep the id of a real attacker proposal and replace its content with a harmless one. Every other read the vote depends on can stay honest, because they all target the real proposal, and castVote succeeds. A false state, voting window or weight only makes the vote revert. The proposal identity is the one field the contract cannot check for the user, and the app can check it locally with no extra read.

Impact: With a log-tampering RPC, app voters read a harmless proposal and sign votes for the attacker's, such as an UpgradeGovernor call to Treasury.upgradeToAndCall, while the wallet shows only a 76 digit id. The RPC must stay compromised for the 7 day vote, and the passed proposal then waits 7 days in the UpgradeTimeLock, where the guardian can cancel it.

Proof of Concept:

1. Read a real NoteGovernor proposal from the testnet RPC through a transport that rewrites only the proposalId in ProposalCreated logs to the id of an "Upgrade Treasury" proposal. No fork is needed.
2. Apply the id and title logic of proposals() and the calldata of castVoteSteps().
3. The page shows the harmless proposal's title, and the wallet is asked to sign castVote with the id of the attacker's Treasury upgrade.

Save it as web/app/poc-vote-id-binding.mjs and run node poc-vote-id-binding.mjs from web/app:

```

import { createPublicClient, custom, http, parseAbiItem, parseAbi, encodeAbiParameters,
  parseAbiParameters, keccak256, toBytes, encodeFunctionData, decodeFunctionData, decodeAbiParameters
} from 'viem';

const RPC = 'https://rpc.testnet.chain.robinhood.com';
const NOTE_GOVERNOR = '0xe56355758963950deaf96E4B8D0619C98516B382';
const TREASURY = '0xa8B09B999232484Aa286FC3EFcB02Ca81C8291C3';
const PROPOSAL_CREATED = parseAbiItem('event ProposalCreated(uint256 proposalId, address proposer,
  address[] targets, uint256[] values, string[] signatures, bytes[] calldatas, uint256 voteStart,
  uint256 voteEnd, string description)');
const P =
  parseAbiParameters('uint256,address,address[],uint256[],string[],bytes[],uint256,uint256,string');
const hashProposal = (t, v, c, d) => BigInt(keccak256(encodeAbiParameters(parseAbiParameters('address[],
  uint256[], bytes[], bytes32'), [t, v, c, keccak256(toBytes(d))]))));

// The attacker's proposal: upgrade the Treasury to an attacker implementation.
const evil = { t: [TREASURY], v: [0n], c: [encodeFunctionData({ abi: parseAbi(['function
  upgradeToAndCall(address,bytes)']), args: ['0xbad00000000000000000000000000000bad', '0x'] )]],
  d: 'Upgrade Treasury' };
const X = hashProposal(evil.t, evil.v, evil.c, evil.d);

// Tampering RPC: forwards everything, rewrites proposalId inside ProposalCreated logs.
const upstream = http(RPC)({});
const tampering = custom({ async request({ method, params }) {
  const res = await upstream.request({ method, params });
  if (method !== 'eth_getLogs') return res;
  return res.map((l) => { const d = [...decodeAbiParameters(P, l.data)]; d[0] = X; return { ...l, data:
    encodeAbiParameters(P, d) }; });
});

for (const [label, transport] of [['honest RPC', http(RPC)], ['tampering RPC', tampering]]) {
  const client = createPublicClient({ transport });
  const [log] = await client.getLogs({ address: NOTE_GOVERNOR, event: PROPOSAL_CREATED, fromBlock:
    120453725n, toBlock: 120453725n });
  const a = log.args;
  // Copied from web/app/src/data/ChainProvider.ts proposals(): id and title come straight from the log.
  const id = a.proposalId.toString();
  const title = a.description.split('\n')[0];
  // Copied from web/app/src/chain/actions.ts castVoteSteps(): the vote is cast on BigInt(p.id).
  const data = encodeFunctionData({ abi: parseAbi(['function castVote(uint256 proposalId, uint8
    support)']), functionName: 'castVote', args: [BigInt(id), 1] });
  const signedId = decodeFunctionData({ abi: parseAbi(['function castVote(uint256 proposalId, uint8
    support)']), data }).args[0];
  const bound = hashProposal(a.targets, a.values, a.calldatas, a.description) === a.proposalId;
  console.log(`${label}: page shows "${title}"`);
  console.log(` wallet signs castVote(${signedId}, 1)`);
  console.log(` signed id is the attacker "Upgrade Treasury" proposal: ${signedId === X}`);
  console.log(` hashProposal(displayed content) == signed id: ${bound} -> with the fix this proposal
    is ${bound ? 'shown' : 'dropped'}`);
}

```

```

tampering RPC: page shows "QA testnet proposal 1789577781: re-affirm NoteCore.keeperRewardQuote (no-op)"
wallet signs castVote(9203504474633476154502856853402861237931052108697328339597242013992035207050, 1)
signed id is the attacker "Upgrade Treasury" proposal: true

```

Recommended Mitigation: Recompute hashProposal locally from the decoded targets, values, calldatas and description, and drop any ProposalCreated log whose proposalId differs. In web/app/src/data/ChainProvider.ts:

```

const PROPOSAL_ID_PARAMS = parseAbiParameters('address[], uint256[], bytes[], bytes32');

```

```

/** OZ Governor.hashProposal computed locally: uint256(keccak256(abi.encode(targets, values, calldatas,
↳ keccak256(description))))). */
function hashProposal(targets: readonly Address[], values: readonly bigint[], calldatas: readonly
↳ Hex[], description: string): bigint {
  return BigInt(keccak256(encodeAbiParameters(PROPOSAL_ID_PARAMS, [targets, values, calldatas,
↳ keccak256(toBytes(description))])));
}

```

```

-   const items = created.map((l) => ({ log: l, a: l.args as PC, gov: govts.find((a) => lower(a) ===
↳ lower(l.address))! }));
+   const items = created.map((l) => ({ log: l, a: l.args as PC, gov: govts.find((a) => lower(a) ===
↳ lower(l.address))! }));
+   .filter(({ a }) => hashProposal(a.targets, a.values, a.calldatas, a.description) ===
↳ a.proposalId);
+   if (items.length === 0) return [];

```

Note Systems: Fixed in commit [cd5cbf2](#).

Cyfrin: Verified.

7.2 Low Risk

7.2.1 A rejected or failed operator revoke in the COUPON-leg bond flow reports a completed purchase as failed and hides the remaining grant

Description: In the COUPON-leg bond flow the operator revoke is an ordinary step of a fail-fast step list, so when it is rejected or fails, the flow's outcome comes from the revoke instead of the purchase receipt. A failed undo of the grant is dropped silently, so BondDepository stays approved as operator for every NoteLegs token in the wallet and nothing on screen says so.

bondBuySteps grants the operator with an undo, buys, then appends the revoke as a plain step:

```
// web/app/src/chain/actions.ts:209-223
if (kind === 'COUPON_LEG') {
  const op = await ensureOperator(a.noteLegs, owner, a.bondDepository, 'Approve COUPON legs');
  if (op) steps.push({ ...op, undo: revokeOperatorStep(a.noteLegs, a.bondDepository, 'Revoke COUPON
    ↪ legs approval') });
}
// ...
steps.push({ label: 'Buy bond', /* ... */ functionName: 'depositWithLimits', /* ... */ });
if (kind === 'COUPON_LEG') steps.push(revokeOperatorStep(a.noteLegs, a.bondDepository, 'Revoke COUPON
  ↪ legs approval'));
```

runSteps stops at the first step that throws, sends the undo of every confirmed step, discards any error the undo throws, and rethrows:

```
// web/app/src/wallet/tx.ts:237-244
try {
  for (let i = 0; i < steps.length; i++) await exec(i, steps[i]);
} catch (e) {
  for (const s of [...confirmed].reverse()) if (s.undo) await exec(steps.indexOf(s), s.undo).catch(()
    ↪ => undefined);
  throw e;
}
```

useTx.send turns the throw into a "Bond purchase failed" toast, and the Bonds route keeps the amount in the form. This happens when:

- the user rejects the revoke after the purchase mined (a second, identical revoke prompt follows from the undo);
- the wallet switches network, account or disconnects after a confirmed prompt, so the next write and the undo both throw before any prompt.

Impact: A user whose bond purchase settled is told it failed, and a retry buys a second bond. BondDepository also keeps an unmentioned setApprovalForAll grant over every NoteLegs token in the wallet; the current implementation cannot use it, but a future implementation behind the 7 day upgrade timelock would inherit it.

Proof of Concept:

1. With a wallet holding COUPON legs of a Live series, open #/bonds, pick that series' COUPON-leg market, enter 10, and click "Review bond" and "Confirm bond".
2. Confirm "Approve COUPON legs" and "Buy bond" in the wallet, then reject "Revoke COUPON legs approval" and the identical prompt that follows.
3. The app shows "Bond purchase failed · Request rejected in wallet." and the amount is still in the form.
4. On chain bondCount went from 0 to 1 and isApprovedForAll(wallet, BondDepository) is true.

Recommended Mitigation: Take the flow's outcome from the purchase receipt: mark the trailing revoke as a cleanup step whose failure does not fail the flow, and report every cleanup or undo step that did not confirm with an error toast that names the remaining grant and offers a button to send the revoke again.

```
- if (kind === 'COUPON_LEG') steps.push(revokeOperatorStep(a.noteLegs, a.bondDepository, 'Revoke COUPON
↳ legs approval'));
+ if (kind === 'COUPON_LEG') steps.push({ ...revokeOperatorStep(a.noteLegs, a.bondDepository, 'Revoke
↳ COUPON legs approval'), cleanup: true });
```

In `runSteps`, a failed cleanup step or undo is reported as `left` instead of failing the flow or being dropped; `useTx.send` raises the toast for each `left` step:

```
for (let i = 0; i < steps.length; i++) {
  const s = steps[i];
  if (s.cleanup) await exec(i, s).catch(() => onStep?.(i, s, 'left'));
  else await exec(i, s);
}
} catch (e) {
  for (const s of [...confirmed].reverse()) {
    const u = s.undo;
    if (u) await exec(steps.indexOf(s), u).catch(() => onStep?.(steps.indexOf(s), u, 'left'));
  }
  throw e;
}
```

Note Systems: Fixed in commit [b366a0d](#).

Cyfrin: Verified.

7.2.2 Governance pages show a hard-coded 48 hour Note timelock delay instead of the live `Timelock.getMinDelay`

Description: The governance routes print the Note Timelock delay as a literal 48 hours instead of the `Timelock.getMinDelay()` value that `ChainProvider.getGovernance` already reads into `governors[].timelockDelaySeconds` and `TimelockOperation.minDelaySeconds`. The proposal page picks the delay from the governor kind and uses it both for the "Timelock" row and to place the "Queued" step:

```
// web/app/src/routes/GovernanceProposal.tsx:43
const delay = p.governor === 'UpgradeGovernor' ? 7 * 86400 : 48 * 3600;
```

The same figure is written as text in three other places:

- the proposals list footer: "Parameter changes take about 8 days end to end (1 d delay + 5 d vote + 48 h timelock)";
- the governance overview queue header: "Timelock 48 h · UpgradeTimelock 7 d";
- the pill on each queued operation: "Core · 48 h".

The Parameters card on the overview page renders the live value, so the same page can show two different delays. On testnet the Note Timelock has a minimum delay of 0 while the UI says 48 hours. A mainnet Timelock whose delay differs, or is later changed with `updateDelay`, shows the same mismatch.

Impact: A holder who opposes a parameter change, such as a fee or cap, is told they have 48 hours to react after the vote closes. When the live delay is shorter, the change can execute as soon as the vote succeeds and the holder acts after it is live. This needs a deployment whose Note Timelock delay is not 48 hours; on testnet the tokens have no value.

Proof of Concept:

1. Serve the testnet build and open `#/governance/proposals`. The footer reads "... (1 d delay + 5 d vote + 48 h timelock)".
2. Open the Note Governor proposal. The Details card shows "Timelock 2d".
3. Open `#/governance`. The queue header reads "Timelock 48 h" while the Parameters card shows "Execution delay 0m".
4. Run `cast call 0xE7fc64BD1C2720cE03175bC8B650fC2703777aFe "getMinDelay()(uint256)" --rpc-url https://rpc.testnet.chain.robinhood.com`. It returns 0.

Recommended Mitigation: Render every delay from `governors[].timelockDelaySeconds` and `TimelockOperation.minDelaySeconds`, and build the footer from `votingDelay + votingPeriod + getMinDelay`. The proposal page adds a `useGovernance` query and shows a dash until it resolves.

In `GovernanceProposal.tsx`:

```
const { data: g } = useGovernance(w.address ?? null);
// ...
const delay = g?.governors.find((x) => x.kind === p.governor)?.timelockDelaySeconds ?? null;
// ...
<KV k="Timelock" v={delay === null ? '-' : fmtDurationUnits(delay)} />
```

In `Governance.tsx`:

```
-           <span className="faint" style={{ fontSize: 12 }}>Timelock 48 h UpgradeTimelock 7
↳ d</span>
+           <span className="faint" style={{ fontSize: 12 }}>Timelock
↳ {fmtDurationUnits(noteG.timelockDelaySeconds)} UpgradeTimelock
↳ {fmtDurationUnits(upG.timelockDelaySeconds)}</span>
-           <td className="hide-m"><Pill tone={o.timelock === 'UpgradeTimelock' ? 'brass' :
↳ 'faint'}>{o.timelock === 'UpgradeTimelock' ? 'Upgrade 7 d' : 'Core 48 h'}</Pill></td>
+           <td className="hide-m"><Pill tone={o.timelock === 'UpgradeTimelock' ? 'brass' :
↳ 'faint'}>{`$${o.timelock === 'UpgradeTimelock' ? 'Upgrade' : 'Core'}
↳ ${fmtDurationUnits(o.minDelaySeconds)}`}</Pill></td>
```

In `GovernanceProposals.tsx`, build the footer from a schedule helper:

```
/** End-to-end schedule from the live reads: governor votingDelay + votingPeriod + timelock getMinDelay.
↳ */
function schedule(govs: GovernorParams[], kind: GovernorParams['kind'], what: string): string {
  const x = govs.find((v) => v.kind === kind);
  if (!x) return `${what}: schedule unavailable`;
  const total = x.votingDelaySeconds + x.votingPeriodSeconds + x.timelockDelaySeconds;
  return `${what} take about ${fmtDurationUnits(total)} end to end
↳ (${fmtDurationUnits(x.votingDelaySeconds)} delay + ${fmtDurationUnits(x.votingPeriodSeconds)} vote
↳ + ${fmtDurationUnits(x.timelockDelaySeconds)} timelock)`;
}
```

Note Systems: Fixed in commit [21f6d5c](#).

Cyfrin: Verified.

7.2.3 Stale feed fallback is unlabelled, and before the strike it renders a 0.00 price and an empty SHIELD book

Description: latestPrint drops a feed answer older than 4 days (judged by the browser clock) and falls back to the price NoteCore recorded at the last observation, which is 0 before the strike. Only the Notes list marks which price it got; the Series and Portfolio pages show the fallback as the latest print.

```
// web/app/src/lib/portfolio.ts:151-154
export function latestPrint(feed: { answer: bigint; updatedAt: number } | null, observed: bigint, now:
↳ number): bigint {
  if (feed && feed.answer > 0n && feed.updatedAt > 0 && feed.updatedAt <= now + 300 && now -
↳ feed.updatedAt <= FEED_FRESH_SECONDS) return feed.answer;
  return observed > 0n ? observed : 0n;
}
```

buildSeries uses this result for the series price and for the USDG value of the SHIELD book. A series in Subscription has no observed price, so once the feed is stale both become 0 while the SHIELD stock is unchanged:

```
// web/app/src/data/ChainProvider.ts:321-322
depositedShield: refPrice > 0n ? (v.totalShieldStock * refPrice) / (10n ** 20n) : 0n, // 18-dec
↳ stock 1e8 price -> 6-dec USDG
lastPrice: latestPrint(feed, v.lastPrice, now),
```

The Series header then prints "Last price 0.00" and a "Deposited" total without the SHIELD side. The Portfolio detail labels the fallback "Latest print" under a footer saying health uses the latest feed print. Neither page reads priceAt, the one field that records the fallback.

Impact: While a feed is more than 4 days old, series and portfolio viewers see a fallback price presented as current. Before the strike the Series page shows 0.00 and hides the SHIELD book, so a user can deposit into a book that looks open; the strike caps the matched notional and the rest is refundable, so that capital only sits idle until the strike. A 4 day gap needs a feed outage or a market closure longer than a long weekend.

Proof of Concept:

1. Open the testnet build with the page clock (Date.now()) moved more than 4 days past the feed's last updatedAt, for example by offsetting Date by 98 hours.
2. Open #/series/71 (TSLA, Subscription). The header reads "Last price 0.00 · 0.0% of ref" and "Deposited 772.8K USDG · Cap 1.00M" (2.79M with the SHIELD book), with no stale marker.
3. Open #/series/10 (AAPL, Live). It reads "Last price 337.48 · 100.0% of S0", the strike close, with no marker.
4. On #/portfolio, expand a series 18 row. It shows "Latest print 340.05", the last observed close, above the footer "Health uses the latest feed print against the barrier level".

Recommended Mitigation: Before the strike, fall back to the stale feed answer instead of 0 so the price and the SHIELD book stay populated. Show the existing FreshMark (relabelled "Feed stale") wherever the fallback price is displayed on the Series and Portfolio pages.

```
const staleFeed = feed && feed.answer > 0n && feed.updatedAt <= now + 300 ? feed.answer : 0n;
const observedOrStale = v.lastPrice > 0n ? v.lastPrice : staleFeed;
// ...
const refPrice = lastFeed > 0n ? lastFeed : observedOrStale;
// ...
lastPrice: latestPrint(feed, observedOrStale, now),
```

Note Systems: Fixed in commit [5c67549](#).

Cyfrin: Verified.

7.2.4 Date formatters throw RangeError past year 275760, so a bond market with a uint48 max conclusion blanks the app

Description: `fmtDate`, `fmtDateY`, `fmtDateTime` and `fmtTime` pass `ts * 1000` straight to `Intl.DateTimeFormat.format`, which throws `RangeError: Invalid time value` for any value above `8.64e15` ms (timestamps above 8,640,000,000,000 s).

```
// web/app/src/lib/format.ts:86-94
const DF = new Intl.DateTimeFormat('en-GB', { day: '2-digit', month: 'short', timeZone: 'UTC' });
// ...
export const fmtDate = (ts: number) => DF.format(ts * 1000);
export const fmtDateY = (ts: number) => DFY.format(ts * 1000);
export const fmtDateTime = (ts: number) => `${DTF.format(ts * 1000)} UTC`;
const TF = new Intl.DateTimeFormat('en-GB', { hour: '2-digit', minute: '2-digit', timeZone: 'UTC',
  ↪ hour12: false });
export const fmtTime = (ts: number) => `${TF.format(ts * 1000)} UTC`;
```

Bond market end times are `uint48` on chain, and `createMarket` only requires the conclusion to be in the future (if `(p.conclusion <= block.timestamp)` revert `InvalidParams()`; in `contracts/src/token/BondDepository.sol#L275-L281`).

`type(uint48).max` (281,474,976,710,655) is the natural value for a market with no end date. The Bonds table formats every open market's conclusion during render:

```
// web/app/src/routes/Bonds.tsx:100
<td className="r hide-m">{m.conclusion > now ? <span
  ↪ className="num">{fmtDate(m.conclusion)}</span> : <Pill
  ↪ tone="faint">Closed</Pill></td>
```

No component in the tree catches render errors, so the throw unmounts the whole React root. The same formatters receive `now + m.vestingSeconds * k` (`Bonds.tsx#L198`) and a bond's `vestEnd` (`Portfolio.tsx#L329`), so a very large `vestingSeconds` has the same effect.

Impact: Once such a market exists, every visitor who opens `#/bonds` gets a blank page (rail and header included) until they reload on another route. Holders claim vested NOTE on the Bonds route, so every bondholder, including holders of other markets, loses the in-app claim path. Calling `BondDepository` directly still works, and no funds move.

Only the owner (the Timelock) can create a market with a conclusion after year 275760. All 42 markets on testnet end by 30 Oct 2026 (latest conclusion 1793390400, longest vesting 604,800 s).

Proof of Concept:

1. Start an anvil fork of testnet (`anvil --fork-url https://rpc.testnet.chain.robinhood.com --chain-id 46630 --port 8545`) and point a local build at it (the testnet RPC in `src/chain/robinhood.ts` and `RPC_HOST.testnet` in `vite.config.ts`).
2. Create a USDG market with `conclusion = 2**48 - 1` from the Timelock:

```
TL=0xE7fc64BD1C2720cE03175bC8B650fC2703777aFe; BD=0x9af40Dbcc0f958d4a7B5B75901895d1cFE40Fbee
cast rpc anvil_impersonateAccount $TL; cast rpc anvil_setBalance $TL 0x56BC75E2D63100000
cast send $BD
  ↪ "createMarket((uint8,address,uint256,uint256,uint256,uint256,uint256,uint48,uint48))" \
  ↪ "(0,0x433947311e248C9fEc39c69Fac2a305661CCb907,0,50000000000000000000000000000000,5,
  ↪ 0000000000000000,25000000000000000000000000000000,86400,281474976710655)" \
  ↪ --from $TL --unlocked
```

3. Open `#/notes`, then click Bonds. The page goes blank and stays blank after clicking back to Notes. The console shows `RangeError: Invalid time value`.

Recommended Mitigation: Return EMPTY from the four formatters for timestamps a Date cannot represent, so a far-future chain value renders as a dash.

In web/app/src/lib/format.ts, add a range check and apply it in each:

```
+const MAX_DATE_TS = 8.64e12;
+const dateOk = (ts: number) => Number.isFinite(ts) && Math.abs(ts) <= MAX_DATE_TS;
-export const fmtDate = (ts: number) => DF.format(ts * 1000);
-export const fmtDateY = (ts: number) => DFY.format(ts * 1000);
-export const fmtDateTime = (ts: number) => `${DTF.format(ts * 1000)} UTC`;
+export const fmtDate = (ts: number) => (dateOk(ts) ? DF.format(ts * 1000) : EMPTY);
+export const fmtDateY = (ts: number) => (dateOk(ts) ? DFY.format(ts * 1000) : EMPTY);
+export const fmtDateTime = (ts: number) => (dateOk(ts) ? `${DTF.format(ts * 1000)} UTC` : EMPTY);
const TF = new Intl.DateTimeFormat('en-GB', { hour: '2-digit', minute: '2-digit', timeZone: 'UTC',
  ↳ hour12: false });
-export const fmtTime = (ts: number) => `${TF.format(ts * 1000)} UTC`;
+export const fmtTime = (ts: number) => (dateOk(ts) ? `${TF.format(ts * 1000)} UTC` : EMPTY);
```

Note Systems: Fixed in commit [c6f373b](#).

Cyfrin: Verified.

7.2.5 Approve-then-act flows simulate the final call only after the approval is signed

Description: runSteps simulates each step only when its turn comes, so in an approve-then-act flow the final call is first simulated after the user has signed the approval and it has mined. The step builders in chain/actions.ts check only the token balance and the current allowance before emitting the approval, so any other reason the final call will revert surfaces only after the approval is paid for.

```
// web/app/src/wallet/tx.ts:195-238
export async function runSteps(steps: TxStep[], account: `0x${string}`, onStep?: ...):
  ↳ Promise<TxResult> {
  // ...
  const exec = async (i: number, s: TxStep) => {
    // ...
    if (!s.noSimulate || s.minResult !== undefined) {
      onStep?.(i, s, 'simulate');
      // Throws with a decoded custom error if the call would revert, surfaced before the wallet prompt.
      const sim = await c.simulateContract(c.wagmiConfig, base as never).catch((e: unknown) => { throw
        ↳ new StepError(s, e); });
      // ...
    }
    // ... sign, wait for the receipt
  };
  try {
    for (let i = 0; i < steps.length; i++) await exec(i, steps[i]);
```

The page gates do not check the following states, and in each the contract reverts before any token moves:

- NoteCore is paused: COUPON and SHIELD deposits revert with EnforcedPause.
- An expired lock was never withdrawn: Create lock reverts with LockExists.
- The buyback is inside its fill interval or below its minimum: Sell NOTE reverts with FillTooSoon or FillBelowMinimum.
- A SHIELD deposit is below the minimum ticket or its prefund exceeds the bound: BelowMinTicket or ExcessPrefund.
- A COUPON-leg bond exceeds the Treasury's holding cap: ExceedsLegShare.
- The account is a contract without IERC1155Receiver, such as an EIP-7702 delegated EOA: CannotReceiveLegs.

- The device clock is behind the chain: SubscriptionClosed, Expired Or UnlockTimeInPast.

Impact: A user who starts one of these flows in one of these states signs one or two approvals, waits for them to mine, then gets a toast with the bare error name, such as "COUPON deposit failed: EnforcedPause". No funds move; the cost is the approval gas and an allowance left standing. Most states are ordinary, such as a guardian pause or an unwithdrawn expired lock.

Proof of Concept:

1. On a testnet build pointed at a fork of Robinhood Chain testnet, pause NoteCore with the guardian (NoteCore.pause()), then open #/series/63 (any series in Subscription) with a funded wallet.
2. The COUPON card shows "Review purchase" enabled and no pause notice. Enter 50 USDG, click Review purchase, then Confirm deposit.
3. The wallet asks for approve(NoteCore, 50000000) on USDG. Sign it; it mines.
4. The runner then simulates depositCoupon, which reverts. The toast reads "COUPON deposit failed: EnforcedPause" and the 50 USDG allowance to NoteCore stays in place.

Recommended Mitigation: Before the first wallet prompt, simulate the whole step list in order with eth_simulateV1 (viem simulateCalls), so each call sees the allowances granted by the earlier steps, and stop at the first failing step. Where the RPC does not serve eth_simulateV1, keep the current per-step behaviour. Add plain messages in describeDecoded for the errors above.

In a new web/app/src/wallet/preflight.ts:

```
export async function simulateSequence(client: Client, steps: readonly SimStep[], account:
↳ `0x${string}`): Promise<{ index: number; error: unknown } | null> {
  if (steps.length < 2 || steps.some((s) => s.noSimulate)) return null;
  const { simulateCalls } = await import('viem/actions');
  let res: Awaited<ReturnType<typeof simulateCalls>>;
  try {
    res = await simulateCalls(client, {
      account,
      calls: steps.map((s) => ({ to: s.address, abi: s.abi as Abi, functionName: s.functionName, args:
↳ (s.args ?? []) as unknown[], value: s.value })),
    } as never);
  } catch {
    return null;
  }
  const index = res.results.findIndex((r) => r.status === 'failure');
  return index < 0 ? null : { index, error: (res.results[index] as { error?: unknown }).error };
}
```

In runSteps (web/app/src/wallet/tx.ts), before the per-step loop:

```
if (eth === On) throw new PreflightError('Not enough ETH for gas on Robinhood Chain. This wallet
↳ holds 0 ETH; top up and try again.');
```

```
+ const pub = c.getPublicClient(c.wagmiConfig, { chainId: robinhoodChain.id });
+ const failing = pub ? await simulateSequence(pub, steps, account) : null;
+ if (failing) throw new StepError(steps[failing.index], failing.error);
```

Note Systems: Fixed in commit [7b0d637](#).

Cyfrin: Verified.

7.2.6 A failed Desk paused() read is coerced to false and opens the deposit form while the Desk is paused

Description: getDesk reads Desk.paused() through the tolerant multicall helper mc, which maps any failed leg to undefined. It then stores Boolean(...) of that value, so a failed read becomes paused: false.

```
// web/app/src/data/ChainProvider.ts:110-115
/** Multicall where an individual read may legitimately revert (payoutFor on an unacceptable leg,
  ↳ optional views): failed legs map to undefined. */
private async mc<T extends readonly ContractFunctionParameters[]>(contracts: T):
  ↳ Promise<Array<unknown>> {
  if (contracts.length === 0) return [];
  const res = await multicall(this.client, { contracts: contracts as unknown as
    ↳ ContractFunctionParameters[], allowFailure: true, multicallAddress: MULTICALL3, batchSize: 2048
    ↳ });
  return res.map((r) => (r.status === 'success' ? r.result : undefined));
}
```

On the Desk, paused and maxDeposit sit in the same tolerant batch, separate from the strict batch that carries NAV and balances. viem runs each batch as its own eth_call to Multicall3. With allowFailure: true, a failed eth_call (HTTP error, timeout, or one rejected item of a JSON-RPC batch) marks every leg of that batch as failed instead of throwing. One failed request clears both deposit gates while the rest of the page loads normally:

```
// web/app/src/data/ChainProvider.ts:489-492
const [all, [totalClaimable, processLimit, minRequest, pausedRaw, maxDepositRaw, userClaimable]] =
  ↳ await Promise.all([
    this.mcStrict([...base, ...userCalls], 'getDesk'),
    this.mc([c('totalClaimable'), c('queueProcessLimit'), c('minRequestQuote'), c('paused'),
      ↳ c('maxDeposit', [owner ?? zeroAddress]), ...(owner ? [c('claimable', [owner])] : [])]),
  ]]);
```

```
// web/app/src/data/ChainProvider.ts:543-545
paused: Boolean(pausedRaw),
// A failed maxDeposit read must not show deposits as closed: only an explicit zero does.
depositsClosed: typeof maxDepositRaw === 'bigint' && maxDepositRaw === 0n,
```

```
// web/app/src/routes/Desk.tsx:359
const depositBlocked = isDeposit && (depositsClosed || paused);
```

The comment on mcStrict says figures the page cannot transact without must reject the query. The pause flag is one of those figures and is on the tolerant path. GaugeController.isKilled goes through the same Boolean(<maybe undefined>) coercion (killed in getGauges).

Impact: While the guardian has paused the Desk, one failed eth_call for the tolerant batch (the public RPC fails requests under load) shows #/desk with an open deposit form. A user with a short allowance pays gas for the USDG approval and keeps an exact-amount allowance to the Desk; the pre-sign simulation then refuses the deposit (ERC4626ExceededMaxDeposit), so no deposit is sent. No funds move.

Proof of Concept:

1. Fork the testnet with anvil, impersonate the guardian 0x0b64B35c6Dd23944D6D4029864D2cA2AA1B66422 and call Desk.pause().
2. Point a production build at the fork and make the RPC reject the eth_call whose calldata contains the paused() selector 0x5c975abb (for example with a proxy in front of the RPC).
3. Open #/desk: the Deposit tab shows no "Paused" row and "Review deposit" is enabled for any amount.

Save the script below as web/app/poc-pause-fail-open.mjs and, with the step 1 fork on port 8555, run RPC=http://127.0.0.1:8555 node poc-pause-fail-open.mjs from web/app:

```
// Save as web/app/poc-pause-fail-open.mjs and run from web/app:
// RPC=http://127.0.0.1:8555 node poc-pause-fail-open.mjs
```

```

// Bundles the app's own src/data/ChainProvider.ts with esbuild (already in node_modules via vite) and
↳ runs getDesk()
// with fetch wrapped so that chosen JSON-RPC eth_call requests fail the way a public RPC fails
// one item of a batch (rate limit). RPC defaults to the public testnet; point it at an anvil fork to
↳ pause first.
import { build } from 'esbuild';
import { pathToFileURL } from 'node:url';
import { tmpdir } from 'node:os';
import { join } from 'node:path';

const TESTNET = 'https://rpc.testnet.chain.robinhood.com';
const RPC = process.env.RPC ?? TESTNET;
const PAUSED_SELECTOR = '5c975abb'; // paused()
const outfile = join(tmpdir(), `ns-chainprovider-${process.pid}.mjs`);
await build({
  stdin: { contents: "export { ChainProvider } from './src/data/ChainProvider.ts';", resolveDir:
    ↳ process.cwd(), loader: 'ts' },
  bundle: true, format: 'esm', platform: 'node', outfile, logLevel: 'silent',
  define: { 'import.meta.env.VITE_NETWORK': '"testnet"' }, loader: { '.json': 'json' },
});
const { ChainProvider } = await import(pathToFileURL(outfile).href);

let mode = 'none';
const realFetch = globalThis.fetch;
const shouldFail = (x) => x.method === 'eth_call' && (mode === 'all' || (mode === 'paused' &&
↳ String(x.params?.[0]?.data ?? '').includes(PAUSED_SELECTOR)));
globalThis.fetch = async (url, init) => {
  if (!String(url).startsWith(TESTNET)) return realFetch(url, init);
  const body = JSON.parse(init.body);
  const items = Array.isArray(body) ? body : [body];
  const keep = items.filter((x) => !shouldFail(x));
  const got = keep.length ? await (await realFetch(RPC, { ...init, body: JSON.stringify(keep)
    ↳ })).json() : [];
  const byId = new Map((Array.isArray(got) ? got : [got]).map((r) => [r.id, r]));
  const res = items.map((x) => shouldFail(x) ? { jsonrpc: '2.0', id: x.id, error: { code: -32005,
    ↳ message: 'request rate exceeded' } } : byId.get(x.id));
  return new Response(JSON.stringify(Array.isArray(body) ? res : res[0]), { status: 200, headers: {
    ↳ 'content-type': 'application/json' } });
};

const desk = '0x814B6d8b37a22479Bee6228032716A99C54ac946';
const onChain = async (to) => {
  const r = await realFetch(RPC, { method: 'POST', headers: { 'content-type': 'application/json' },
    ↳ body: JSON.stringify({ jsonrpc: '2.0', id: 1, method: 'eth_call', params: [{ to, data: '0x' +
    ↳ PAUSED_SELECTOR }, 'latest'] } ) });
  return BigInt((await r.json()).result) === 1n;
};

console.log(`rpc ${RPC} · on chain: Desk.paused() = ${await onChain(desk)}`);
for (const m of ['none', 'paused']) {
  mode = m;
  const p = new ChainProvider();
  const d = await p.getDesk(null).then((x) => `paused=${x.paused} depositsClosed=${x.depositsClosed}`,
    ↳ (e) => `rejected (${e.message.split('\n')[0].slice(0, 60)})`);
  console.log(`failing eth_call: ${m.padEnd(6)} | getDesk: ${d}`);
}

```

```

rpc http://127.0.0.1:8555 · on chain: Desk.paused() = true
failing eth_call: none | getDesk: paused=true depositsClosed=true
failing eth_call: paused | getDesk: paused=false depositsClosed=false

```

Recommended Mitigation: Keep a failed pause read as unknown (null) instead of false, and disable Desk deposits while it is unknown.

1. In web/app/src/data/ChainProvider.ts, getDesk keeps only a real boolean, and DeskState.paused in types.ts becomes boolean | null:

```
paused: typeof pausedRaw === 'boolean' ? pausedRaw : null,
```

2. In web/app/src/routes/Desk.tsx, DepositCard treats an unknown pause state as blocked. The "Deposits" row reads "Unknown" and the button reads "Pause state unknown" in that case:

```
const pauseUnknown = paused === null;
const depositBlocked = isDeposit && (depositsClosed || paused !== false);
const depositBlockedCopy = pauseUnknown ? 'The Desk pause state could not be read. Deposits stay
↳ disabled until it loads; reload to try again.' : paused ? 'The Desk is paused. Deposits and exits
↳ resume once the guardian unpauses it.' : 'Deposits are closed while a held stock has no valid
↳ price. They reopen once the stock is priced again.';
```

Note Systems: Fixed in commit [22a11a9](#).

Cyfrin: Verified.

7.2.7 Failed RPC reads render as zero or empty results on the Notes, Series, Portfolio and Governance pages

Description: Several ChainProvider reads turn a failed RPC call into a valid-looking zero or empty answer, and several page heads check only isLoading. The pages then state "not found", zero totals or "no activity" as facts when the app has no data.

getSeries catches every error and returns null, the same value it returns for an id that does not exist, so a timeout reaches the page as "no such series":

```
// web/app/src/data/ChainProvider.ts:200-206
async getSeries(id: number): Promise<Series | null> {
  const core = need(ADDRESSES.noteCore, 'getSeries');
  let v: SeriesView;
  try {
    v = (await readContract(this.client, { address: core, abi: noteCoreAbi, functionName:
    ↳ 'getSeries', args: [BigInt(id)] }))) as unknown as SeriesView;
  } catch { return null; }
  if (v.status === 0) return null;
```

The same pattern appears in these places:

- Series page: a failed read shows "Series not found" with no Retry.
- Portfolio activity: a failed log scan becomes an empty history, and "Export CSV" writes no activity rows.
- Portfolio and Notes heads: a failed or offline-paused query reads as 0 positions, 0.00 USDG and "None scheduled".
- Desk, Stake and Bonds: failed per-wallet reads drop the wallet's claimable amount, requests and bonds.
- Governance: getGovernance never rejects, so a failed read shows 0 veNOTE supply and proposals as Pending with 0 votes.

Impact: When a read fails, which is routine on the rate-limited public RPC, users are told as fact that a series does not exist, that a portfolio is empty or that a defeated proposal is pending, and an exported CSV can be filed as complete. No transaction is built from these screens, and the figures return once the RPC recovers.

Proof of Concept:

1. On a testnet build, open `#/notes`, block `rpc.testnet.chain.robinhood.com` in DevTools and navigate to `#/series/71`, which exists. After about 20 s the page shows "Series not found" with no Retry.
2. Unblock, connect a wallet with positions, block again and open `#/portfolio`. The head reads "0.00 USDG · 0 positions" and "0 NOTE" above "Positions unavailable".
3. Block the RPC and load `#/notes` fresh. The head reads "Live series 0" and "Next strike None scheduled" above "Could not load series".
4. To show the governance case, where only the Multicall3 reads fail, save the script below as `web/app/poc-failed-reads.mjs` and run `node poc-failed-reads.mjs` from `web/app`.

```
// Save as web/app/poc-failed-reads.mjs and run from web/app: node poc-failed-reads.mjs [owner]
↳ [seriesId]
// Bundles the app's own ChainProvider with esbuild and wraps fetch so chosen JSON-RPC requests fail the
↳ way the
// public RPC fails them (503, 429, log limit, node error). Shows what the provider hands to the pages.
import { build } from 'esbuild';
import { pathToFileURL } from 'node:url';
import { tmpdir } from 'node:os';
import { join } from 'node:path';

const outfile = join(tmpdir(), `ns-failed-reads-${process.pid}.mjs`);
await build({
  stdin: { contents: "export { ChainProvider } from './src/data/ChainProvider.ts';", resolveDir:
    ↳ process.cwd(), loader: 'ts' },
  bundle: true, format: 'esm', platform: 'node', outfile, logLevel: 'silent',
  define: { 'import.meta.env.VITE_NETWORK': '"testnet"' }, loader: { '.json': 'json' },
});
const { ChainProvider } = await import(pathToFileURL(outfile).href);
const owner = process.argv[2] ?? '0xdE8e1F48d4b21195B169d2Ac2D95F529FEf793C9';
const sid = Number(process.argv[3] ?? 71);

// failing(item) -> true: the item fails (503 when every item of the HTTP request fails, else a JSON-RPC
↳ error)
let failing = () => false;
const realFetch = globalThis.fetch;
globalThis.fetch = async (url, init) => {
  const body = JSON.parse(init.body);
  const items = Array.isArray(body) ? body : [body];
  if (items.every((x) => failing(x))) return new Response('Service Unavailable', { status: 503 });
  const keep = items.filter((x) => !failing(x));
  const got = await (await realFetch(url, { ...init, body: JSON.stringify(keep) })).json();
  const byId = new Map((Array.isArray(got) ? got : [got]).map((r) => [r.id, r]));
  const res = items.map((x) => failing(x) ? { jsonrpc: '2.0', id: x.id, error: { code: -32005, message:
    ↳ 'request failed' } } : byId.get(x.id));
  return new Response(JSON.stringify(Array.isArray(body) ? res : res[0]), { status: 200, headers: {
    ↳ 'content-type': 'application/json' } });
};

const run = async (label, f, fn) => {
  failing = f;
  const r = await fn(new ChainProvider()).then((v) => v, (e) => `REJECTED:
    ↳ ${e.message.split('\n')[0].slice(0, 70)}`);
  console.log(`${label.padEnd(38)} ${typeof r === 'string' ? r : JSON.stringify(r)}`);
};

const pf = (p) => p.getPortfolio(owner).then((x) => ({ positions: x.positions.length, historyRows:
  ↳ x.history.length, historyError: x.historyError ?? 'field absent' }));
const sr = (p) => p.getSeries(sid).then((s) => (s === null ? 'null (page renders "Series not found")' :
  ↳ { id: s.id, status: s.status }));
const gv = (p) => p.getGovernance(null).then((g) => ({
  veSupply: String(g.ve.totalSupply), noteLocked: String(g.ve.totalLocked),
```

```

proposals: g.proposals.map((x) => `${x.state} for=${x.forVotes}`), queuedOps:
↳ g.timelockOps.filter((o) => o.state !== 'Done').length,
});
const multicall = (x) => x.method === 'eth_call' && String(x.params?.[0]?.data ??
↳ '').startsWith('0x82ad56cb');

console.log(new Date().toISOString(), 'owner', owner);
await run('getPortfolio, no failure', () => false, pf);
await run('getPortfolio, eth_getLogs fails', (x) => x.method === 'eth_getLogs', pf);
await run(`getSeries(${sid}), no failure`, () => false, sr);
await run(`getSeries(${sid}), every request 503`, () => true, sr);
await run('getSeries(9999), no failure', () => false, (p) => p.getSeries(9999).then((s) => (s === null
↳ ? 'null (unknown id)' : s.id)));
await run('getGovernance, no failure', () => false, gv);
await run('getGovernance, multicalls fail', multicall, gv);

```

```

getPortfolio, eth_getLogs fails      {"positions":2,"historyRows":0,"historyError":"field absent"}
getSeries(71), every request 503     null (page renders "Series not found")
getGovernance, no failure            {"veSupply":"158879700690336304116150","noteLocked":"13472364791
↳ 2563227513203","proposals":["Defeated for=265015889419243676092"],"queuedOps":0}
getGovernance, multicalls fail      {"veSupply":"0","noteLocked":"0","proposals":["Pending
↳ for=0"],"queuedOps":6}

```

Recommended Mitigation: Report a failed read as an error or as unknown, and have every head figure check for it. In `getSeries`, return null only for a revert:

```

} catch (e) {
  // Only a revert (unknown id) means "no such series"; a transport failure must reach the page as
  ↳ an error.
  if (e instanceof BaseError && e.walk((x) => x instanceof ContractFunctionRevertedError)) return
  ↳ null;
  throw e;
}

```

Have history flag a failed scan, and `getGovernance` reject when a displayed read is missing:

```

return { positions, history: history ?? [], historyError: history === null };
// ...
const unread = [0, 1, ...user.map((_, i) => base.length + i)].filter((i) => r[i] === undefined);
if (unread.length) throw new Error(`getGovernance: ${unread.length} veNOTE or balance reads
↳ failed`);

```

On the pages, show a dash while the data is unknown, and add an `isError` branch with `Retry` to the `Series` and `governance` routes:

```

const headUnknown = isLoading || isError || data === undefined;

```

Note Systems: Fixed in commit [e6efa77](#).

Cyfrin: Verified.

7.2.8 Deposit confirm sheets state the maturity breach payoff at the latest print for the whole deposit

Description: The COUPON and SHIELD confirm sheets compute the breach payoff from the latest price print and the full typed amount, and state it as what the depositor receives. NoteCore pays that payoff at S0 (fixed at strike) and only on the matched part of the deposit. Deposits are only possible during subscription (subscriptionOpen), when s.s0 is always null, so the sheet never has S0 to show.

CouponCard falls back to the latest print and divides the whole deposit by it:

```
// web/app/src/routes/Series.tsx:187-190
const ref = toNumber(s.s0 ?? s.lastPrice, 8);
// ...
const shares = ref > 0 ? n / ref : 0;
```

The card row labels that print as S0, and the confirm sentence states the result as the amount received:

```
// web/app/src/routes/Series.tsx:207
<KV k="If breached at maturity" v={/* ... */} hint={`Physical settlement: amount S0 (${fmtNum(ref, 2)})}
↳ Stock Tokens.`} />
```

```
// web/app/src/routes/Series.tsx:222
plain=<{>/* ... */ If it closes below {fmtPct(s.barrierBps, 0)} of S0 at maturity, you receive
↳ <strong>{fmtNum(shares, 4)} {s.underlying.symbol}</strong> instead of your USDG.</>
```

ShieldCard values the whole stock deposit at the OracleAdapter reference price (useDisplayRef returns referencePrice while s.s0 is null) and states that as the USDG received on a breach:

```
// web/app/src/routes/Series.tsx:252-256
const refView = useDisplayRef(s);
const ref = toNumber(refView.ref, 8);
// ...
const notional = stock * ref;
```

```
// web/app/src/routes/Series.tsx:317
plain=<{>/* ... */ If {s.underlying.symbol} closes below {fmtPct(s.barrierBps, 0)} of S0 at maturity,
↳ your stock is delivered and you receive <strong>{usd(notional)} USDG</strong>.</>
```

NoteCore sizes each position on the matched notional n (couponUnits = p.couponDeposit.mulDiv(n, d) and shieldUnits = p.shieldStock.mulDiv(n, s.totalShieldStock), contracts/src/core/NoteCore.sol#L1079-L1085) and on a breach converts at S0 (_quoteToStock(units, s.s0), contracts/src/core/NoteCore.sol#L1192-L1196).

Impact: A depositor reads the breach line as their downside, but it is off by the price move before strike and by the unmatched fraction. The gap is material when the price moves several percent in the eight days series 74 to 81 stay open, or when that side is over-subscribed. The unmatched part is refundable.

Proof of Concept:

1. Open /series/<id> for any series in subscription and type 1000 into the COUPON amount field.
2. The card shows "If breached at maturity 1.2956 NVDA" with the hint "Physical settlement: amount ÷ S0 (771.85)", where 771.85 is the latest print; S0 does not exist yet.
3. Press "Review purchase". The sheet says "If it closes below 65% of S0 at maturity, you receive 1.2956 NVDA instead of your USDG". The wallet then asks to approve and deposit 1,000 USDG.
4. On struck series 64 (getSeries(64) on NoteCore 0xE47BBec63D836643F8FBC50e61011c9eb278a55e), the SHIELD sheet for 1.3 NVDA at print 771.85 stated 1,003.41 USDG. With S0 777.57 and 9.5% matched, NoteCore pays 96.36 USDG on a breach.

Recommended Mitigation: Describe the breach payoff as the matched amount converted at S0, and show the figure as a fully matched estimate at the latest print. After strike the card keeps the exact S0 wording. All changes are in `web/app/src/routes/Series.tsx`.

1. In `CouponCard`, record whether S0 is known:

```
const ref = toNumber(s.s0 ?? s.lastPrice, 8);  
+ const struck = s.s0 !== null;
```

2. On the "If breached at maturity" row, append " est." to the symbol while `struck` is false and switch the hint:

```
hint={struck ? `Physical settlement: amount S0 (${fmtNum(ref, 2)}) Stock Tokens.` : `Physical  
  ↳ settlement: matched amount S0, fixed at strike. Estimated fully matched at the latest print  
  ↳ (${fmtNum(ref, 2)})`}
```

3. In the COUPON confirm sheet, state the rule and give the figure as the fully matched estimate:

```
plain=<{</ * ... */ If it closes below {fmtPct(s.barrierBps, 0)} of S0 at maturity, you receive your  
  ↳ matched USDG S0 in {s.underlying.symbol} instead of your USDG. S0 is fixed at strike; fully  
  ↳ matched at the latest print ({fmtNum(ref, 2)}) that is <strong>{fmtNum(shares, 4)}  
  ↳ {s.underlying.symbol}</strong>.</>
```

4. In the SHIELD confirm sheet, do the same for the USDG received:

```
plain=<{</ * ... */ If {s.underlying.symbol} closes below {fmtPct(s.barrierBps, 0)} of S0 at maturity,  
  ↳ your matched stock is delivered and you receive its value at S0 in USDG. S0 is fixed at strike;  
  ↳ fully matched at the {refBasis} that is <strong>{usd(notional)} USDG</strong>.</>
```

Note Systems: Fixed in commit [5e07f1e](#).

Cyfrin: Verified.

7.2.9 Subscription progress adds COUPON deposits and SHIELD notional and compares the sum with a per side notional cap

Description: The subscription views add COUPON deposits and SHIELD notional into one "deposited" figure and draw it as progress towards `notionalCap`, but the cap bounds the matched notional, to which each side contributes in full. NoteCore does not cap deposits: at strike it matches `min(COUPON, SHIELD, cap)` and refunds the rest. The sum hides which side is long.

```
// web/app/src/components/NotesBook.tsx:162-167  
const raised = s.depositedCoupon + s.depositedShield;  
// ...  
<Meter value={toNumber(raised, 6)} max={toNumber(s.notionalCap, 6)} label={` ${compact(raised)}  
  ↳ of ${compact(s.notionalCap)} USDG cap` } />  
<span className="srow__fig num">{compact(raised)} of {compact(s.notionalCap)} cap { /* ...  
  ↳ */ }</span>
```

The same sum is used in:

- the "Deposited" columns and the per stock group total in `NotesBook.tsx`;
- the "matched" sort key in `notesBook.ts`;
- the "Deposited" stat in the `Series.tsx` header;
- the public analytics book in `analytics/src/book.ts`.

Impact: A depositor picks a series and side from a figure that hides the imbalance. Series 26 reads "0.57M of 1.00M cap", which looks like room, yet at least 91% of a new SHIELD deposit would come back unmatched, and a balanced, exactly full book would read "2.00M of 1.00M cap". Nothing is lost, but the long side's stock or USDG sits idle until strike and costs a refund transaction.

Proof of Concept:

1. Open /notes, Subscribing tab. Series 48 shows "1.55M of 1.00M cap" and a full bar; series 26 shows "0.57M of 1.00M cap".
2. Open /series/26. The header shows "Deposited 0.57M USDG, Cap 1.00M".
3. Deposit SHIELD for any amount. At strike NoteCore matches $\min(0.05M, 0.52M, 1.00M)$, so about 9% of the new SHIELD deposit converts and the rest is refundable.

Recommended Mitigation: Add one helper for the notional NoteCore can match from the current book and use it wherever a subscription figure is compared with the cap. Print the two sides separately and rename the "Deposited" column to "Matchable". Totals that are not compared with the cap keep the sum.

In web/app/src/lib/notesBook.ts:

```
export function matchable(s: Pick<Series, 'depositedCoupon' | 'depositedShield' | 'notionalCap'>):  
  ↳ bigint {  
    const m = s.depositedCoupon < s.depositedShield ? s.depositedCoupon : s.depositedShield;  
    return m < s.notionalCap ? m : s.notionalCap;  
  }
```

In SeriesRow (NotesBook.tsx), and in the same way in the other views:

```
const raised = matchable(s);  
// ...  
  <span className="srow_fig num">COUPON {compact(s.depositedCoupon)} SHIELD  
    ↳ {compact(s.depositedShield)} of {compact(s.notionalCap)} cap {fmtPct(s.couponFloorBps, 1)}  
    ↳ to {fmtPct(s.couponCapBps, 1)} <span className="faint">est.</span></span>
```

Note Systems: Fixed in commit [fcb04ac](#).

Cyfrin: Verified.

7.2.10 Notes and Portfolio print different distance to barrier figures for the same series and colour them on different scales

Description: The Notes page measures "to barrier" as the print above the barrier relative to the barrier level, while Portfolio measures it relative to the print, and each page applies its own thresholds to its own figure. Notes computes it as below, and the analytics series book (analytics/src/book.ts#L101 and #L153) uses the same formula:

```
// web/app/src/lib/notesBook.ts:82-89  
/** Distance from the last print to the barrier as a percentage of the barrier level; null without a  
  ↳ print. */  
export function barrierDistance(s: Series): number | null {  
  // ...  
  return (toNumber(s.lastPrice, 8) / barrier - 1) * 100;  
}
```

and colours it green from 10% up:

```
// web/app/src/components/NotesBook.tsx:22  
const distTone = (d: number | null) => (d === null ? '' : d < 0 ? 'tone-down' : d < 10 ? 'tone-warn' :  
  ↳ 'tone-up');
```

Portfolio computes $(\text{price} - \text{barrier}) / \text{price}$, classes the position "At risk" below 10% and "Watch" below 20% of that figure, and prints it with the same "to barrier" wording:

```
// web/app/src/lib/portfolio.ts:53-57
const bufferBps = bpsOf(s.lastPrice - barrier, s.lastPrice);
// ...
const cls: HealthClass = s.lastPrice < barrier ? 'breached' : bufferBps < 1_000 ? 'atRisk' :
↳ bufferBps < 2_000 ? 'watch' : 'sound';
```

```
// web/app/src/routes/Portfolio.tsx:269-270
<Pill tone={HEALTH_TONE[health.cls]}>{HEALTH_LABEL[health.cls]}</Pill>
<span className="cell-sub num">{health.bufferBps !== null ?
↳ `${health.bufferBps >= 0 ? '+' : ''}${fmtPct(health.bufferBps, 1)} to
↳ barrier` : ''}{/* ... */}</span>
```

Impact: A holder who checks a position on both pages sees two "to barrier" figures for the same series and print, on every Live series; series 12 reads "+43.2% to barrier" on Notes and "+30.1% to barrier" on Portfolio. The Notes figure overstates how far the price can fall. For a print between 10.0% and 25% above the barrier level, Notes colours the row green while Portfolio shows "At risk" or "Watch", and no live series sits in that band today. No transaction changes.

Proof of Concept:

1. Open /notes, Live tab, and find series 12: the row reads "+43.2% to barrier".
2. With a wallet holding a series 12 leg, open /portfolio: the same position reads "Sound, +30.1% to barrier".
3. For a print 10.5% above a barrier, Notes shows a green "+10.5% to barrier" and Portfolio a red "At risk, +9.5% to barrier".

Recommended Mitigation: Define "to barrier" once, as the drop from the print to the barrier relative to the print (the measure Portfolio health and its thresholds already use), and use it on Notes and on the analytics page. The Notes amber threshold of 10% then coincides with the Portfolio "At risk" threshold.

1. Add the shared helper to web/app/src/lib/format.ts:

```
/**
 * Headroom above the barrier as a percentage of the current price: how far the price can fall before it
 ↳ breaches,
 * negative below the barrier. The one definition of "to barrier" (Portfolio health, lib/portfolio.ts,
 ↳ uses the same).
 */
export function barrierBufferPct(price: number, barrier: number): number | null {
  return price > 0 && barrier > 0 ? (1 - barrier / price) * 100 : null;
}
```

2. In barrierDistance (web/app/src/lib/notesBook.ts), import barrierBufferPct and return it. Its guard replaces the barrier === 0 check, and the doc comment should say "as a percentage of the last print":

```
- if (barrier === 0) return null;
- return (toNumber(s.lastPrice, 8) / barrier - 1) * 100;
+ return barrierBufferPct(toNumber(s.lastPrice, 8), barrier);
```

3. In seriesRow and tableRows (web/app/analytics/src/book.ts), import the helper and use it for both distance figures:

```
- const dist = price ? (price / (s0 * s.barrierBps / 10_000) - 1) * 100 : null;
+ const dist = price ? barrierBufferPct(price, s0 * s.barrierBps / 10_000) : null;
// ...
- const distance = s0 && price ? ((toNumber(price, 8) / toNumber((s0 * BigInt(s.barrierBps)) /
↳ 10_000n, 8)) - 1) * 100 : null;
+ const distance = s0 && price ? barrierBufferPct(toNumber(price, 8), toNumber((s0 *
↳ BigInt(s.barrierBps)) / 10_000n, 8)) : null;
```

Note Systems: Fixed in commit [4d4bc3e](#).

Cyfrin: Verified.

7.2.11 Analytics series table labels the per observation coupon as "Coupon p.a."

Description: The Table view of the Series book on the public analytics page labels a column "Coupon p.a." and fills it with `couponBps`, which NoteCore defines per observation, without annualising it. Before strike the same cell shows the per observation floor to cap range.

```
// web/app/analytics/src/book.ts:163
s.couponBps === null ? `${pct(s.couponFloorBps)} to ${pct(s.couponCapBps)}` : pct(s.couponBps),
```

```
// web/app/analytics/src/book.ts:171
const TABLE_HEAD = ['Series', 'Underlying', 'Status', 'Strike', 'Last price', 'To barrier', 'Coupon
↳ p.a.', 'Notional', 'Observations', 'Next observation'];
```

The app itself labels the same number "Coupon / obs" on the series page, and Portfolio annualises it with `annualisedCouponBps` and marks the result "p.a. est., net".

Impact: Anyone reading the analytics table (built by `npm run build:analytics` and served at `note.systems/analytics`, no access gate) sees an annual coupon that is the per observation rate. For every weekly series on testnet the column reads 44 times lower than the annualised net figure (4.00% against 177.29%); for the 180 day series 65 it reads 1.53% against 2.64%. The error understates yield. The page has no deposit action and the app pages where deposits happen label the figure correctly, so the cost is a wrong public statistic. The table is not the default view: the book opens on stock cards.

Proof of Concept:

1. Open the analytics page, Series book, select Table and status All.
2. The "Coupon p.a." column shows 4.00% for series 64 (weekly observations).
3. `/series/64` in the app shows "Coupon / obs 4.00%"; `/portfolio` for a holder shows "177.3% p.a. est., net".

Recommended Mitigation: Head the column with what it holds, as the app's series page does. If an annual figure is wanted on this page, add it as a separate column from `annualisedCouponBps` with an `est.` marker.

In `web/app/analytics/src/book.ts`, rename the seventh `TABLE_HEAD` entry:

```
-const TABLE_HEAD = ['Series', 'Underlying', 'Status', 'Strike', 'Last price', 'To barrier', 'Coupon
↳ p.a.', 'Notional', 'Observations', 'Next observation'];
+const TABLE_HEAD = ['Series', 'Underlying', 'Status', 'Strike', 'Last price', 'To barrier', 'Coupon /
↳ obs', 'Notional', 'Observations', 'Next observation'];
```

Note Systems: Fixed in commit [8e86802](#).

Cyfrin: Verified.

7.2.12 Desk withdraw card compares the amount with idle assets instead of the idle left after queue reservations

Description: The Desk withdraw card chooses between "paid from idle assets now" and "queued" by comparing the amount with `Desk.idleAssets`. The Desk redeems directly only up to idle assets minus the NAV reserved for open withdrawal requests, so a withdrawal that fits in idle assets but not in the unreserved part is presented as instant and sent as `requestWithdraw`.

The page passes the raw idle balance into the card:

```
// web/app/src/routes/Desk.tsx:99
<DepositCard nav={nav} shareDecimals={d.shareDecimals} idle={d.idleAssets}
↳ minHold={d.minHoldSeconds} /* ... */ />
```

The card derives `queued` from it and shows it as the threshold:

```
// web/app/src/routes/Desk.tsx:345-352
const idleN = toNumber(idle, 6);
// ...
const queued = !isDeposit && !exitBlocked && n * nav > idleN;
```

```
// web/app/src/routes/Desk.tsx:379
<KV k="Idle assets" v={`${fmtNum(idleN, 0)} USDG`} hint="Withdrawals above idle assets are
  ↳ queued." />
```

When queued is false the confirm sheet is titled "Withdraw from Desk" and says:

```
// web/app/src/routes/Desk.tsx:404
: <>You redeem <strong>{fmtNum(n, 4)} shares</strong> for about <strong>{fmtNum(n * nav,
  ↳ 2)} USDG</strong>, paid from idle assets now.</>
```

The step builder then applies the real limit and falls through to the queue:

```
// web/app/src/chain/actions.ts:147
if (!forceQueue && assets <= idle && shares <= maxRedeem) return [{ label: 'Redeem Desk shares', /*
  ↳ ... */ functionName: 'redeem', /* ... */ }];
```

The Desk contract caps maxRedeem at free idle, which is idle assets minus previewRedeem of the escrowed shares (contracts/src/token/Desk.sol#L978-L983, _freeIdle at #L1129-L1133).

Impact: A depositor who withdraws an amount between free idle and idle assets confirms a sheet that promises USDG now, then signs requestWithdraw, which escrows the shares in the FIFO queue and pays nothing immediately. No funds are lost: the request fills later at fill-time NAV or can be cancelled, and the wallet prompt, the "Queue withdrawal" toast step and the "Exit available" row (maxWithdraw) show the queue. The gap exists whenever the queue holds requests; the testnet reserve is 0 today (idle 648,724.78 USDG at block 125718471).

Proof of Concept:

1. With open withdrawal requests on the Desk, open /desk, select Withdraw and enter a share amount worth more than idle assets minus the queue value and less than idle assets.
2. The card shows "Idle assets" above the amount and no "Queued" row; the button reads "Review withdrawal".
3. The sheet reads "You redeem ... shares for about ... USDG, paid from idle assets now." Press Confirm.
4. The wallet asks to sign requestWithdraw(shares, owner) on the Desk. After mining, the shares are escrowed and the USDG balance is unchanged.

The scripts below build a queue reserve on a fork. Start anvil --fork-url https://rpc.testnet.chain.robinhood.com --chain-id 46630 --retries 5 --timeout 45000, save the first as web/app/poc-desk-queue-setup.mjs and run FORK_RPC=http://127.0.0.1:8545 node poc-desk-queue-setup.mjs from web/app:

```
// Fork only. Builds a Desk queue reserve from real holders: every holder except OWNER, largest first,
  ↳ queues its
// whole balance with requestWithdraw (impersonated) until the reserve passes RESERVE_USDG.
import { createPublicClient, createWalletClient, http, parseAbi, parseAbiItem } from 'viem';
const FORK = process.env.FORK_RPC ?? 'http://127.0.0.1:8545';
const OWNER = (process.env.OWNER ?? '0x12fce5c890adf3681947fcf2d9fdcde96c5478dc').toLowerCase();
const RESERVE = BigInt(process.env.RESERVE_USDG ?? 380000) * 10n ** 12n; // shares have 12 decimals, NAV
  ↳ is about 1
const D = '0x814B6d8b37a22479Bee6228032716A99C54ac946';
const chain = { id: 46630, name: 'fork', nativeCurrency: { name: 'ETH', symbol: 'ETH', decimals: 18 },
  ↳ rpcUrls: { default: { http: [FORK] } }, contracts: { multicall3: { address:
  ↳ '0xcA11bde05977b3631167028862bE2a173976CA11' } } };
const pc = createPublicClient({ chain, transport: http(FORK, { timeout: 180_000 }) });
const wc = createWalletClient({ chain, transport: http(FORK, { timeout: 180_000 }) });
const abi = parseAbi(['function requestWithdraw(uint256 shares, address receiver) returns (uint256)',
  ↳ 'function balanceOf(address) view returns (uint256)']);
const latest = await pc.getBlockNumber();
```

```

const holders = new Set();
for (let from = 120437317n; from <= latest; from += 500_000n) { // addresses.testnet.json deployBlock
  const logs = await pc.getLogs({ address: D, event: parseAbiItem('event Transfer(address indexed from,
    ↳ address indexed to, uint256 value)'), fromBlock: from, toBlock: from + 499_999n > latest ? latest
    ↳ : from + 499_999n });
  for (const l of logs) holders.add(l.args.to.toLowerCase());
}
holders.delete(OWNER); holders.delete(D.toLowerCase());
↳ holders.delete('0x0000000000000000000000000000000000000000');
const list = [...holders];
const bals = await pc.multicall({ contracts: list.map((h) => ({ address: D, abi, functionName:
  ↳ 'balanceOf', args: [h] }))) });
const ranked = list.map((h, i) => [h, bals[i].result ?? 0n]).sort((a, b) => (b[1] > a[1] ? 1 : -1));
const pick = []; let acc = 0n;
for (const [h, b] of ranked) { if (acc >= RESERVE) break; acc += b; pick.push([h, b]); }
let ok = 0, fail = 0;
async function one([h, b]) {
  try {
    await pc.request({ method: 'anvil_impersonateAccount', params: [h] });
    await pc.request({ method: 'anvil_setBalance', params: [h, '0x56BC75E2D63100000'] });
    const hash = await wc.writeContract({ account: h, address: D, abi, functionName: 'requestWithdraw',
      ↳ args: [b, h] });
    (await pc.waitForTransactionReceipt({ hash })).status === 'success' ? ok++ : fail++;
  } catch { fail++; }
}
for (let i = 0; i < pick.length; i += 40) await Promise.all(pick.slice(i, i + 40).map(one));
console.log(`requestWithdraw from ${pick.length} holders: ${ok} mined, ${fail} failed (hold not elapsed
  ↳ or reverted)`);

```

Then save the second as web/app/poc-desk-queue-reserve.mjs and run SHARES=299999.9999 FORK_RPC=http://127.0.0.1:8545 node poc-desk-queue-reserve.mjs from web/app:

```

// Run from web/app against an anvil fork (FORK_RPC). Loads the app's ChainProvider, format.ts and
↳ actions.ts through
// Vite SSR with the testnet RPC rewritten to the fork, then reproduces the Desk withdraw card decision.
import { createServer } from 'vite';
const FORK = process.env.FORK_RPC ?? 'http://127.0.0.1:8545';
const OWNER = process.env.OWNER ?? '0x12fce5c890adf3681947fcf2d9fdcd96c5478dc';
process.env.VITE_NETWORK = 'testnet';
const vite = await createServer({ root: process.cwd(), configFile: false, logLevel: 'error', appType:
  ↳ 'custom', server: { middlewareMode: true, hmr: false, ws: false },
  plugins: [{ name: 'fork-rpc', enforce: 'pre', transform(code, id) {
    if (id.includes('/src/') && code.includes('https://rpc.testnet.chain.robinhood.com')) return
    ↳ code.replaceAll('https://rpc.testnet.chain.robinhood.com', FORK);
    if (id.endsWith('ChainProvider.ts')) return code.replace('transport: http(undefined, {', 'transport:
    ↳ http(undefined, { timeout: 180_000,');
  } ] ] });
const { ChainProvider } = await vite.ssrLoadModule('/src/data/ChainProvider.ts');
const { toNumber, fmtNum, fmtUsd } = await vite.ssrLoadModule('/src/lib/format.ts');
const { deskWithdrawSteps, toUnits } = await vite.ssrLoadModule('/src/chain/actions.ts');
const d = await new ChainProvider().getDesk(OWNER);
const amt = process.env.SHARES ?? toNumber(d.user.shares, d.shareDecimals).toFixed(4);
// Copied from web/app/src/routes/Desk.tsx (DepositCard): nav, idleN, n and the queued flag
const nav = toNumber(d.navPerShareWad, 18);
const n = Number(amt) > 0 ? Number(amt) : 0;
const idleN = toNumber(d.idleAssets, 6);
const queued = n * nav > idleN; // exitBlocked is false for this wallet (maxWithdraw > 0)
console.log(`idleAssets ${fmtUsd(d.idleAssets)} USDG, queue reserve ${fmtUsd(d.queuedAssets)} USDG,
  ↳ maxWithdraw(owner) ${fmtUsd(d.user.maxWithdraw)} USDG`);
console.log(`withdraw ${amt} shares (about ${fmtNum(n * nav, 2)} USDG): queued=${queued}`);
console.log(`sheet title: "${queued ? 'Request withdrawal' : 'Withdraw from Desk'}"; sheet text:
  ↳ ${queued ? "...escrowed in the FIFO queue..." : "You redeem ${fmtNum(n, 4)} shares for about
  ↳ ${fmtNum(n * nav, 2)} USDG, paid from idle assets now."}`);

```

```
const steps = await deskWithdrawSteps(OWNER, toUnits(amt, d.shareDecimals));
console.log(`deskWithdrawSteps sends: ${steps.map((s) => `${s.functionName}(${s.args.join(', ')}`)
↳ `${s.label}`").join(' ; ')}');
await vite.close();
process.exit(0);
```

```
idleAssets 408,084.39 USDG, queue reserve 307,760.21 USDG, maxWithdraw(owner) 100,324.18 USDG
withdraw 299999.9999 shares (about 300,000.00 USDG): queued=false
sheet title: "Withdraw from Desk"; sheet text: "You redeem 299,999.9999 shares for about 300,000.00
↳ USDG, paid from idle assets now."
deskWithdrawSteps sends: requestWithdraw(299999999900000000, 0x12fce5c890adf3681947fcf2d9fdcd96c5478dc)
↳ "Queue withdrawal"
```

Recommended Mitigation: Pass the idle balance net of the queue reservation into the card, the same figure `_freeIdle` uses (`DeskState.queuedAssets` already holds `convertToAssets(escrowedShares)`), and relabel the row to match.

1. In `Desk()` (`web/app/src/routes/Desk.tsx`), compute free idle and pass it as `idle`:

```
const nav = toNumber(d.navPerShareWad, 18);
// Desk._freeIdle: idle USDG minus the NAV reserved for open queue requests. Desk.maxRedeem stops
↳ here, so a direct
// redeem above it is sent as requestWithdraw.
const freeIdle = d.idleAssets > d.queuedAssets ? d.idleAssets - d.queuedAssets : 0n;
// ...
<DepositCard nav={nav} shareDecimals={d.shareDecimals} idle={freeIdle}
↳ minHold={d.minHoldSeconds} /* ... */ />
```

2. In `DepositCard`, rename the row and its hint:

```
- <KV k="Idle assets" v={`${fmtNum(idleN, 0)} USDG`} hint="Withdrawals above idle assets are
↳ queued." />
+ <KV k="Free idle assets" v={`${fmtNum(idleN, 0)} USDG`} hint="Idle USDG not reserved for
↳ open queue requests (Desk.maxRedeem). Withdrawals above it are queued." />
```

Note Systems: Fixed in commit [70fecfd1](#).

Cyfrin: Verified.

7.2.13 `runSteps` assumes an EOA signer, so the offered Safe connector cannot complete a write

Description: Every write in the app goes through `runSteps`, which assumes the connected account is an EOA that pays its own gas and whose `eth_sendTransaction` result is a chain transaction hash. The app offers a Safe connector for use inside `Safe{Wallet}`, where neither holds.

```
// web/app/src/wallet/wagmiCore.ts:43
safe({ allowedDomains: [/app\.safe\.global$/], shimDisconnect: false })),
```

First, the gas preflight refuses to start when the account's own ETH balance is zero. A Safe normally holds no ETH: its owners or a relayer pay for execution.

```
// web/app/src/wallet/tx.ts:200-201
const eth = await c.getBalance(c.wagmiConfig, { address: account, chainId: robinhoodChain.id
↳ }).then((b) => b.value).catch(() => null);
if (eth === 0n) throw new PreflightError('Not enough ETH for gas on Robinhood Chain. This wallet holds
↳ 0 ETH; top up and try again.');
```

Second, inside `Safe{Wallet}` the provider answers `eth_sendTransaction` with the `safeTxHash` (the Safe's EIP-712 message hash), which never appears on chain. `@safe-global/safe-apps-provider 0.18.6` sends the transaction through `sdk.txs.send` and returns `resp.safeTxHash` (`node_modules/@safe-global/safe-apps-provider/dist/provider.js#L76-L94`).

runSteps passes that value to waitForTransactionReceipt, which wagmi calls with timeout: 0, so the promise never settles:

```
// web/app/src/wallet/tx.ts:228-231
const hash = await c.writeContract(c.wagmiConfig, { ...base, gas } as never).catch((e: unknown) =>
  ↳ { throw new StepError(s, e); });
hashes.push(hash);
onStep?.(i, s, 'wait', hash);
const receipt = await c.waitForTransactionReceipt(c.wagmiConfig, { hash, chainId:
  ↳ robinhoodChain.id, confirmations: 1 });
```

Impact: A Safe on Robinhood Chain is accepted by NoteCore as a depositor, yet the app cannot complete any write for it, withdrawals and claims included. A Safe with no ETH is refused before any prompt; one with ETH gets its first step proposed, then the card waits forever on the unknown safeTxHash. No funds are at risk.

Proof of Concept:

1. Open the app as a Safe App in Safe{Wallet} on Robinhood Chain with a Safe that holds USDG and no ETH. Confirm any action (for example a Desk deposit). The toast reads "Not enough ETH for gas on Robinhood Chain. This wallet holds 0 ETH; top up and try again." and no Safe transaction is proposed.
2. Send some ETH to the Safe and repeat. Safe{Wallet} shows the approval transaction; after it is signed, the card stays on "Waiting for Approve USDG (1/2)...".

Recommended Mitigation: Skip the 0 ETH preflight for a Safe, and resolve the safeTxHash to the executing transaction's hash before waiting for a receipt. The hash lookup imports @safe-global/safe-apps-sdk, which wagmi's Safe connector already installs; add it to package.json at the locked version (9.1.0) so the import does not rely on hoisting. Batching approve and action into one Safe transaction also cuts the Safe flow to one signature.

1. In runSteps (web/app/src/wallet/tx.ts), skip the balance read for a Safe:

```
// Skipped for a Safe: its owners or a relayer pay the gas, and a Safe usually holds no ETH itself.
const isSafe = c.getAccount(c.wagmiConfig).connector?.type === 'safe';
const eth = isSafe ? null : await c.getBalance(c.wagmiConfig, { address: account, chainId:
  ↳ robinhoodChain.id }).then((b) => b.value).catch(() => null);
```

2. In web/app/src/wallet/wagmiCore.ts, add resolveTxHash, and in runSteps pass the writeContract result through c.resolveTxHash(sent) before waitForTransactionReceipt:

```
export async function resolveTxHash(hash: `0x${string}`): Promise<`0x${string}`> {
  if (getAccount(wagmiConfig).connector?.type !== 'safe') return hash;
  const { default: SafeAppsSDK } = await import('@safe-global/safe-apps-sdk');
  const sdk = new SafeAppsSDK({ allowedDomains: [/app\.safe\.global$/] });
  for (;;) {
    const d = await sdk.txs.getBySafeTxHash(hash).catch(() => null);
    if (d?.txHash) return d.txHash as `0x${string}`;
    if (d && (d.txStatus === 'CANCELLED' || d.txStatus === 'FAILED')) throw new Error(`The Safe
      ↳ transaction was ${d.txStatus.toLowerCase()}.`);
    await new Promise((r) => window.setTimeout(r, 5000));
  }
}
```

Note Systems: Fixed in commit [c8c183a](#).

Cyfrin: Verified.

7.2.14 A route chunk that fails to load unmounts the whole app because no error boundary wraps the lazy routes

Description: Every route is a `React.lazy` import of a content-hashed chunk, and the only boundary around the routes is a `Suspense`, which handles loading but not failure. When the chunk request fails, `React.lazy` throws during render, nothing catches it, and React unmounts the entire root.

```
// web/app/src/App.tsx:9-14
const Notes = lazy(() => import('./routes/Notes'));
const Series = lazy(() => import('./routes/Series'));
const Portfolio = lazy(() => import('./routes/Portfolio'));
const Desk = lazy(() => import('./routes/Desk'));
// ...
```

```
// web/app/src/components/Shell.tsx:137-141
<div className="shell__content">
  {IS_TESTNET && <TestnetFaucet />}
  <Suspense fallback={<RouteFallback />}>
    <Outlet />
  </Suspense>
```

The common trigger is a deploy: a tab that loaded the previous `index.html` still references the previous chunk names. If the host no longer serves those files, the first visit to a route whose chunk changed gets a 404. A network drop during a route's first visit has the same result. The deploy-freshness banner that offers a reload lives inside the tree that unmounts, so it never appears.

The chain data provider has a related gap: it caches the import promise with `??=`, so a rejected `ChainProvider` import is kept for the life of the page and every later read fails with it.

```
// web/app/src/data/index.tsx:24-29
class LazyChainProvider implements DataProvider {
  private p: Promise<DataProvider> | null = null;
  private get(): Promise<DataProvider> {
    this.p ??= import('./ChainProvider').then((m) => new m.ChainProvider());
    return this.p;
  }
}
```

Impact: After a deploy that changes a route's code, any open tab that navigates to that route for the first time shows a blank page with no message, and the rail, header and wallet chip disappear with it. Only a manual reload recovers the tab. A user with a transaction in flight loses the progress and result toasts, though the transaction itself still proceeds in the wallet and on chain. No funds are at risk. Whether old chunks return 404 depends on the hosting setup, which is not in this repository; most static hosts serve only the current deployment's files.

Proof of Concept:

1. Run `VITE_NETWORK=testnet npm run build` in `web/app`.
2. Save the script below as `web/app/poc-stale-chunk.mjs` and run `node poc-stale-chunk.mjs`.
3. Open `http://localhost:4175/#/notes`, wait for the page, then click Desk. The page goes blank and stays blank after clicking Notes, and the console shows `TypeError: Failed to fetch dynamically imported module: .../assets/Desk-<hash>.js`.

```
// Serves a copy of dist/ and deletes the Desk route chunk right after index.html has been served, which
↳ is what a
// redeploy does to a tab that loaded the previous build. Build first: VITE_NETWORK=testnet npm run
↳ build
import { createServer } from 'node:http';
import { cpSync, existsSync, readdirSync, readFileSync, rmSync } from 'node:fs';
import { extname, join } from 'node:path';
const DIR = join(process.cwd(), '.poc-dist');
rmSync(DIR, { recursive: true, force: true }); cpSync('dist', DIR, { recursive: true });
```

```
const TYPES = { '.html': 'text/html', '.js': 'text/javascript', '.css': 'text/css', '.json':
  ↳ 'application/json', '.svg': 'image/svg+xml', '.woff2': 'font/woff2' };
const desk = readdirSync(join(DIR, 'assets')).find((f) => /^Desk-.*\.js$/ .test(f));
createServer((req, res) => {
  const p = join(DIR, decodeURIComponent(req.url.split('?')[0]) === '/' ? 'index.html' :
    ↳ decodeURIComponent(req.url.split('?')[0]));
  if (!existsSync(p)) { res.writeHead(404); res.end(); console.log('404', req.url); return; }
  res.writeHead(200, { 'content-type': TYPES[extname(p)] ?? 'application/octet-stream' });
  ↳ res.end(readFileSync(p));
  if (p.endsWith('index.html') && existsSync(join(DIR, 'assets', desk))) { rmSync(join(DIR, 'assets',
    ↳ desk)); console.log(`served index.html, removed assets/${desk} (simulated redeploy)`); }
}).listen(4175, () => console.log('open http://localhost:4175/#/notes, wait for the page, then click
  ↳ Desk'));
```

```
served index.html, removed assets/Desk-DtqQSKUC.js (simulated redeploy)
404 /assets/Desk-DtqQSKUC.js
root children after clicking Desk: 0 errors: ["Failed to fetch dynamically imported module:
  ↳ http://localhost:4175/assets/Desk-DtqQSKUC.js"]
```

Recommended Mitigation: Wrap the routed outlet in an error boundary, keyed by path, that keeps the shell mounted and offers a reload. Drop a rejected ChainProvider import so the next read retries it. Keeping the previous deploy's assets/ on the host for a while also avoids the 404 in the first place.

1. In web/app/src/components/Shell.tsx, wrap the outlet in Shell (importing Component from React and EmptyState from ./ui):

```
<RouteErrorBoundary key={loc.pathname}>
  <Suspense fallback={<RouteFallback />}>
    <Outlet />
  </Suspense>
</RouteErrorBoundary>
```

2. In the same file, add the boundary. It tells a failed chunk load apart from other render errors:

```
class RouteErrorBoundary extends Component<{ children: ReactNode }, { error: unknown }> {
  state = { error: null as unknown };
  static getDerivedStateFromError(error: unknown) { return { error }; }
  render() {
    if (this.state.error === null) return this.props.children;
    const chunk = /dynamically imported module|Importing a module script failed|error loading
      ↳ dynamically imported module/i.test(String((this.state.error as Error)?.message ??
      ↳ this.state.error));
    return (
      <EmptyState
        title={chunk ? 'A newer version of the app is available' : 'This page could not be displayed'}
        body={chunk ? 'This tab is running an older build whose files are no longer on the server.'
          ↳ 'Reload to continue.' : 'Reload to try again.'}
        action={<button type="button" className="btn btn--primary" onClick={() =>
          ↳ location.reload()}>Reload</button>}
      </EmptyState>
    );
  }
}
```

3. In LazyChainProvider.get() in web/app/src/data/index.tsx, clear the cached promise on rejection:

```
- this.p ??= import('./ChainProvider').then((m) => new m.ChainProvider());
+ this.p ??= import('./ChainProvider').then((m) => new m.ChainProvider(), (e: unknown) => { this.p =
  ↳ null; throw e; });
```

Note Systems: Fixed in commit [08d82e5](#).

Cyfrin: Verified.

7.2.15 ConfirmSheet re-runs its focus effect on every parent render and moves keyboard focus back to Cancel each second

Description: The focus and keyboard effect in ConfirmSheet lists the whole props object `p` as a dependency, so it re-runs on every parent render. Each run calls the cleanup (which restores focus to the element that opened the sheet) and then focuses the data-autofocus Cancel button again.

```
// web/app/src/components/Sheet.tsx:38-58
useEffect(() => {
  if (!p.open) return;
  const prev = document.activeElement as HTMLElement | null;
  const el = ref.current;
  const first = el?.querySelector<HTMLElement>('[data-autofocus]') ??
    ↪ el?.querySelector<HTMLElement>('button');
  first?.focus();
  // ...
  return () => { document.removeEventListener('keydown', onKey); if (main) main.style.overflow = '';
    ↪ prev?.focus?.(); };
}, [p.open, p]);
```

```
// web/app/src/components/Sheet.tsx:82
<button type="button" className="btn btn--secondary" onClick={p.onClose}
  ↪ data-autofocus>Cancel</button>
```

Every caller passes a new props object on each render, and the parents of almost every sheet re-render once per second through `useNow()` (Series, Desk, Bonds, Stake, Buyback, Portfolio, the governance routes and the automation card). For example, the COUPON card:

```
// web/app/src/routes/Series.tsx:180
const canDeposit = subscriptionOpen(s, useNow());
```

```
// web/app/src/routes/Series.tsx:219-220
<ConfirmSheet
  open={open} onClose={() => setOpen(false)}>
```

Impact: On every confirm sheet, a keyboard-only user who presses Tab to reach the Confirm button finds focus back on Cancel within the next clock tick. Pressing Enter more than about 0.5 s after the Tab closes the sheet instead of confirming. Screen reader users hear focus move from the sheet to the trigger button and back twice per second. Mouse and touch users are not affected. The consent step fails closed: nothing is signed and no funds are involved.

Proof of Concept:

1. Build and serve the app (VITE_NETWORK=testnet npm run build, then serve dist/), connect a wallet holding USDG and open `#/series/<id>` for a series in subscription.
2. Type an amount in the COUPON card and click Review purchase. The confirm sheet opens with focus on Cancel.
3. Press Tab once. Focus moves to Confirm deposit.
4. Wait one second and press Enter. The sheet closes as if Cancel had been pressed; no wallet prompt appears.

Recommended Mitigation: In ConfirmSheet (web/app/src/components/Sheet.tsx), read `onClose` through a ref and run the effect only when open changes, so a parent re-render no longer resets focus:

```
+ const onClose = useRef(p.onClose);
+ onClose.current = p.onClose;
// ...
```

```

-     if (e.key === 'Escape') { e.preventDefault(); p.onClose(); }
+     if (e.key === 'Escape') { e.preventDefault(); onClose.current(); }
    // ...
-   }, [p.open, p]);
+   }, [p.open]);

```

Note Systems: Fixed in commit [cb5559a](#).

Cyfrin: Verified.

7.2.16 runSteps accepts the receipt of a cancelling transaction as confirmation of the step it replaced

Description: runSteps treats a step as confirmed whenever waitForTransactionReceipt returns a receipt with status === 'success', without passing onReplaced or checking that the receipt belongs to the hash it sent.

```

// web/app/src/wallet/tx.ts:228-235
const hash = await c.writeContract(c.wagmiConfig, { ...base, gas } as never).catch((e: unknown) =>
  ↳ { throw new StepError(s, e); });
hashes.push(hash);
onStep?.(i, s, 'wait', hash);
const receipt = await c.waitForTransactionReceipt(c.wagmiConfig, { hash, chainId:
  ↳ robinhoodChain.id, confirmations: 1 });
if (receipt.status !== 'success') throw new Error(`${s.label} reverted on-chain (${hash.slice(0,
  ↳ 10)})`);
receipts.push({ hash, logs: ((receipt as { logs?: TxReceiptLogs['logs'] }).logs ?? []) as
  ↳ TxReceiptLogs['logs'] });
onStep?.(i, s, 'done', hash);
confirmed.push(s);

```

When the watched hash is dropped and another transaction at the same nonce is mined, viem resolves with the replacement's receipt instead of rejecting. A wallet "Cancel" is a 0-value self-transfer at the same nonce, so viem reports reason: 'cancelled' and still returns a success receipt. The step is counted as confirmed, the flow continues or ends, and useTx.send shows a "confirmed" toast with an explorer link built from the original hash.

A wallet "Speed up" (same call, higher fee) counts correctly, but hashes, receipts[] .hash and lastHash keep the original hash, which is never mined, so the explorer link points to a transaction that does not exist.

Impact: A user who cancels a pending action in the wallet sees a "confirmed" toast linking a hash that never lands on chain, though no funds move. A cancelled trailing revoke in a COUPON-leg bond purchase leaves the operator grant in place under a success toast. Robinhood Chain includes transactions almost at once, so the cancel wins only when the original is held back, for example behind an earlier pending nonce.

Proof of Concept:

1. On an Anvil fork of the testnet with automine off, open #/desk, enter 5 USDG and confirm the deposit.
2. While "Waiting for Deposit to Desk" is shown, press "Cancel" in the wallet, then mine a block.
3. The wallet's 0-value self-transfer mines at the same nonce. The app toasts "Desk deposit confirmed" with a link to the original hash, which has no receipt on chain.
4. The Desk share balance and the USDG balance are unchanged.

Recommended Mitigation: Pass onReplaced and fail the step when the replacement is a cancel or a different transaction. For a repriced transaction, keep the step and record the hash that was mined.

In runSteps (web/app/src/wallet/tx.ts), replace the receipt wait and the bookkeeping after it:

```

-     const receipt = await c.waitForTransactionReceipt(c.wagmiConfig, { hash, chainId:
  ↳ robinhoodChain.id, confirmations: 1 });
+     // viem resolves with the receipt of whatever transaction mined at this nonce. A wallet "speed up"
  ↳ re-sends the same
+     // call (repriced) and still counts; a cancel or any other replacement means this step never ran.
+     const replaced: { reason?: 'replaced' | 'repriced' | 'cancelled' } = {};

```

```

+   const receipt = await c.waitForTransactionReceipt(c.wagmiConfig, { hash, chainId:
↳ robinhoodChain.id, confirmations: 1, onReplaced: (r) => { replaced.reason = r.reason; } });
+   if (replaced.reason === 'cancelled' || replaced.reason === 'replaced') throw new StepError(s, new
↳ Error(`${s.label} was ${replaced.reason} in the wallet, so it did not run.`));
+   if (receipt.status !== 'success') throw new Error(`${s.label} reverted on-chain (${hash.slice(0,
↳ 10)})`);
-   receipts.push({ hash, logs: ((receipt as { logs?: TxReceiptLogs['logs'] }).logs ?? []) as
↳ TxReceiptLogs['logs'] });
-   onStep?.(i, s, 'done', hash);
+   const mined = receipt.transactionHash;
+   if (mined !== hash) hashes[hashes.length - 1] = mined;
+   receipts.push({ hash: mined, logs: ((receipt as { logs?: TxReceiptLogs['logs'] }).logs ?? []) as
↳ TxReceiptLogs['logs'] });
+   onStep?.(i, s, 'done', mined);

```

Note Systems: Fixed in commit [04ef062](#).

Cyfrin: Verified.

7.2.17 Stock Token bond markets store the 6 decimal Treasury.valueOf result as a WAD, so the token value renders 1e12 too small

Description: ChainProvider.getBonds stores Treasury.valueOf(token, 1e18) as quoteUsdWad for a Stock Token market without rescaling it. valueOf returns quote units (USDG, 6 decimals), while every consumer reads quoteUsdWad as an 18 decimal WAD.

```

// web/app/src/data/ChainProvider.ts:621-630
tr ? this.mc(stockTokens.map((t) => ({ address: tr, abi: treasuryAbi, functionName: 'valueOf',
↳ args: [t as Address, WAD] } as const))) : Promise.resolve([] as unknown[]),
// ...
return [t, { sym, dec, usdWad: big(stockVals[i]) }] as const;

```

```

// web/app/src/data/ChainProvider.ts:647-651
const quoteUsdWad = kind === 'USDG' ? WAD : kind === 'STOCK' ? (sm?.usdWad ?? 0n) :
↳ (legVal.get(legSeriesId) ?? 0n);
// ...
const derived = quotePerNote !== undefined && quotePerNote > 0n && quoteUsdWad > 0n ?
↳ (quoteUsdWad * WAD) / quotePerNote : 0n;
const priceWad = p > 0n ? p : derived;

```

```

// web/app/src/routes/Bonds.tsx:52-60
const unitUsd = (m: BondMarket) => (m.quoteUsdWad !== undefined ? toNumber(m.quoteUsdWad, 18) :
↳ m.quoteToken.kind === 'COUPON_LEG' ? 1 : (quoteUsd.get(m.quoteToken.symbol) ?? 0));
// ...
const unitsPerNote = (m: BondMarket) => ratio(pricePerNote(m), unitUsd(m));

```

In the contracts, Treasury.quoteValue returns amount * price * 10^{QUOTE_DECIMALS} / 10^(decimals + feedDecimals), and BondDepository.quoteValueWad multiplies the valueOf result by 10^(18 - QUOTE_DECIMALS) before using it as a WAD (contracts/src/token/BondDepository.sol#L523).

On testnet Treasury.valueOf(AAPL, 1e18) returns 342750000 (342.75 USDG), which the app reads as 3.4275e-10 USDG.

Impact: A Stock Token bond buyer sees AAPL valued at 0.00 USDG and a "Price per NOTE" subtitle of about 145,878,920 AAPL instead of 0.000146 AAPL. "You receive" and the calldata come from payoutFor and the token's own decimals, so the buyer signs the right amount. If payoutFor reverts (for example QuoteNotLive), priceWad falls back to derived, shows "You receive 0.00 NOTE" and the purchase stops before any signature. No Stock Token market exists on testnet today (40 COUPON leg, 2 USDG); the display breaks once governance creates one.

Proof of Concept:

1. Create a Stock Token bond market on a testnet fork (commands below), then open `#/bonds` in a build pointed at the fork and select the AAPL market.
2. The card shows the subtitle "145,878,920.495988 AAPL" and the hint "Treasury values AAPL at 0.00 USDG per token."
3. Type 1. "You receive" shows 6,855.00 NOTE, which matches `payoutFor(42, 1e18)` on chain.

Commands for step 1 (the Timelock owns BondDepository; the deployer is the feed keeper):

[illegible]

Recommended Mitigation: In `ChainProvider.getBonds`, rescale the `valueOf` result from quote decimals to a WAD (`scaleDecimals` and `USDG_DECIMALS` are already in scope):

```
-      return [t, { sym, dec, usdWad: big(stockVals[i]) }] as const;
+      // Treasury.valueOf returns quote units (USDG, 6 decimals); scale to WAD as
↳ BondDepository.quoteValueWad does.
+      return [t, { sym, dec, usdWad: scaleDecimals(big(stockVals[i]), USDG_DECIMALS, 18) }] as const;
```

Note Systems: Fixed in commit [e03aedc](#).

Cyfrin: Verified.

7.2.18 Two flows from one wallet can both pass simulation against the same allowance and the second reverts on chain

Description: The only guard against overlapping flows is `busy`, which is local state of each `useTx()` instance. Two components on one page can run approve-then-act flows for the same token and spender at the same time, and each step is simulated against the allowance before the other flow's transactions mine.

```
// web/app/src/wallet/tx.ts:273-285
export function useTx(): UseTx {
  // ...
  const [busy, setBusy] = useState(false);
  // ...
  setBusy(true); setPhase('Preparing...');
}
```

On a series page the COUPON card and the SHIELD card each call `useTx()` (`web/app/src/routes/Series.tsx#L176`, `web/app/src/routes/Series.tsx#L246`) and each button is disabled only by its own `tx.busy`. Both flows spend USDG through `NoteCore`. `ensureAllowance` reads the allowance when the steps are built and skips the approval if it already covers the amount:

```
// web/app/src/wallet/tx.ts:73-79
const [cur, bal, dec] = await Promise.all([rd<bigint>('allowance', [owner, spender]),
↳ rd<bigint>('balanceOf', [owner]), rd<number>('decimals').catch(() => 18)]);
// ...
if (cur >= amount) return null;
```

```
return { label: `Approve ${symbol}`, address: token, abi: erc20AllowanceAbi, functionName: 'approve',  
  ↪ args: [spender, amount] };
```

If the SHIELD flow is started while the COUPON flow's 10 USDG approval has mined but its deposit has not, the SHIELD flow sees 10 USDG of allowance, skips its USDG approval, and its depositShield simulation passes. Whichever deposit mines first consumes the allowance and the other reverts on chain. Both simulations run before either deposit is signed (web/app/src/wallet/tx.ts#L206-L209), so the simulate-before-write check does not catch it. Two tabs on the Desk page fail the same way: each approves the exact amount, approve overwrites, and the second deposit reverts.

Impact: The user pays gas for a reverted deposit and sees "SHIELD deposit failed · Token allowance too low: allowed 0, need 3688248 (base units). Approve again." No funds are lost and a retry works. It needs the user to start a second flow while the first is still waiting for a prompt, which the two cards side by side on one page make easy.

Proof of Concept:

1. Open a series in Subscription (for example #/series/76) with a wallet that has no USDG allowance to NoteCore.
2. In the COUPON card enter 10 and confirm. Confirm "Approve USDG"; leave the "Deposit COUPON" prompt open.
3. In the SHIELD card enter 0.1 and confirm. Confirm "Approve AAPL"; the SHIELD flow has no USDG approval step because 10 USDG is already approved.
4. Confirm "Deposit COUPON", then "Deposit SHIELD". The COUPON deposit mines; the SHIELD deposit is mined with status 0 and the app shows "SHIELD deposit failed".

Recommended Mitigation: Allow one flow per account at a time in runSteps, so a second flow is refused before its first prompt. This covers every card and route in one tab. Flows in two tabs would also need a cross-tab lock such as navigator.locks where the host allows it.

In web/app/src/wallet/tx.ts, move the body of runSteps into runStepsUnlocked (with the onStep type pulled out as OnStep) and wrap it in a per-account lock:

```
const inFlight = new Set<string>();  
  
export async function runSteps(steps: TxStep[], account: `0x${string}`, onStep?: OnStep):  
  ↪ Promise<TxResult> {  
  const key = account.toLowerCase();  
  if (inFlight.has(key)) throw new PreflightError('Another transaction from this wallet is still in  
    ↪ progress. Wait for it to finish, then try again.');
```

```
  inFlight.add(key);  
  try { return await runStepsUnlocked(steps, account, onStep); } finally { inFlight.delete(key); }  
}
```

Note Systems: Fixed in commit [2b6dcaa](#).

Cyfrin: Verified.

7.2.19 Series pages never render a deferred observation because the app does not read the oracle adapter state

Description: ChainProvider.outcome() classifies each observation from Observed logs and the clock only, so it never returns 'deferred'. An observation that NoteCore has deferred (the oracle adapter found no qualifying price) is shown as "Awaiting crank", although the app already has a 'deferred' outcome with its own label.

```
// web/app/src/data/ChainProvider.ts:330-350
private outcome(v: SeriesView, status: SeriesStatus, index: number, closeTs: number, now: number,
  o?: { barrierHolds: boolean; autocall: boolean; status: number }, logsOk = true):
  ↳ ObservationOutcome {
  // ...
  if (status === 'Autocalled' || status === 'Matured') return 'pending';
  return now > closeTs + 300 ? 'due' : 'pending';
}
```

That label is never reached:

```
// web/app/src/lib/format.ts:191
case 'deferred': return 'Deferred';
```

The crank card for the same state tells the user the core is paused or the feed has not posted yet:

```
// web/app/src/components/Automation.tsx:54
? 'A scheduled close has passed but the price cannot be recorded yet (the core is paused or the
  ↳ feed has not posted). observe becomes runnable once it can.'
```

NoteCore emits ObservationDeferred and leaves nextObs unchanged when the adapter cannot resolve the close (contracts/src/core/NoteCore.sol#L876-L880).

Impact: Holders of a series with a deferred observation see "Awaiting crank", status "Observing" and a disabled Run button with copy suggesting a short wait. They are not told that the coupon, autocall and barrier outcome of that close now wait on a governance force or cancel, which takes at least the deferral window plus the force delay. No transaction follows from this display and no funds move. Deferral needs the primary and backup feeds to be silent for the whole pre-close window, which the production maxPreCloseLag of 24 hours makes rare.

Proof of Concept:

1. Fork testnet from a block before 1 Oct 2026 with no COIN feed round inside the pre-close window, move chain time one hour past the next close of series 6 (1790798400) and call observe(6) with these commands:

```
anvil --fork-url https://rpc.testnet.chain.robins.com --chain-id 46630 --retries 5 --timeout 45000 &
R=http://127.0.0.1:8545; K=0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266
cast rpc evm_setNextBlockTimestamp 1790802000 --rpc-url $R; cast rpc evm_mine --rpc-url $R
cast rpc anvil_impersonateAccount $K --rpc-url $R
cast send 0xE47BBec63D836643F8FBC50e61011c9eb278a55e 'observe(uint256)' 6 --from $K --unlocked
↳ --rpc-url $R
```

2. NoteCore emits ObservationDeferred(6, 2, 1790798400) and OracleAdapter.observation(feed, 1790798400) has state 1 (Deferred).
3. Open /series/6 with the page clock at chain time. Row 2 shows "Awaiting crank", the status pill is "Observing" and the crank card shows "Observation pending" with Run disabled; the word "Deferred" appears nowhere.

Recommended Mitigation: For a Live series whose next close has passed, read OracleAdapter.observation(feed, effectiveCloseOf(close)) and return 'deferred' for that index when the state is Deferred or ForcePending. This adds two reads per Live series with a passed close.

1. In web/app/src/chain/protocolAbi.ts, add the adapter getter to oracleAdapterAbi:

```
+ 'function observation(address feed, uint40 closeTs) view returns ((uint8 state, uint40 deferredSince,
  ↳ uint40 forceExecuteAfter, uint256 price))',
```

2. In `web/app/src/data/ChainProvider.ts` (importing `oracleAdapterAbi`), read the adapter state of the next unprocessed close in `getSeries` and use it when building the observations:

```
// OracleAdapter.observation state of the next unprocessed close: Deferred (1) or ForcePending (2)
↳ means observe()
// found no qualifying price and the close waits for a retry or for governance to force or cancel
↳ it.
let deferredIndex = -1;
const adapter = OPTIONAL_ADDRESSES.oracleAdapter;
if (status === 'Live' && adapter && v.nextObs < v.observations.length &&
↳ Number(v.observations[v.nextObs]) <= now) {
  try {
    const eff = await readContract(this.client, { address: ADDRESSES.noteCore!, abi: noteCoreAbi,
↳ functionName: 'effectiveCloseOf', args: [Number(v.observations[v.nextObs])] });
    const rec = await readContract(this.client, { address: adapter, abi: oracleAdapterAbi,
↳ functionName: 'observation', args: [v.feed, Number(eff)] });
    if (rec.state === 1 || rec.state === 2) deferredIndex = v.nextObs;
  } catch { /* adapter unreadable: fall back to the calendar outcome */ }
}
// ...
return { index, closeTs, price: /* ... */, outcome: index === deferredIndex ? 'deferred' :
↳ this.outcome(v, status, index, closeTs, now, o, logsOk) };
}
```

Note Systems: Fixed in commit [548cb69](#).

Cyfrin: Verified.

7.2.20 Lock form rounds the unlock down to a Thursday boundary, so a 1 week lock can end within seconds and cannot vote on gauges

Description: The Lock form computes the unlock as `weekFloor(now + weeks * WEEK)`, which rounds down to Thursday 00:00 UTC, so a lock chosen as `N` weeks ends between `N - 1` and `N` weeks from now. At the 1 week setting it ends at the next Thursday 00:00 UTC, anywhere from one second to seven days away.

The minimum-duration guard cannot fire. `minEnd` uses the same round-down, and its `+ WEEK` branch runs only when `weekFloor(now + WEEK) <= now`, which is never true. So `minEnd` equals the 1 week unlock and the "at least one full week away" error never shows, while the header pill promises "min 1 week".

```
// web/app/src/routes/GovernanceLock.tsx:45-60
const chosenEnd = weekFloor(now + weeks * WEEK);
const end = mode === 'increase' ? (lock?.end ?? chosenEnd) : chosenEnd;
// ...
const minEnd = weekFloor(now + VE_MINTIME) + (weekFloor(now + VE_MINTIME) <= now ? WEEK : 0);
// ...
if (end < minEnd) return 'Unlock must be at least one full week away. (UnlockTimeInPast)';
```

`veNOTE` accepts any future week boundary, but gauge voting requires a lock that ends after the next boundary, and the 1 week lock ends exactly at it. The Extend tab uses the same `chosenEnd`. The "4 y" chip (209 weeks) rounds to a date past four years during the first three days after each boundary and shows the "more than four years away" error.

Impact: A user who picks the shortest lock gets one that can end within hours while the page promises at least a week, and the review sheet shows only a date, which does not reveal it. That lock can never vote on gauges and may carry no weight on later proposals. The NOTE stays withdrawable at the earlier unlock.

Proof of Concept:

1. On a testnet build against a fork, connect a wallet holding NOTE with no lock and open `#/governance/lock` on a Wednesday at 23:00 UTC.
2. Enter 10000 NOTE and set "Lock for" to 1. The pill reads "Max 4 y · min 1 week", the hint reads "1 weeks · unlock" and no error appears.

3. Review and confirm. The approval and `createLock` succeed; the lock ends one hour later at Thursday 00:00 UTC.
4. Open `#/governance/gauges`: it shows "Lock needed. Gauge votes need a lock that outlasts the next week boundary".

Recommended Mitigation: Round the chosen unlock up to the first week boundary at least `weeks` weeks away, capped at the last boundary inside four years, and build `minEnd` the same way. Show the remaining time next to the unlock date in the hint and review sheet.

```
export const weekCeil = (t: number) => Math.ceil(t / WEEK) * WEEK;

- const chosenEnd = weekFloor(now + weeks * WEEK);
+ const chosenEnd = Math.min(weekCeil(now + weeks * WEEK), weekFloor(now + VE_MAXTIME));
// ...
- const minEnd = weekFloor(now + VE_MINTIME) + (weekFloor(now + VE_MINTIME) <= now ? WEEK : 0);
+ const minEnd = weekCeil(now + VE_MINTIME);
```

Note Systems: Fixed in commit [597ef59](#).

Cyfrin: Verified.

7.2.21 The bundle defines no `Buffer` global, so every transaction from a Coinbase Wallet mobile app session fails before it reaches the wallet

Description: `@coinbase/wallet-sdk` encodes WalletLink requests with the Node `Buffer` global, and the Vite build neither polyfills nor defines it. Every `eth_sendTransaction`, `personal_sign` and `eth_signTypedData*` from a WalletLink session therefore throws `ReferenceError: Buffer is not defined` inside the page. WalletLink is how the SDK talks to the Coinbase Wallet mobile app, and the app enables it:

```
// web/app/src/wallet/wagmiCore.ts:41
coinbaseWallet({ appName: APP_META.name, appLogoUrl: APP_META.icons[0], preference: 'all', version:
  ↳ '4' }),
```

Connecting and switching chain do not touch `Buffer`, so the session connects and shows the right network, then fails at the first write. Every write in the app goes through one `writeContract` call in `runSteps` (`tx.ts#L228`), so all of them fail, exits included.

The `buffer` package is already in the bundle as a dependency of other wallet libraries, but nothing assigns it to `globalThis`. The Coinbase Wallet extension, the Smart Wallet and the Coinbase Wallet in-app browser are not affected.

Impact: A user who connects the Coinbase Wallet mobile app through the Coinbase Wallet row cannot perform any write: faucet, deposits, subscriptions, redemptions and withdrawals all fail with the toast "Buffer is not defined". No request reaches the phone and nothing is spent, and the same account still works through the in-app browser, the extension or another wallet.

Proof of Concept:

1. Build and serve the testnet app and open it in a desktop browser without the Coinbase Wallet extension.
2. Click Connect, then "Coinbase Wallet". In the popup choose the Coinbase Wallet app and scan the QR code with the phone.
3. The header shows the account as connected on Robinhood Chain Testnet.
4. Click "Get 1,000 USDG" in the faucet. The toast "Faucet 1,000 USDG failed / Buffer is not defined" appears within a second and the phone shows no request.

Recommended Mitigation: Add buffer 6.0.3 as a dependency and install it as `globalThis.Buffer` in `wagmi-Core.ts`, which loads before any connector runs. Also add `buffer`, `base64-js` and `ieee754` to the wallet chunk pattern in `vite.config.ts`, so the eagerly loaded `wagmiCore` chunk does not pull in the 1.8 MB wallet SDK chunk.

```
import { Buffer } from 'buffer';
// ...
const g = globalThis as { Buffer?: unknown };
if (g.Buffer === undefined) g.Buffer = Buffer;
```

Note Systems: Fixed in commit [4f191f6](#).

Cyfrin: Verified.

7.2.22 `describeError` reads the calldata printed in a viem error message as revert data, so a wallet on another network is reported as a contract revert

Description: When an error has no structured revert payload, `revertData` scans the message of every node in the error chain for a hex string shaped like revert data, and viem's transaction errors print the step's own calldata in their message. A node's details is read first, but errors viem raises itself, such as `ChainMismatchError` when the wallet is on another network, have none. `revertData` then returns the calldata, no custom error matches its selector, and `describeError` reports `<Contract>.<function> reverted (<selector>)`.

```
// web/app/src/chain/revert.ts:32-40
for (const n of all) {
  const txt = n.details ?? n.message ?? '';
  if (!/revert(data/i.test(txt)) continue;
  // Selector plus whole 32-byte words only; addresses (40 hex) and hashes (64 hex) in the text are
  ↪ not revert data.
  for (const m of txt.matchAll(/0x[0-9a-fA-F]{8,}/g)) {
    const hex = m[0].length - 2;
    if (hex !== 40 && hex !== 64 && (hex - 8) % 64 === 0) return m[0] as `0x${string}`;
  }
}
```

The app checks the network only before the first step of a flow, so a wallet that switches network between steps fails in viem's own chain check. Wallet disconnects and account switches reach the user as raw library text for the same reason.

Impact: A user whose wallet changes network between the two prompts of a two step flow (Desk deposit, Stake, Bonds, SHIELD or COUPON deposit, lock creation) is told a contract reverted when nothing was sent, and may abandon a valid deposit or report a contract fault. No funds move, and the approval from step 1 is reused on the next attempt.

Proof of Concept:

1. Connect a wallet on Robinhood Chain testnet, open `#/desk`, enter 5 in the deposit card, review and confirm.
2. Confirm "Approve USDG" in the wallet and wait for it to mine.
3. Before the "Deposit to Desk" prompt appears, switch the wallet to Ethereum mainnet.
4. No prompt appears. The toast reads "Desk deposit failed · Desk.deposit reverted (0x6e553f65)", and 0x6e553f65 is the selector of `deposit(uint256,address)`.

Recommended Mitigation: Read revert data only from a node's details or from plain errors that have no `shortMessage`, and name the network, disconnect and account cases in `describeError` before the revert branch.

In `revertData` (`web/app/src/chain/revert.ts`):

```
-   const txt = n.details ?? n.message ?? '';
+   const txt = n.details || (n.shortMessage === undefined ? n.message : '') || '';
```

In `describeError` (`web/app/src/wallet/tx.ts`), after the `PreflightError` check:

```

if (byName('ChainMismatchError') || byName('ConnectorChainMismatchError')) {
  return `Your wallet is on another network, so this step was not sent. Switch back to
  ↳ ${robinhoodChain.name} and try again.`;
}
if (byName('ConnectorNotConnectedError')) return `Your wallet disconnected, so this step was not sent.
↳ Reconnect and try again.`;
if (byName('ConnectorAccountNotFoundError')) return `Your wallet switched to another account, so this
↳ step was not sent. Select the account that started this flow, or start again.`;

```

Note Systems: Fixed in commit [fbf7d0c](#).

Cyfrin: Verified.

7.2.23 Effective strike close is discarded because viem decodes uint40 as a number, so calendar changes never reach the strike date or deposit deadline

Description: ChainProvider.effectiveStrikeCloses keeps NoteCore.effectiveObservationTs(id, 0).effective only when it is a bigint, but the output is a uint40 and viem decodes every integer up to 48 bits as a JS number. The check is always false and the map is always empty.

```

// web/app/src/data/ChainProvider.ts:238-243
ids.forEach((id, i) => {
  const v = r[i] as readonly [bigint, bigint] | { effective: bigint } | undefined;
  if (!v) return;
  const eff = Array.isArray(v) ? v[1] : (v as { effective: bigint }).effective;
  if (typeof eff === 'bigint' && eff > 0n) out.set(id, Number(eff));
});

```

buildSeries then falls back to the scheduled close, both for the strike row and for depositDeadline, which subscriptionOpen uses to enable the COUPON and SHIELD deposit buttons:

```

// web/app/src/data/ChainProvider.ts:299-300
const subscriptionEnd = Number(v.subscriptionEnd);
const strikeClose = effectiveStrikeClose ?? Number(v.observations[0] ?? subscriptionEnd);

```

NoteCore closes the book at the effective close and reverts SubscriptionClosed after it. The app's early close handling (the "Moved forward from ..." hint, the fill window close, the strike row at its effective close) only runs in preview mode, where the mock provider writes the effective close into observations[0] directly.

Impact: Once governance changes the MarketCalendar under an existing series, the app shows the scheduled strike instead of the effective one. A later strike shows the old date and "Awaiting crank" while finalizeStrike reverts TooEarly; an earlier one leaves deposits open in the app after the book has closed, so a user may pay for an approval before the deposit reverts. No funds move, and the earlier case also needs a same-day subscriptionEnd, which only Timelock-created series have.

Proof of Concept:

1. On a testnet fork, create through the Timelock a series with subscriptionEnd 2026-10-01 19:00 UTC and its strike at 20:00 UTC (series 82), then call MarketCalendar.setEarlyClose(20727, 1790874000) to close that day at 17:00 UTC.
2. At 2026-10-01 18:00 UTC (fork and device clock), open #/series/82. The pill says "Subscription open", Strike says "01 Oct, 20:00 UTC", "Deposits close" has no early close hint, and the deposit cards accept an amount.
3. Click "Review purchase" with 20 USDG. The deposit reverts SubscriptionClosed(82).
4. Call effectiveObservationTs(82, 0). It returns effective 1790874000, which the app ignored.

Recommended Mitigation: Normalise the decoded value with the file's num helper so both widths are accepted. On the Series page, print the scheduled time from `eff?.scheduled` in the "moved to" line, because `observations[0].closeTs` then holds the effective close.

```
ids.forEach((id, i) => {
  // uint40 outputs decode to a JS number in viem (every uint up to 48 bits does), so take either
  ↪ width.
  const v = r[i] as readonly [number | bigint, number | bigint] | { effective: number | bigint } |
  ↪ undefined;
  if (!v) return;
  const eff = num(Array.isArray(v) ? v[1] : (v as { effective: number | bigint }).effective);
  if (eff > 0) out.set(id, eff);
});
```

Note Systems: Fixed in commit [d0bf98f](#).

Cyfrin: Verified.

7.2.24 Lock figures add stakedNoteEq to veNOTE LockedBalance.amount, which already includes it, so the sNOTE part of every lock is counted twice

Description: The app treats `veNOTE.locked(owner).amount` as the plain NOTE in a lock and adds `stakedNoteEq` on top. On chain, amount is already the total NOTE-equivalent: `lockFromStaked` adds the sNOTE value to it, voting weight is computed from amount alone, and `withdraw` pays `amount - stakedNoteEq` as NOTE. Every figure built from the stored lock therefore counts the sNOTE part twice.

```
// web/app/src/routes/GovernanceLock.tsx:111-112
<KV k="Voting power now" v={` ${fmtVe(veWeight(lock.amount + lock.stakedNoteEq,
  ↪ lock.end, now))} veNOTE` } />
<KV k="Locked NOTE" v={` ${fmtNote(lock.amount, 2)} NOTE` } />
```

The same sum or label appears in:

- the Lock form's "Voting power before / after" preview (`GovernanceLock.tsx`);
- the Governance page's voting power, share of supply and "Locked NOTE" (`Governance.tsx`);
- the Portfolio "NOTE across modules" total, the "Locked" row and the "veNOTE lock ends" item (`Portfolio.tsx`).

The Preview mock builds `amount` as plain NOTE, so Preview mode hides the error. Figures read directly from chain (gauge vote card, proposal weight, Portfolio "Voting power") are correct and contradict the lock cards for the same wallet.

Impact: Every wallet whose lock holds sNOTE sees its voting power, share of supply and locked NOTE overstated, up to 2x for an sNOTE-only lock, and sizes top ups or extensions from that figure. No funds or calldata are affected: votes use chain weights, so the inflated share only misleads decisions made outside the app.

Proof of Concept:

1. On a testnet fork, open `#/governance/lock`, lock 1,000 NOTE for two years, then add 1,500 NOTE worth of sNOTE on the Increase tab. On chain `locked().amount` is 2,500e18 and `balanceOf` about 1,238e18.
2. Reload the page. "Your lock" shows "Voting power now 1,981 veNOTE" and "Locked NOTE 2,500.00 NOTE" next to "Locked sNOTE 1,500 ~ 1,500 NOTE".
3. Open `#/governance/gauges`: it says "Split your 1,238 veNOTE across gauges" for the same wallet.
4. Open `#/portfolio`: "Locked 4,000.00 NOTE eq." for a wallet whose only NOTE is the 2,500 in the lock.

Recommended Mitigation: Add a helper that splits a lock into its total and its plain NOTE part, and use it wherever a lock amount is summed or labelled. Replace every `lock.amount + lock.stakedNoteEq` with `lockParts(lock).total`, and show `lockParts(lock).plain` where the app labels plain NOTE. Correct the `VeLock.amount` comment and build the Preview lock the way the chain stores it.

```
export function lockParts(l: VeLock): { total: bigint; plain: bigint } {  
  return { total: l.amount, plain: l.amount - l.stakedNoteEq };  
}
```

```
- <KV k="Locked NOTE" v={`${fmtNote(lock.amount, 2)} NOTE`} />  
+ <KV k="Locked NOTE" v={`${fmtNote(lockParts(lock).plain, 2)} NOTE`} />
```

Note Systems: Fixed in commit [0daa3fa](#).

Cyfrin: Verified.

7.2.25 Buttons outside ConfirmSheet skip the preview gate, so a `?preview=1` link sends real transactions built from mock figures

Description: The app refuses writes in preview mode only in `useActionGate`, which `ConfirmSheet` and the Automation buttons read, while nine direct action buttons call `tx.send` with no gate. `?preview=1` in the URL switches every read to the `MockProvider` but leaves the wallet connected to the real chain, and the shared send path checks only the wallet and the chain id:

```
// web/app/src/wallet/tx.ts:280-284  
const send = useCallback<UseTx['send']>(async (title, stepsOrFn, opts) => {  
  if (!w.address) { toast({ kind: 'error', title: 'Connect a wallet first' }); return null; }  
  if (w.chainId !== robinhoodChain.id) {  
    try { await w.switchChain(); } catch { toast({ kind: 'error', title: `Switch to  
    ↳ ${robinhoodChain.name}`, body: `Chain id ${robinhoodChain.id}` }); return null; }  
  }  
}
```

The ungated buttons:

- Stake: "Withdraw ready sNOTE" and "Cancel all requests"
- Governance lock: "Withdraw expired lock"
- Desk: "Process queue", "Cancel request" and "Claim filled withdrawals"
- Gauges: "Claim gauge rewards" and "Claim coupons"
- Testnet faucet

The `MockProvider` gives every connected wallet 500 sNOTE ready to withdraw, so both Stake buttons render in preview, with the mock amount on the Withdraw button:

```
// web/app/src/routes/Stake.tsx:184-185  
{readyToWithdraw && <button className="btn btn--primary btn--block" disabled={tx.busy}  
  ↳ onClick={() => tx.send('Withdraw NOTE', () => unstakeRedeemSteps(owner,  
  ↳ ready))}>{tx.busy ? tx.phase ?? 'Working' : `Withdraw ${fmtSh(ready)} sNOTE`} </button>  
<button className="btn btn--secondary btn--block" disabled={tx.busy} onClick={() =>  
  ↳ tx.send('Cancel cooldown', cancelUnstakeSteps())}>Cancel all requests</button>
```

Impact: A holder who opens a `?preview=1` link with a wallet connected sees sample figures and a disabled Confirm on every sheet, yet these buttons send real transactions: "Cancel all requests" restarts the cooldown on every real sNOTE withdrawal request, and "Withdraw 500.0000 sNOTE" redeems an amount taken from the mock data. The wallet prompt shows the real call and the funds stay with the user.

Proof of Concept:

1. With a wallet that holds sNOTE, open `#/stake` and request two unstakes so that one has matured.
2. Open `#/stake?preview=1`. The Unstake tab shows the mock queue with "Withdraw 500.0000 sNOTE" and "Cancel all requests" enabled; the "Review unstake" sheet has Confirm disabled for preview mode.
3. Click "Withdraw 500.0000 sNOTE". The wallet asks to sign `sNOTE.redeem(50000000000000000000000000000000, you, you)`, which consumes 500 sNOTE of the real matured requests.
4. Click "Cancel all requests". The wallet asks to sign `sNOTE.cancelWithdrawRequest()`, which clears every open request.

Recommended Mitigation: Enforce the preview rule in `runSteps`, which every write passes through, so no button can send while `USE_CHAIN` is false. Passing `gate.disabled` to the direct buttons, as `Automation.tsx` does, also disables them.

```
export async function runSteps(steps: TxStep[], account: `0x${string}`, onStep?: (i: number, step:
  ↳ TxStep, phase: 'simulate' | 'sign' | 'wait' | 'done', hash?: `0x${string}`) => void):
  ↳ Promise<TxResult> {
+ // Preview mode renders MockProvider figures: no write may be built from them, whichever button
  ↳ started it.
+ if (!USE_CHAIN) throw new PreflightError('Preview mode: the figures on this page are sample data and
  ↳ transactions are disabled.');
```

```
const c = await core();
```

Note Systems: Fixed in commit [53510af](#).

Cyfrin: Verified.

7.2.26 `rollAll` and `rollRange` are sent after an empty simulation, and the toast lists the stale preview's series as opened

Description: `runSteps` reads the return value of its pre-sign simulation only for steps that set `minResult`, so when `RollPolicy.rollAll()` or `rollRange` simulates to an empty id list (nothing is due any more), the app still opens the wallet prompt and sends the call. The contract does not revert in that case: it returns the ids created, empty when nothing was due.

The Issuance calendar card then builds its success toast from its last preview read (refetched every 60 s), not from the receipt:

```
// web/app/src/components/Automation.tsx:363-371
const roll = (w: (typeof windows)[number]) => {
  if (!rp || w.due.length === 0) return;
  const step = single
    ? { label: 'RollPolicy.rollAll()', address: rp, abi: rollPolicyAbi, functionName: 'rollAll' as
      ↳ const, args: [] as const }
    : { label: `RollPolicy.rollRange(${w.start}, ${w.count})`, address: rp, abi: rollPolicyAbi,
      ↳ functionName: 'rollRange' as const, args: [BigInt(w.start), BigInt(w.count)] as const };
  void tx.send(single ? 'Roll every due series' : `Roll due series ${w.start + 1} to ${w.start +
    ↳ w.count}`, [step], {
    successBody: `${w.due.length} new series opened (${w.due.map((d) => d.symbol).join(', ')}).`,
    ↳ invalidate: [['series'], ['rollPreview'], ['rollUnderlyings']],
  });
};
```

The single-stock `RollCard` builds its bounty text from the preview the same way.

Impact: When another caller or the Chainlink upkeep rolls inside the card's preview window, a user who clicks "Roll 8 series" signs a transaction that opens nothing and pays no bounty, and is told eight named series opened. After a partial roll the toast still names all eight and omits the bounty actually paid. No funds are at risk and the gas is small on this chain.

Proof of Concept:

1. On a fork of the testnet, open `#/notes` when the Issuance calendar card says "8 of 8 stocks due" and shows "Roll 8 series".
2. Before the card refetches, have another account call `RollPolicy.rollAll()`.
3. Click "Roll 8 series". The app simulates `rollAll()`, gets `[]`, and the wallet still asks to sign `rollAll()`.
4. The transaction succeeds with no Rolled event and no bounty. The toast reads "Roll every due series confirmed. 8 new series opened (AAPL, AMZN, COIN, HOOD, META, MSFT, NVDA, TSLA)."

Recommended Mitigation: Let a step check its simulated return value before the wallet prompt, and refuse the batch roll when the id list is empty. Build both roll toasts from the Rolled events in the receipt, which needs the Rolled event added to `rollPolicyAbi` in `web/app/src/chain/protocolAbi.ts`.

1. In `web/app/src/wallet/tx.ts`, add `checkResult?: (result: unknown) => string | null` to `TxStep`, simulate when it is set, and apply it in `runSteps`:

```
if (!s.noSimulate || s.minResult !== undefined || s.checkResult) {  
  // ...  
  const refused = s.checkResult?.((sim as { result?: unknown }).result);  
  if (refused) throw new PreflightError(`${s.label}: ${refused}`);  
}
```

2. In `RollAllCard` in `web/app/src/components/Automation.tsx`, pass `checkResult` on both the `rollAll` and the `rollRange` step, and take the toast from the receipt. `RollCard` switches its `successBody` to `rollSuccessBody` the same way.

```
const checkResult = (r: unknown) => (Array.isArray(r) && r.length > 0 ? null : 'nothing is due any  
  ↳ more; another caller opened these series after the last read. Nothing was sent.');
```

```
// ... both steps get `checkResult`  
successBody: (res) => rollSuccessBody(res, rp, list), invalidate: [['series'], ['rollPreview'],  
  ↳ ['rollUnderlyings']],
```

3. In the same file, add the receipt-based toast:

```
export async function rollSuccessBody(res: TxResult, policy: `0x${string}`, names: readonly { address:  
  ↳ string; symbol: string }[]): Promise<string> {  
  const { parseEventLogs } = await import('viem');  
  const logs = res.receipts.flatMap((r) => r.logs).filter((l) => l.address.toLowerCase() ===  
    ↳ policy.toLowerCase());  
  const rolled = parseEventLogs({ abi: rollPolicyAbi, eventName: 'Rolled', logs: logs as never, strict:  
    ↳ false }).map((l) => l.args as { underlying: string; seriesId: bigint; bountyPaid: bigint });  
  if (rolled.length === 0) return 'The transaction confirmed but opened no series: another caller rolled  
    ↳ first. No bounty was paid.';  
  const sym = (u: string) => names.find((n) => n.address.toLowerCase() === u.toLowerCase())?.symbol ??  
    ↳ `${u.slice(0, 6)}${u.slice(-4)}`;  
  const paid = rolled.reduce((a, r) => a + r.bountyPaid, 0n);  
  const bounty = paid > 0n ? `Bounty ${fmtUsd(paid)} USDG paid to your wallet.` : 'No bounty was paid:  
    ↳ RollPolicy held no USDG for it.';  
  return `${rolled.length} new series opened (${rolled.map((r) => `${sym(r.underlying)} Series  
    ↳ ${r.seriesId}`).join(', ')}). ${bounty}`;  
}
```

Note Systems: Fixed in commit [dd2bf63](#).

Cyfrin: Verified.

7.2.27 Stake APY keeps annualising the last sNOTE reward stream after it has ended and labels it as trailing 30 d rewards

Description: ChainProvider.getStake computes the Stake page APY from streamAmount, streamStart and streamEnd without checking streamEnd against the clock, so a fully vested stream keeps showing, annualised, as the current yield.

```
// web/app/src/data/ChainProvider.ts:561-568
const now = Math.floor(Date.now() / 1000);
const ta = big(totalAssets);
const sa = big(streamAmount); const ss = num(streamStart); const se = num(streamEnd);
let apyBpsEst = 0;
if (ta > 0n && se > ss) {
  const perYear = (sa * BigInt(365 * 86_400)) / BigInt(se - ss);
  apyBpsEst = Number((perYear * 10_000n) / ta);
}
```

sNOTE keeps these fields after a stream ends and overwrites them only on the next notifyReward, which on this deployment comes with the next buyback sale.

The page labels the figure a trailing 30 day yield, although it is one 7 day stream multiplied by 365/7, and shows it next to a Streaming stat that already says the stream is over:

```
// web/app/src/routes/Stake.tsx:59
<Stat label="APY" value={fmtPct(st.apyBpsEst)} est sub="From trailing 30 d rewards" />
```

Impact: After a buyback sale, the Stake page shows its 7 day stream as the APY until the next sale, though stakers earn nothing once it ends. A holder who stakes on that figure locks NOTE for the cooldown and earns 0 until another sale. No principal is lost, and it needs sales more than 7 days apart.

Proof of Concept:

1. Start an Anvil fork of the testnet with --auto-impersonate and point a testnet build of the app at it.
2. Fund the buyback reserve and sell 5,000 NOTE through RevenueRouter.sellNoteForQuote, which starts a 7 day sNOTE stream.
3. Advance the fork by 8 days with no further sale.
4. Open #/stake with the device clock at chain time. The APY reads about 46% "From trailing 30 d rewards" while Streaming reads "0 NOTE, No active stream".

Recommended Mitigation: Annualise the stream only while it is running, and label the figure as what it is.

1. In ChainProvider.getStake (web/app/src/data/ChainProvider.ts), skip the APY once the stream has ended:

```
-   if (ta > 0n && se > ss) {
+   if (ta > 0n && se > ss && se > now) {
```

2. In web/app/src/routes/Stake.tsx, replace the APY stat's "From trailing 30 d rewards" label, and reword the page note to match:

```
<Stat label="APY" value={fmtPct(st.apyBpsEst)} est sub={streamActive ? `Current stream
↳ annualised ends in ${fmtDuration(streamLeft, 2)}` : 'No active stream, nothing is
↳ paid'} />
// ...
<PreviewNote>APY annualises the current reward stream over total staked and is 0 while no
↳ stream is running; it is not a guarantee.{USE_CHAIN ? '' : 'Figures illustrative in
↳ Preview mode.'}</PreviewNote>
```

Note Systems: Fixed in commit [62e5b30](#).

Cyfrin: Verified.

7.2.28 `note.systems` lacks DNSSEC, a CAA record, a registrar update lock and HSTS preload, so a DNS or first-visit attacker can serve a different app

Description: Whoever controls what `note.systems` resolves to, or answers the first plain HTTP request, controls the JavaScript that builds every transaction the user signs, and the domain has none of the controls that make that harder or visible:

- The zone is unsigned (no DS record, no DNSKEY), so a resolver cannot detect a spoofed answer.
- There is no CAA record, so any public CA may issue a certificate for the domain.
- The only registry status is `clientTransferProhibited`, so changing the nameservers needs only access to the registrar account.
- HSTS has no `includeSubDomains` or `preload` and the domain is not preloaded, so a first visit goes out in cleartext and follows the 301 it gets back.

Registrar, DNS, CDN and mail all sit in one Hostinger account. These are host settings outside the repository.

Impact: An attacker who takes over the registrar or DNS account, spoofs a resolver answer or intercepts a first visit can serve a copy of the app, with a valid certificate, whose step builders send approvals and deposits to the attacker. Users who approve it lose the approved tokens. Each missing control removes one layer that would prevent or detect this.

Proof of Concept: Query the DNS records, the registry status and the response headers. The DS, DNSKEY and CAA queries return nothing.

```
$ dig +short DS note.systems @1.1.1.1
$ dig +short DNSKEY note.systems @1.1.1.1
$ dig +short CAA note.systems @1.1.1.1
$ dig +short NS note.systems @1.1.1.1
hermes.dns-parking.com.
artemis.dns-parking.com.
$ dig +short MX note.systems @1.1.1.1
5 mx1.hostinger.com.
10 mx2.hostinger.com.
$ curl -sL https://rdap.org/domain/note.systems | jq -c '{status, dnssec: .secureDNS.delegationSigned}'
{"status":["client transfer prohibited"],"dnssec":false}
$ curl -sI https://note.systems/app/ | grep -i -E '^(server|platform|panel|strict-transport-security):'
platform: hostinger
panel: hpanel
strict-transport-security: max-age=15552000
server: hcdn
$ curl -s 'https://hstspreload.org/api/v2/status?domain=note.systems' | jq -c .
{"name":"note.systems","status":"unknown","bulk":false,"preloadedDomain":""}
$ curl -sI http://note.systems/app/ | grep -i -E '^(HTTP|location)'
HTTP/1.1 301 Moved Permanently
location: https://note.systems/app/
```

Recommended Mitigation: Follow the [SEAL Domain & DNS Security](#) baseline. Enable DNSSEC and publish the DS record, set `clientUpdateProhibited` and `clientDeleteProhibited` (or a registry lock) on a registrar account protected by a hardware key, and alert on DNS record changes and new Certificate Transparency entries. Restrict certificate issuance:

```
note.systems. CAA 0 issue "letsencrypt.org"
note.systems. CAA 0 issuewild ";"
note.systems. CAA 0 iodef "mailto:security@note.systems"
```

Send HSTS for the whole domain and submit it to the preload list once every subdomain serves HTTPS:

```
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

Note Systems: Acknowledged. Registrar, DNS and hosting settings rather than code. Panel changes in progress.

7.2.29 note.systems mail policy does not reject spoofed mail, and its MX publishes no MTA-STS policy

Description: The domain publishes SPF with a softfail and DMARC with no enforcement:

```
v=spf1 include:_spf.mail.hostinger.com ~all
v=DMARC1; p=none
```

~all asks receivers to accept mail from unlisted servers and only mark it. p=none asks them to take no action when neither SPF nor DKIM aligns. The record has no rua, so nobody at Note Systems receives the aggregate reports that would show a spoofing campaign. DKIM is set up (selector hostingermail-a), but DMARC does not require it.

The domain has no MTA-STS policy (_mta-sts) and no TLS-RPT record (_smtp._tls), so a sending server has no signal that TLS is required at the Hostinger MX and delivers in cleartext when an on-path attacker strips STARTTLS.

Impact: Anyone can send mail as security@note.systems, support@note.systems or any other domain address, and delivery depends only on each provider's spam filter. The mail can push a fake migration or "urgent re-approval" linking to a lookalike app, which the user still has to follow off the real site. Mail to the domain, including reports to security@note.systems, can be read or suppressed in transit until MTA-STS is enforced.

Proof of Concept:

1. Query the mail records:

```
$ dig +short TXT note.systems @1.1.1.1
"v=spf1 include:_spf.mail.hostinger.com ~all"
$ dig +short TXT _dmarc.note.systems @1.1.1.1
"v=DMARC1; p=none"
$ dig +short TXT hostingermail-a._domainkey.note.systems @1.1.1.1 | cut -c1-60
hostingermail-a.dkim.mail.hostinger.com.
"v=DKIM1;k=rsa;p=MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQE
$ dig +short TXT _mta-sts.note.systems @1.1.1.1
$ dig +short TXT _smtp._tls.note.systems @1.1.1.1
```

2. The _mta-sts and _smtp._tls queries return nothing.

Recommended Mitigation: Move mail authentication to enforcement ([SEAL DNSSEC and email security](#)).

1. Publish DMARC with reports first, then move to quarantine and reject once the reports show only Hostinger and DKIM-signed mail:

```
_dmarc.note.systems. TXT "v=DMARC1; p=reject; sp=reject; adkim=s; aspf=s; rua=mailto:dmarc@note.systems"
```

2. Change SPF to a hard fail:

```
note.systems. TXT "v=spf1 include:_spf.mail.hostinger.com -all"
```

3. Publish an MTA-STS policy in enforce mode at <https://mta-sts.note.systems/.well-known/mta-sts.txt>, with the matching records:

```
_mta-sts.note.systems. TXT "v=STSV1; id=2026092901"
_smtp._tls.note.systems. TXT "v=TLSRPTv1; rua=mailto:tls-reports@note.systems"
```

Note Systems: Acknowledged. DNS records and a hosted policy file rather than code. The DNS changes are being applied through the hosting panel.

7.3 Informational

7.3.1 Review sheets hard-code the Network row as Robinhood Chain on the testnet build

Description: Every ConfirmSheet caller passes the literal string 'Robinhood Chain' for its "Network" row instead of robinhoodChain.name, so the testnet build labels every transaction review with the mainnet chain name. The literal appears eleven times: Desk.tsx:409, Series.tsx:229 and :326, Bonds.tsx:220, Stake.tsx:201, Buy-back.tsx:232, GovernanceLock.tsx:190, GovernanceGauges.tsx:193, GovernanceProposal.tsx:235, Portfolio.tsx:523 and components/Automation.tsx:501.

```
// web/app/src/routes/Desk.tsx:405-410
rows=[
  { k: isDeposit ? 'Deposit' : 'Shares', v: isDeposit ? `${fmtNum(n, 2)} USDG` : fmtNum(n, 4) },
  { k: 'NAV per share', v: `${fmtNum(nav, 4)} USDG` },
  { k: isDeposit ? 'Shares out' : 'USDG out', v: isDeposit ? fmtNum(shares, 4) : `${fmtNum(n * nav, 2)}
    ↳ USDG` },
  { k: 'Network', v: 'Robinhood Chain', mono: false },
]
```

The header pill, the connect modal and the wrong-chain gate read the build's chain definition, robinhoodChain (web/app/src/chain/robinhood.ts#L48-L53), which is "Robinhood Chain Testnet" (46630) on the testnet build.

Impact: On the testnet build, the last screen before the wallet prompt names the mainnet chain while the header says testnet. No transaction goes to the wrong chain: useActionGate disables Confirm unless the wallet is on robinhoodChain.id, and the wallet shows the real network. On the mainnet build the literal is correct.

Proof of Concept:

1. Serve a testnet build and open #/desk. The header pill reads "Robinhood Chain Testnet".
2. Enter 10 in the deposit amount and click "Review deposit". The sheet's last row reads "Network · Robinhood Chain".

Recommended Mitigation: Define the row once from robinhoodChain.name and use it in every sheet. In web/app/src/components/Sheet.tsx, add it next to SummaryRow:

```
/** Last review-sheet row: the chain this build targets ("Robinhood Chain" or "Robinhood Chain
    ↳ Testnet"). */
export const NETWORK_ROW: SummaryRow = { k: 'Network', v: robinhoodChain.name, mono: false };
```

In each of the eleven callers, import NETWORK_ROW and replace the literal row. Portfolio.tsx pushes it as rows.push(NETWORK_ROW).

```
-      { k: 'Network', v: 'Robinhood Chain', mono: false },
+      NETWORK_ROW,
```

Note Systems: Fixed in commit [bcee01e](#).

Cyfrin: Verified.

7.3.2 Live routes render mock-provider copy and the ?preview flag gives a false reason for disabled transactions

Description: Three PreviewNote sentences written for the mock provider are rendered without a USE_CHAIN check, so the live testnet build shows them under real chain data. The governance overview tells the reader that live addresses are placeholders:

```
// web/app/src/routes/Governance.tsx:197
<PreviewNote>Preview data mirrors veNOTE, NoteGovernor, UpgradeGovernor, TimelockController and
↳ SeriesRegistry reads. Addresses shown are placeholders until deployment.</PreviewNote>
```

The proposals list calls the on-chain proposals samples:

```
// web/app/src/routes/GovernanceProposals.tsx:92
<PreviewNote>Proposal ids are uint256 hashes of targets, values, calldatas and the description hash
↳ (hashProposal). Sample proposals are illustrative.</PreviewNote>
```

The buyback page ends its note with "Figures illustrative in Preview mode." unconditionally (web/app/src/routes/Buyback.tsx#L97). Other routes already gate the same sentence:

```
// web/app/src/routes/Stake.tsx:120
<PreviewNote>APY is an estimate from trailing rewards and current total staked; it is not a
↳ guarantee.{USE_CHAIN ? '' : ' Figures illustrative in Preview mode.'}</PreviewNote>
```

Separately, useActionGate gives one reason for both causes of !USE_CHAIN (USE_CHAIN is !IS_PREVIEW && !PREVIEW_FLAG). On the testnet build with ?preview=1 only the flag disables the action, yet the Confirm tooltip says the contracts are not deployed:

```
// web/app/src/components/Sheet.tsx:25-27
export function useActionGate(): { disabled: boolean; reason: string | null } {
  const w = useWallet();
  if (!USE_CHAIN) return { disabled: true, reason: 'Preview mode. Contracts are not deployed yet;
↳ transactions are disabled.' };
}
```

The header already tells the two cases apart (Shell.tsx shows the "Preview flag set (?preview=1)" pill only when !USE_CHAIN && !IS_PREVIEW).

Impact: A voter may discount a live proposal or distrust a correct contract address. Under ?preview=1 the stated reason for the disabled Confirm button is false. No transaction or value is affected.

Proof of Concept:

1. Serve a testnet build. Open #/governance: the page foot reads "Addresses shown are placeholders until deployment." under live data and explorer links.
2. Open #/governance/proposals: the on-chain Note Governor proposal is listed above "Sample proposals are illustrative."
3. Open ?preview=1#/desk, enter 10 and click "Review deposit". The header shows "Preview data" and "Testnet"; the disabled Confirm tooltip reads "Preview mode. Contracts are not deployed yet; transactions are disabled."

Recommended Mitigation: Gate the mock-only sentences on USE_CHAIN as Stake.tsx, Desk.tsx, Bonds.tsx and GovernanceLock.tsx do, and split the gate reason on IS_PREVIEW so the preview flag case names the flag. Each route imports USE_CHAIN from ../chain/robinhood and Sheet.tsx also imports IS_PREVIEW.

1. In web/app/src/components/Sheet.tsx, useActionGate gives the preview flag its own reason:

```
if (IS_PREVIEW) return { disabled: true, reason: 'Preview mode. Contracts are not deployed yet;
↳ transactions are disabled.' };
if (!USE_CHAIN) return { disabled: true, reason: 'Preview flag set (?preview=1): figures come from the
↳ mock provider and transactions are disabled. Remove the flag to use the live contracts.' };
```

2. In `web/app/src/routes/Governance.tsx`, render the overview note only off-chain:

```
{!USE_CHAIN && <PreviewNote>Preview data mirrors veNOTE, NoteGovernor, UpgradeGovernor,  
  ↳ TimelockController and SeriesRegistry reads. Addresses shown are placeholders until  
  ↳ deployment.</PreviewNote>}
```

3. In `web/app/src/routes/GovernanceProposals.tsx`, keep the note and append the mock sentence only off-chain:

```
<PreviewNote>Proposal ids are uint256 hashes of targets, values, calldatas and the description  
  ↳ hash (hashProposal).{USE_CHAIN ? '' : ' Sample proposals are illustrative.'}</PreviewNote>
```

4. In `web/app/src/routes/Buyback.tsx`, end the note the same way, with `{USE_CHAIN ? '' : ' Figures illustrative in Preview mode.'}` in place of the fixed sentence.

Note Systems: Fixed in commit [31a7acc](#).

Cyfrin: Verified.

7.3.3 App implements eight presentation rules differently from its specs

Description: The Notes, analytics and Portfolio specs state eight presentation rules that the routes implement differently. Two of them change what a figure means:

- The Settled table sorts by scheduled maturity ('ends') instead of the settled date it displays, so an early autocall sorts by a maturity it never reached.
- "Received to date" is summed per series but shown on each leg row, so a wallet holding both legs sees its COUPON claims repeated on the SHIELD row and in the CSV.

```
// web/app/src/components/NotesBook.tsx:252  
{ key: 'matched', label: 'Matched', sort: 'matched', right: true }, { key: 'settled', label: 'Settled',  
  ↳ sort: 'ends', right: true },
```

```
// web/app/src/lib/portfolio.ts:157-161  
export function receivedToDate(history: ReadonlyArray<HistoryItem>, seriesId: number): bigint {  
  let sum = 0n;  
  for (const h of history) if (h.seriesId === seriesId && (h.kind === 'claim' || h.kind === 'redeem' ||  
    ↳ h.kind === 'couponPaid')) sum += h.quoteAmount;  
  return sum;  
}
```

The other six are presentation drift:

- The stock view Rows/Table toggle is local state, so a shared `view=table` link opens in Rows.
- Above 16 underlyings the stock board stays and a symbols-only selector is added, instead of the board becoming a selector.
- The ladder and subscription rows show fewer fields than the spec lists.
- The analytics series table has "To barrier" but no "To autocall" column.
- The Portfolio "Next event" tone is red for at risk or breached and uncoloured otherwise, instead of following the worst health class.
- The "Autocall" outcome is shown on COUPON rows whose only remaining observation is maturity.

Impact: A holder of both legs of a series sees the same received amount on both rows and in the exported CSV, and the settled list is ordered differently from the dates in its own column. No transaction is affected.

Proof of Concept:

1. Open `#/notes?stock=<symbol>&view=table`. The stock view opens in Rows.
2. On `#/notes`, expand Settled with a series that autocalled before a series that matured. The Settled dates are out of order.
3. With a wallet holding both legs of one series after a coupon claim, open `#/portfolio`. Both rows show the claimed amount, and "Export CSV" repeats it in `received_usdg_6dp` for both.

Recommended Mitigation: Change the code for the settled sort, the received figure, the view toggle, the autocall column, the Next event tone and the Autocall outcome, and update the spec to the shipped board and row contents. Sort the Settled column on `settledTs`, and tag claims and redemptions with their leg so `receivedToDate` can sum per leg:

```
-   { key: 'matched', label: 'Matched', sort: 'matched', right: true }, { key: 'settled', label:  
→ 'Settled', sort: 'ends', right: true },  
+   { key: 'matched', label: 'Matched', sort: 'matched', right: true }, { key: 'settled', label:  
→ 'Settled', sort: 'settled', right: true },
```

```
for (const l of cl as L[]) push(l, 'claim', big(l.args.quoteOut), 0n, LEG_COUPON);  
for (const l of rd as L[]) push(l, 'redeem', big(l.args.quoteOut), big(l.args.stockOut),  
→ Number(l.args.leg) as Leg);  
// ...  
for (const h of history) if (h.seriesId === seriesId && (leg === undefined || h.leg === undefined ||  
→ h.leg === leg) && (h.kind === 'claim' || h.kind === 'redeem' || h.kind === 'couponPaid')) sum +=  
→ h.quoteAmount;
```

Note Systems: Fixed in commit [4235058](#).

Cyfrin: Verified.

7.3.4 Failure toast does not say that approvals confirmed earlier in the flow remain on chain

Description: When a later step of a multi-step flow is rejected or fails, `runSteps` withdraws only grants that carry an undo (the ERC-1155 operator approval). ERC-20 approvals from `ensureAllowance` stay on chain, and the failure toast describes only the failing step.

```
// web/app/src/wallet/tx.ts:239-244  
} catch (e) {  
  // A step failed or was rejected after earlier ones confirmed: withdraw any grant those steps left  
  → standing, best  
  // effort, most recent first. The original failure is what the caller sees.  
  for (const s of [...confirmed].reverse()) if (s.undo) await exec(steps.indexOf(s), s.undo).catch(()  
  → => undefined);  
  throw e;  
}
```

```
// web/app/src/wallet/tx.ts:300-304  
} catch (e) {  
  const se = e instanceof StepError ? e : null;  
  const ctx = se ? { contract: contractName(se.step.address), fn: se.step.functionName } :  
  → undefined;  
  toast({ kind: 'error', title: `${title} failed`, body: await describeError(se ? se.cause : e,  
  → ctx) });  
  return null;  
}
```

Impact: After a rejected SHIELD deposit the wallet keeps an exact allowance of the stock and of the USDG prefund to NoteCore, and after a rejected Desk deposit an exact USDG allowance to the Desk (a UUPS proxy). Each allowance is capped at the amount the user reviewed and a retry reuses it, so nothing is lost. The user is told "failed" with no mention that two of the three transactions went through.

Proof of Concept:

1. Open a series in Subscription (for example `#/series/76`), enter 0.5 in the SHIELD card and confirm the deposit.
2. Confirm "Approve AAPL" and "Approve USDG", then reject "Deposit SHIELD".
3. The toast reads "SHIELD deposit failed · Request rejected in wallet." Both allowances to NoteCore remain (stock: 500000000000000000n, usdg: 18441236n).

Recommended Mitigation: Carry the confirmed steps on the `StepError` and name the confirmed approvals in the failure toast. All changes are in `web/app/src/wallet/tx.ts`.

1. Give `StepError` a confirmed: `TxStep[] = []` field and add a helper that builds the toast suffix:

```
/** Failure-toast suffix naming ERC-20 approvals that confirmed before the flow stopped; they stay on chain. */
export function leftAllowanceNote(se: StepError | null): string {
  const left = (se?.confirmed ?? []).filter((s) => s.functionName === 'approve');
  return left.length ? `The approvals already confirmed (${left.map((s) => s.label).join(', ')} stay on chain and are reused if you retry.` : '';
}
```

2. In the catch of `runSteps`, record the confirmed steps before rethrowing:

```
for (const s of [...confirmed].reverse()) if (s.undo) await exec(steps.indexOf(s), s.undo).catch(() => undefined);
+ if (e instanceof StepError) e.confirmed = [...confirmed];
throw e;
```

3. In `useTx`, append the note to the toast body:

```
toast({ kind: 'error', title: `${title} failed`, body: `${await describeError(se ? se.cause : e, ctx)}${leftAllowanceNote(se)}` });
```

Note Systems: Fixed in commit [12aeebb](#).

Cyfrin: Verified.

7.3.5 Portfolio Remaining column prints a minus zero and the full prefund as coming back for SHIELD positions before strike

Description: The "Remaining · est." cell for a SHIELD position prepends a literal minus sign to `fmtUsd(flows.remainingGross)` whenever observations remain. Before strike, `cashFlows` computes `remainingGross` from a coupon of 0 because `couponBps` is still null.

```
// web/app/src/lib/portfolio.ts:84-88
const k = seriesEnded(s.status) ? 0n : BigInt(remainingObservations(s.observations).length);
const c = BigInt(s.couponBps ?? 0);
const gross = (p.units * c) / BPS;
// ...
const remainingGross = gross * k;
```

```
// web/app/src/routes/Portfolio.tsx:280-283  |
```

```

: (flows.remainingCount > 0 ? <>-{fmtUsd(flows.remainingGross)}<span
↳   className="cell-sub">{fmtUsd(flows.prefundExpectedBack)} prefund
↳   back</span></> : ...)}

```

The Coupon column in the same row already renders "Set at strike" when couponBps is null.

Impact: Every SHIELD position in a series still in Subscription shows "-0.00" remaining coupons and its entire prefund under "prefund back". Every COUPON position there shows "0.00" with "3 coupons left". On testnet one wallet with 31 SHIELD positions has 16 rendered as -0.00; the other 15 are in Live series and show real figures. The figures are labelled estimates and no action depends on them.

Proof of Concept:

1. Connect a wallet with a SHIELD deposit in a series in Subscription (for example 0x49BC44E021b29D9e1B8e8Cf5ae124cb6b2767425 on testnet, series 24 to 39) and open `#/portfolio`.
2. The SHIELD rows show "Remaining · est." as "-0.00" with "112.66 prefund back" (series 24), while the Coupon column says "Set at strike".

Recommended Mitigation: Render "Set at strike" in the Remaining cell while couponBps is null, as the Coupon column does, and print the minus only for a non-zero amount.

In `web/app/src/routes/Portfolio.tsx`, change the "Remaining · est." cell (the COUPON branch is unchanged):

```

- {couponLeg
+ {s.couponBps === null && flows.remainingCount > 0
+   ? <span className="faint">Set at strike</span>
+   : couponLeg
+   ? (/* ... */)
-   : (flows.remainingCount > 0 ? <>-{fmtUsd(flows.remainingGross)}<span
↳   className="cell-sub">{fmtUsd(flows.prefundExpectedBack)} prefund back</span></> : <span
↳   className="faint"></span>)}
+   : (flows.remainingCount > 0 ? <>{flows.remainingGross > 0n ? '-' :
↳   ''}{fmtUsd(flows.remainingGross)}<span className="cell-sub">{fmtUsd(flows.prefundExpectedBack)}
↳   prefund back</span></> : <span className="faint"></span>)}

```

Note Systems: Fixed in commit [0ceb1b3](#).

Cyfrin: Verified.

7.3.6 npm test suite does not check what the step builders ask the wallet to sign

Description: No harness in `npm test` encodes the transactions built by `src/chain/actions.ts` or asserts their arguments, approval sizes, chain pinning or the `minResult` output floor in `runSteps`. The only runner harness stubs simulation with an undefined result, so the floor branch in `tx.ts` never runs under test:

```

// web/app/qa/tx-flow-check.mjs:27
export async function simulateContract(_c: any, a: any) { calls.push({ fn: 'simulate', args: a });
↳   return { result: undefined }; }

```

Each of these one-line mutations changes what a user signs and leaves `npm test` green:

- `deskDepositSteps` sends `deposit(owner, assets)` instead of `deposit(assets, owner)`.
- `runSteps` drops `chainId: robinhoodChain.id` from its calls.
- `got < s.minResult` is inverted to `got > s.minResult`.
- `ensureAllowance` approves `2n ** 256n - 1n` instead of `amount`.

Impact: The step builders are correct today, so no user is affected. A later regression in argument order, approval size, chain pinning or the output floor would pass typecheck && npm test && build and ship.

Proof of Concept:

1. In web/app/src/wallet/tx.ts change if (got < s.minResult) to if (got > s.minResult).
2. Run npm test in web/app: every harness passes and the script exits 0.
3. With the harness below added, node qa/tx-encoding-check.mjs reports FAIL runner refuses to sign when the simulated output is below minResult and exits 1.

Recommended Mitigation: Add a harness that runs the step builders and runSteps against a stubbed wagmi core, encodes and decodes every step with its own ABI, and asserts function, argument order, amounts, approval size, chainId on every call and both minResult branches. Add it to the test script in web/app/package.json.

```
// Encode with the step's own ABI, decode it again: fails on a wrong selector, arity or type.
const signed = (step) => decodeFunctionData({ abi: step.abi, data: encodeFunctionData({ abi: step.abi,
↳ functionName: step.functionName, args: step.args }) });

test('runner refuses to sign when the simulated output is below minResult', async () => {
  reset();
  core.script.simResult = 3_979_999n;
  await assert.rejects(m.runSteps([floorStep()], owner), /quote moved/);
  assert.equal(core.calls.filter((c) => c.fn === 'write').length, 0);
});
```

Note Systems: Fixed in commit [b182b41](#).

Cyfrin: Verified.

7.3.7 Amount forms enable Review from Number(amt), so input finer than the token decimals encodes a zero amount

Description: Every amount form enables its Review button when Number(amt) > 0, but the calldata amount is toUnits(amt, decimals), which drops digits past decimals. An input such as 0.0000000000001 on a 12 decimal field passes the gate and encodes 0. AmountInput does not limit the number of fraction digits.

```
// web/app/src/routes/Desk.tsx:343
const n = Number(amt) > 0 ? Number(amt) : 0;
```

```
// web/app/src/chain/actions.ts:25-30
export function toUnits(v: string | number, decimals: number): bigint {
  const s = String(v).trim();
  if (!s || !/^\d*\.\d*$/.test(s)) return 0n;
  const [i, f = ''] = s.split('.');
  const frac = (f + '0'.repeat(decimals)).slice(0, decimals);
  return BigInt(i || '0') * 10n ** BigInt(decimals) + BigInt(frac || '0');
```

The same Number(amt) > 0 gate is at Series.tsx#L171 (COUPON and SHIELD deposits), Bonds.tsx#L156, Stake.tsx#L157, GovernanceLock.tsx#L38 and Buyback.tsx#L181. For the Desk withdrawal the zero reaches a step that the contract accepts:

```
// web/app/src/chain/actions.ts:147
if (!forceQueue && assets <= idle && shares <= maxRedeem) return [{ label: 'Redeem Desk shares',
↳ address: a.desk, abi: deskAbi, functionName: 'redeem', args: [shares, owner, owner], minResult:
↳ floorOf(assets), resultLabel: 'USDG base units' }];
```

Impact: For the Desk withdrawal, `redeem(0, owner, owner)` simulates successfully (it returns 0 and `minResult` is `floorOf(0) = 0`), so the wallet asks the user to sign a transaction that moves nothing and costs 2,646,599 gas on a testnet fork. `depositShield`, `depositWithLimits`, `requestWithdraw`, `createLock` and `lockFromStaked` revert `ZeroAmount()` in the pre-sign simulation, so the user sees a decoded error and signs nothing. The exception is a COUPON leg bond, where a missing operator approval is signed first and then revoked. The case needs an input with 7 to 25 fraction digits, and no funds move.

Proof of Concept:

1. Connect a wallet holding Desk shares, open `#!/desk`, choose Withdraw and type `0.00000000000001`.
2. "Review withdrawal" is enabled and the sheet shows "0.0000" shares and "0.00 USDG".
3. Confirm. The wallet prompts for `redeem(0, owner, owner)`, which succeeds on chain and leaves the share balance unchanged.

Recommended Mitigation: Gate each form on the encoded amount: replace `Number(amt) > 0` in each form's `n` with `toUnits(amt, <field decimals>) > 0n`.

1. In `DepositCard` (`web/app/src/routes/Desk.tsx`), use 6 decimals for deposits and `shareDecimals` for withdrawals:

```
- const n = Number(amt) > 0 ? Number(amt) : 0;
+ const n = toUnits(amt, tab === 'deposit' ? 6 : shareDecimals) > 0n ? Number(amt) : 0;
```

2. Make the same change in `BondCard` (`Bonds.tsx`, `qDec`), `SellCard` (`Buyback.tsx`, 18), `StakeCard` (`Stake.tsx`, 18 to stake, `sd` to unstake) and `GovernanceLock` (`GovernanceLock.tsx`, 18, or `g?.stakedShareDecimals ?? 18` for `sNOTE`).
3. In `web/app/src/routes/Series.tsx`, give `parseAmt` a decimals argument, and pass 6 from `CouponCard` and 18 from `ShieldCard`:

```
function parseAmt(v: string, decimals: number) { const n = Number(v); return Number.isFinite(n) && n >
  ↳ 0 && toUnits(v, decimals) > 0n ? n : 0; }
```

Note Systems: Fixed in commit [2e4e4c5](#).

Cyfrin: Verified.

7.3.8 Both viem clients use one rate limited public RPC endpoint with no fallback transport

Description: The wagmi config and the `ChainProvider` read client each use one `http()` transport, which resolves to the chain's only RPC URL, the public endpoint. The CSP `connect-src` allows only that host, so adding a second provider needs a code change.

```
// web/app/src/wallet/wagmiCore.ts:45
  transports: { [robinhoodChain.id]: http() },
```

```
// web/app/src/data/ChainProvider.ts:83-86
  private readonly client: Client = createClient({
    chain: robinhoodChain,
    transport: http(undefined, { batch: { batchSize: 50, wait: 8 } }),
  });
```

Impact: Every read, pre-signing simulation, gas estimate and receipt wait goes through that one endpoint, for all users at once. The public testnet endpoint rate limits: 60 parallel batches of 50 `eth_blockNumber` calls get 54 HTTP 429 responses and 6 successes. When it throttles, routes stay on skeletons or show error cards and the transaction runner cannot simulate or confirm. Only availability is affected; no funds are at risk.

Proof of Concept:

1. Serve a testnet build, open `#/desk` in a browser and open the devtools Network panel.
2. Block the host `rpc.testnet.chain.robinhood.com` in the Network panel and reload the page.
3. Every JSON-RPC request goes to `rpc.testnet.chain.robinhood.com` and is blocked, none is retried on another endpoint, and the page stays on skeletons or shows error cards.

Recommended Mitigation: Accept extra endpoints at build time, use viem's fallback transport in both clients, and add the extra origins to the CSP.

1. In `web/app/src/chain/robinhood.ts`, export the endpoint list:

```
export const RPC_URLS: readonly string[] = [
  robinhoodChain.rpcUrls.default.http[0],
  ...((import.meta.env?.VITE_RPC_FALLBACK as string | undefined) ?? '').split(',').map((s) =>
    ↪ s.trim()).filter(Boolean),
];
```

2. In `web/app/src/data/ChainProvider.ts`, build the read client's transport from it (importing fallback from viem):

```
- transport: http(undefined, { batch: { batchSize: 50, wait: 8 } }),
+ transport: fallback(RPC_URLS.map((u) => http(u, { batch: { batchSize: 50, wait: 8 } }))),
```

3. In `web/app/src/wallet/wagmiCore.ts`, do the same for the wagmi config (importing fallback from wagmi):

```
- transports: { [robinhoodChain.id]: http() },
+ transports: { [robinhoodChain.id]: fallback(RPC_URLS.map((u) => http(u)), { rank: false } ) },
```

4. In `web/app/vite.config.ts`, allow the extra origins in `connect-src`:

```
const RPC_FALLBACK_ORIGINS = (process.env.VITE_RPC_FALLBACK ?? '').split(',').map((s) =>
  ↪ s.trim()).filter(Boolean).map((u) => new URL(u).origin);
// ...
['connect-src', ['self', RPC_HOST[network], ...RPC_FALLBACK_ORIGINS, ...wcConnect, ...cbConnect]],
```

Note Systems: Fixed in commit [ef13ead](#).

Cyfrin: Verified.

7.3.9 Reference price, market status and the Governance core label read bundled addresses instead of the modules NoteCore and SeriesRegistry resolve

Description: `readReferencePrice` and `readMarketState` in `chain/market.ts` call the `OracleAdapter` and `MarketCalendar` addresses compiled into the bundle, while `NoteCore` resolves both through module slots that governance swaps with a 24 hour grace and no new app build. After a swap the app keeps reading the retired modules.

```
// web/app/src/chain/market.ts:103-106
async function readMarketState(nowSec: number): Promise<MarketState> {
  const cal = OPTIONAL_ADDRESSES.marketCalendar;
  if (!cal) throw new Error('MarketCalendar address not configured');
  const v = await readOnce<MarketTuple>(cal, marketCalendarAbi, 'marketState', [BigInt(nowSec)]);
```

```
// web/app/src/chain/market.ts:129-133
async function readReferencePrice(feed: Hex): Promise<ReferencePrice> {
  const adapter = OPTIONAL_ADDRESSES.oracleAdapter;
  if (!adapter) return { ok: false, errorName: null };
  try {
    const v = await readOnce<RefTuple>(adapter, oracleAdapterAbi, 'referencePrice', [feed]);
```

getGovernance labels the bundled NoteCore with the registry's active version and never reads activeCore() or versionOf(core), so after a core replacement the Modules card shows "NoteCore v2" next to the v1 address the app still writes to.

```
// web/app/src/data/ChainProvider.ts:746
{ key: `NoteCore v${num(activeV, 1)}`, address: core, kind: 'registry', version: `v${num(activeV,
  ↪ 1)}`, pending: null },
```

Impact: No signed value or button state depends on these reads: the SHIELD prefund comes from NoteCore.requiredPrefund and deadlines from NoteCore.effectiveObservationTs. After a module swap, the Series page reference price, levels and "Notional covered" estimate, and the market status pill, can differ from what NoteCore uses. After a core replacement, deposits to the deprecated core still succeed and settle, so no funds are at risk.

Proof of Concept:

1. Fork the testnet with anvil. Through the Timelock (delay 0 on testnet, deployer is proposer), call NoteCore.setOracle(A2) with a freshly deployed OracleAdapter(owner, guardian) and NoteCore.setCalendar(C2) with a new MarketCalendar that marks Tuesday 2026-09-29 a holiday. Then call SeriesRegistry.register(newCore) and deprecate(1), and move past the 24 hour grace.
2. On Tuesday at 15:30 ET open #/notes: the pill reads "Market open" while NoteCore.calendar() reports the day closed.
3. Push an NVDA close round of 230.00 at 15:58 ET and an after-hours round of 234.60 at 16:40 ET, then open #/series/48 at 18:00 ET. "Reference (indicative)" shows 230.00, while NoteCore.oracle() returns 234.60 and NoteCore prices the SHIELD prefund (latestPrice) from that adapter.
4. Open #/governance: the Modules card shows "NoteCore v2" for 0xE47B...a55e, whose versionOf is 1.

Recommended Mitigation: Resolve the adapter and calendar from NoteCore.oracle() and NoteCore.calendar(), label the core row with versionOf(core), and warn on the Governance page when activeCore() is not the bundled core. A failed pointer read falls back to the existing "unavailable" states.

1. In web/app/src/chain/market.ts, add a cached pointer read. Add function calendar() view returns (address) to noteCoreAbi.

```
const MODULE_TTL_MS = 5 * 60_000;
let modules: { at: number; p: Promise<{ oracle: Hex | undefined; calendar: Hex | undefined }> } | null
  ↪ = null;
export function protocolModules(): Promise<{ oracle: Hex | undefined; calendar: Hex | undefined }> {
  const core = ADDRESSES.noteCore;
  if (!core) return Promise.resolve({ oracle: OPTIONAL_ADDRESSES.oracleAdapter, calendar:
    ↪ OPTIONAL_ADDRESSES.marketCalendar });
  if (modules && Date.now() - modules.at < MODULE_TTL_MS) return modules.p;
  const p = Promise.all([
    readOnce<Hex>(core, noteCoreAbi, 'oracle'),
    readOnce<Hex>(core, noteCoreAbi, 'calendar'),
  ]).then(([oracle, calendar]) => ({ oracle, calendar }));
  modules = { at: Date.now(), p };
  p.catch(() => { if (modules?.p === p) modules = null; });
  return p;
}
```

readMarketState then reads (await protocolModules()).calendar, and readReferencePrice reads (await protocolModules()).oracle inside its existing try.

2. In getGovernance (web/app/src/data/ChainProvider.ts), add versionOf(core) and activeCore() to the multicall as coreV and activeCore. Label the core with its own version and return both in registry as coreVersion and activeCore (extend the type in types.ts and the Preview mock to match):

```
- { key: `NoteCore v${num(activeV, 1)}`, address: core, kind: 'registry', version: `v${num(activeV,
  ↪ 1)}`, pending: null },
```

```
+ { key: `NoteCore v${num(coreV, 1)}`, address: core, kind: 'registry', version: `v${num(coreV, 1)}`, pending: null },
```

3. In the Modules card of `web/app/src/routes/Governance.tsx`, show a notice when the registry's active core is not the bundled one:

```
{g.registry.activeCore && g.registry.activeCore.toLowerCase() !== (g.modules.find((m) => m.kind ===
  'registry')?.address ?? '').toLowerCase() && (
  <p className="card__body down" role="alert" style={{ fontSize: 13 }}>This build writes to NoteCore
    v{g.registry.coreVersion}; the registry's active core is v{g.registry.activeVersion} (<Addr
    a={g.registry.activeCore} explorer={explorer} />). Series on v{g.registry.coreVersion} still
    settle there; new series are not shown until the app is updated.</p>
  )}
```

Note Systems: Fixed in commit [6c7ece3](#).

Cyfrin: Verified.

7.3.10 CSP connect-src omits <https://www.walletlink.org>, so the Coinbase Wallet SDK cannot fetch WalletLink responses missed while its socket was down

Description: `cspDirectives` allows the WalletLink socket `wss://www.walletlink.org` but not the HTTPS origin of the same relay, so the browser blocks the Coinbase Wallet SDK's GET `https://www.walletlink.org/events?unseen=true` call. The SDK uses that call after every reconnect to recover wallet responses the relay stored while the socket was down. When it fails, the SDK only logs the error and the response is lost.

```
// web/app/vite.config.ts:75
const cbConnect = ['https://keys.coinbase.com', 'https://rpc.wallet.coinbase.com',
  'https://cca-lite.coinbase.com', 'https://as.coinbase.com', 'wss://www.walletlink.org'];
```

The app offers WalletLink because the connector uses preference: 'all'. The same list also allows `https://cca-lite.coinbase.com` and `https://as.coinbase.com`, whose only client is an inline SDK telemetry script that script-src 'self' already blocks.

Impact: A user who connects the Coinbase Wallet mobile app through WalletLink loses any response sent while the socket is down, so the request never resolves and a retry signs the same action twice. Today only chain switch, add chain and watch asset requests reach the relay, since WalletLink signing fails earlier on a missing Buffer global.

Proof of Concept:

1. Serve the testnet build with its CSP header, open `#/notes` and click "Connect wallet", then "Coinbase Wallet".
2. The SDK opens `wss://www.walletlink.org/rpc` for the QR option.
3. DevTools shows Refused to connect because it violates the document's Content Security Policy for `https://www.walletlink.org/events?unseen=true`.

Recommended Mitigation: Add the HTTPS origin of the WalletLink relay to `cbConnect` and drop the two telemetry hosts.

```
- const cbConnect = ['https://keys.coinbase.com', 'https://rpc.wallet.coinbase.com',
  'https://cca-lite.coinbase.com', 'https://as.coinbase.com', 'wss://www.walletlink.org'];
+ const cbConnect = ['https://keys.coinbase.com', 'https://rpc.wallet.coinbase.com',
  'wss://www.walletlink.org', 'https://www.walletlink.org'];
```

Note Systems: Fixed in commit [f14e95d](#).

Cyfrin: Verified.

7.3.11 Six UI strings and figures contradict what the deployed contracts and the wallet SDK do

Description: Six strings and figures on the testnet build are hard-coded or computed from the wrong value, so they describe behaviour the deployed contracts or the bundled wallet SDK do not have:

- The Stake and Desk "30 d" deltas and history cards always read "+0.00%" and "X -> X", because Chain-Provider returns a single history point.
- The Stake cooldown reads "then 48 h window", but sNOTE requests stay redeemable at any time after the cooldown.
- The paused Desk says deposits and exits resume "once the guardian unpauses it", but the pause blocks deposits only and only the Timelock can unpause.
- The Buyback page says "Bought NOTE is burned", but RevenueRouter streams it to sNOTE stakers over 7 days.
- The strike crank card offers "up to 0.00 USDG", because it reads `accruedFees` before the strike books any fee; the caller is paid up to `keeperRewardQuote`.
- The connect modal says "Nothing is stored in your browser", but the Coinbase Wallet SDK keeps its session id and secret in `localStorage` and `IndexedDB`.

```
// web/app/src/routes/Desk.tsx:360
const depositBlockedCopy = paused ? 'The Desk is paused. Deposits and exits resume once the guardian
↳ unpauses it.' : 'Deposits are closed while a held stock has no valid price. They reopen once the
↳ stock is priced again.';
```

```
// web/app/src/components/Automation.tsx:61
const reward = `${fmtUsd(data.keeperReward)} USDG`;
```

Impact: No funds are at risk and no transaction changes. During a Desk pause a holder may skip a withdrawal that would succeed, and a strike crank caller is told it earns nothing when it earns up to 2.00 USDG on testnet.

Proof of Concept:

1. On an anvil fork of the testnet, pause the Desk from the guardian and open `#/desk`. The deposit card says exits are paused while the Withdraw tab still offers "Review withdrawal", and the NAV card reads "0.9610 -> 0.9610".
2. Open `#/buyback`: the sell card says "Bought NOTE is burned."
3. Open `#/series/24`, whose strike is due, and run the crank. The card and toast say "up to 0.00 USDG", while the receipt carries `KeeperPaid(24, caller, 2000000)`.
4. Pick Coinbase Wallet in the connect modal, close the popup and reload. `localStorage` still holds the walletlink session keys and `IndexedDB` lists `cbwsdk`.

Recommended Mitigation: Hide the 30 d delta and history card while there is only one point, show `keeperRewardQuote` for a strike, and correct the cooldown, pause, buyback and connect modal copy.

```
const nav30 = navHist.length > 1 ? (ratio(nav, navHist[0], 1) - 1) * 100 : null;
// ...
const shown = strike ? data.keeperRewardQuote : data.keeperReward;
```

Note Systems: Fixed in commit [a108674](#).

Cyfrin: Verified.

7.3.12 CI app job runs the ABI check without compiled contracts, so the check passes without comparing anything

Description: The CI app job runs the ABI check without compiled contracts, and the check exits 0 when they are missing. `qa/abi-check.mjs` compares the app's ABIs in `src/chain/*Abi.ts` with the artifacts in `contracts/out`, and skips unless `ABI_CHECK_REQUIRED` is set:

```
// web/app/qa/abi-check.mjs:9-13
const outDir = resolve(process.argv[2] ?? join(root, '..', '..', 'contracts', 'out'));
if (!existsSync(outDir)) {
  console.log(`abi-check: no compiled artifacts at ${outDir} (run forge build in contracts/ first);
  ↳ nothing checked`);
  process.exit(process.env.ABI_CHECK_REQUIRED ? 1 : 0);
}
```

`contracts/out/` is git-ignored. The app job builds no contracts, sets no `ABI_CHECK_REQUIRED` and does not take the contracts job's output, so `npm test` takes the early exit on every pipeline:

```
# .gitlab-ci.yml:82-90
app:
  stage: app
  image: node:22
  script:
    - cd web/app
    - npm ci
    - npm run typecheck
    - npm test
    - VITE_NETWORK=mainnet npm run build
```

Impact: The ABIs match today, so no user is affected. If a contract change alters a function signature and the app's ABI is not updated, the app job still passes and the mismatch shows up only at runtime, as failed reads or wrongly decoded values.

Proof of Concept:

1. In a clean checkout, run `npm test` in `web/app`. The ABI step prints "abi-check: no compiled artifacts ... nothing checked" and the suite exits 0.
2. In `web/app/src/chain/protocolAbi.ts`, change 'function `seriesCount()` view returns (uint256)' to 'function `seriesCount()` view returns (uint128)' and run `npm test` again. It still exits 0.
3. Run `forge build` in `contracts/`, then `node qa/abi-check.mjs` in `web/app`. It reports the output mismatch on `seriesCount()` and exits 1.

Recommended Mitigation: Compile the contracts in the app job before `npm test` and make the ABI check mandatory there, installing Foundry the same way the contracts jobs do:

```
app:
  stage: app
  image: node:22
  variables:
    ABI_CHECK_REQUIRED: "1"
  before_script:
    - !reference [.foundry-install, before_script]
  script:
    - (cd contracts && forge build)
    - cd web/app
    - npm ci
    - npm run typecheck
    - npm test
    - VITE_NETWORK=mainnet npm run build
```

Alternatively, publish `contracts/out/` as an artifact of the `contracts` job, take it in app with needs, and run `contracts` on `web/app/**` changes too.

Note Systems: Fixed in commit [c3468f6](#).

Cyfrin: Verified.

7.3.13 App hard-codes 6 decimals for the quote token instead of reading `NoteCore.quoteDecimals`

Description: The app hard-codes 6 decimals for the quote token wherever it converts between user input, base units and display values, while `NoteCore` reads the token's decimals at construction, stores them in `quoteDecimals` and scales every quote amount with that value. The app scales by neither value:

```
// web/app/src/routes/Series.tsx:235
tx.send('COUPON deposit', () => depositCouponSteps(owner, s.id, toUnits(amt, 6)), {
  ↳ successBody: `${usd(n)} USDG deposited into series ${s.id}.` }).then((r) => { if (r)
  ↳ setAmt(''); });
```

```
// web/app/src/data/ChainProvider.ts:38-39
const USDG_DECIMALS = 6;
const ONE_USDG = 10n ** BigInt(USDG_DECIMALS);
```

The same constant is used for Desk deposits (`Desk.tsx`), the NOTE sale quote (`actions.ts`), portfolio values (`lib/portfolio.ts`) and `fmtUsd` (`lib/format.ts`). `noteCoreAbi` does not declare `quoteDecimals()`.

Impact: Both values are 6 today, so no amount is wrong on the current deployments. If `NoteCore` is deployed against a quote token with other decimals, every deposit, approval, balance check and USDG figure in the app is off by a power of ten; with 18 decimals a 50 USDG deposit is sent as 5e-11 tokens and reverts with `BelowMinTicket`.

Proof of Concept:

1. Call `quoteDecimals()` on the testnet `NoteCore` and `decimals()` on the testnet mock USDG and on mainnet USDG. All three return 6.
2. Search `web/app/src` for `quoteDecimals`. The only match is a comment in `lib/portfolio.ts`, and `noteCoreAbi` has no such entry.
3. On a testnet build, open `#/series/63`, enter 50 in the COUPON card and confirm. The wallet asks for approve(`NoteCore`, 50000000), built as `toUnits("50", 6)` whatever the token reports.

Recommended Mitigation: Read `NoteCore.quoteDecimals()` and the token's `decimals()` when the provider loads and disable the deposit forms if either differs from 6. Scaling from the read value is the longer-term fix.

In `web/app/src/chain/protocolAbi.ts`, add the getter to `noteCoreAbi`:

```
'function quoteDecimals() view returns (uint8)',
```

In `web/app/src/data/ChainProvider.ts`, check both values:

```
async quoteDecimalsOk(): Promise<boolean> {
  const [core, token] = await Promise.all([
    readContract(this.client, { address: ADDRESSES.noteCore!, abi: noteCoreAbi, functionName:
      ↳ 'quoteDecimals' }),
    readContract(this.client, { address: ADDRESSES.usdg!, abi: erc20Abi, functionName: 'decimals' }),
  ]);
  return Number(core) === USDG_DECIMALS && Number(token) === USDG_DECIMALS;
}
```

Note Systems: Fixed in commit [27f1ab1](#).

Cyfrin: Verified.

7.3.14 Access gate covers only /app/, so the analytics page stays public when the gate is switched on

Description: The access gate is scoped to /app/ at every layer, so the analytics page at /analytics/ is never gated. The server rewrite rule that the host's assembly step writes into dist/.htaccess matches only requests under /app/, and the gate page sets its cookie with Path=/app:

```
// web/app/public/gate/gate.js:58-62
function setAccessCookie(token) {
  var parts = [COOKIE + '=' + token, 'Path=/app', 'Max-Age=2592000', 'SameSite=Lax'];
  if (location.protocol === 'https:') parts.push('Secure');
  document.cookie = parts.join('; ');
}
```

The client-side check is injected only into app/index.html (web/app/vite.config.ts). The analytics page is a separate bundle built by vite.analytics.config.ts, which has no gate plugin, and it reads the same protocol data as the app.

web/shared/gate/enabled.txt is off today, so neither path is gated.

Impact: When the owner switches the gate on, anyone without the code can still open /analytics/ and see the series book, Desk NAV, treasury balances and governance state the gate hides behind /app/. The figures are public on chain, so no private data is exposed.

Proof of Concept:

1. Set web/shared/gate/enabled.txt to on and build both bundles with the host's assembly step.
2. In a fresh browser profile with no ns_access cookie, open /app/. The gate page is served.
3. In the same profile, open /analytics/. The analytics page loads and shows live protocol figures without asking for the code.

Recommended Mitigation: Match /analytics/ as well as /app/ in the rewrite rule that assemble.sh writes, and set the cookie with Path=/ so the browser sends it to both paths:

```
var parts = [COOKIE + '=' + token, 'Path=/', 'Max-Age=2592000', 'SameSite=Lax'];
```

Update the cookie line in web/shared/gate/README.md to match.

Note Systems: Fixed in commit [02a54db](#). States the host's server-side rewrite rule, outside this repository, now matches all four gated surfaces too, shipping at the next assembly.